

Elektrotehnički fakultet
Univerzitet u Banjoj Luci

Mihajlo Savić

Arhitektura računara

Skripta za generaciju 2025/2026

Banja Luka 2025.

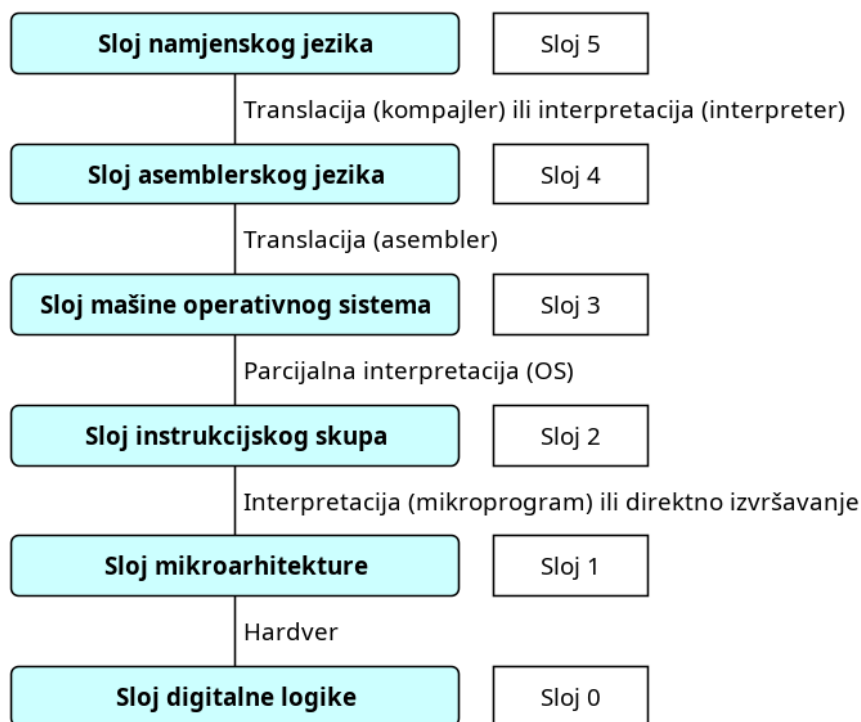
Sadržaj

1. Višeslojna organizacija računara.....	3
2. Kratka istorija, generacije i redovi veličine.....	6
3. Procesor - organizacija.....	11
4. Procesor - izvršavanje instrukcija.....	14
5. Procesor - CISC i RISC.....	15
6. Multiprocesori.....	16
7. Višejezgreni procesori i HyperThreading.....	18
8. Multiračunari.....	20
9. Masovno multiprocesorske mašine.....	22
10. Pipelining.....	23
11. Superskalarnost.....	25
12. SIMD i vektorski procesori.....	26
13. Izvršavanje instrukcija van redoslijeda.....	28
14. Arhitektura jezgra modernog procesora.....	30
15. Primarna memorija.....	32
16. Keš memorija.....	33
17. Statička i dinamička memorija (SRAM i DRAM).....	35
18. DRAM moduli.....	36
19. Hijerahijska organizacija memorije.....	38
20. Magnetna memorija.....	40
21. Solid State Drive - SSD.....	42
22. Trajna radna memorija - Non-volatile RAM.....	45
23. Simulatori i emulatori.....	46
24. Virtuelizacija.....	47
25. x86 i virtuelizacija.....	48
26. Tipovi VMM.....	50
27. XEN, KVM i paravirtuelizacija.....	51
28. Virtuelizacija na nivou operativnog sistema.....	52
29. Matična ploča.....	53
30. Magistrale.....	55
31. PCI i PCI Express.....	56
32. USB.....	58
33. Načini povezivanja sekundarne memorije.....	60
34. RAID.....	62
35. Telekomunikacioni uređaji.....	65
36. Ulazni i izlazni uređaji.....	68
37. Audio i video uređaji.....	70
38. Grafičke kartice.....	74
39. Pregled ISA.....	76
40. Intel procesori.....	78
41. AMD procesori.....	80
42. ARM procesori.....	81

1. Višeslojna organizacija računara

Od ranije vam je poznato da način rada koji odgovara ljudima ne odgovara i računarima. Mi težimo simbolima i apstraktnom, računari brojevima i logici. Da bi omogućili da svi budu srećni moramo pronaći način da premostimo taj jaz. Jedan od klasičnih načina je da definišemo jezike za jednu i drugu stranu, a nakon toga omogućimo komunikaciju između njih. Ljudi će koristiti njima prihvatljive simbole, a računari mašinski kod. Sam proces generisanja mašinskog koda može biti obavljen unaprijed (**translacija** - kao u slučaju kompajlera/prevodilaca) ili korak po korak za svaku instrukciju (**interpretacija** - kao u slučaju skript jezika). Iako postoje bitne razlike između ova dva pristupa, mnogo više je sličnosti. Osnovna razlika sa stanovišta ovog predmeta leži u činjenici ko kontroliše izvršavanja. Kod translacije, nakon generisanja mašinskog koda, izvorni program je nepotreban i mašinski kod preuzima kontrolu. Kod interpretacije izvorni kod predstavlja samo podatke za interpreter koji kontroliše izvršavanje. U modernim okruženjima je najčešće u upotrebi hibridni pristup koji koristi obe tehnike u zavisnosti od potrebe jer je i prevođenje moguće bolje izvesti ako imamo pristup i originalnom kodu u toku izvršavanja.

Da bi sve navedeno funkcionisalo na razuman način, potrebno je da postoji neka sličnost između jezika. Prevedeno u praktični svijet, to znači da ćemo rijetko imati na raspolaganju direktan prelaz sa namjenskih jezika visokog nivoa na najniži nivo mašinskog koda. To dovodi do pojave više međujezika koje možemo posmatrati kao slojeve u organizaciji. Inženjeri vole slojeve jer pravilno raslojen sistem omogućava da ignorišemo sve osim slojeva koji nas interesuju, na primjer, veb aplikaciju apsolutno ne interesuje koje napone generiše mrežna kartica na kojim pinovima i na koji način. Na taj način sve što nije od direktnog i trenutnog interesa za nas guramo u "polje tuđeg problema".



Slika 1. Primjer slojevite organizacije

Na slici 1 je prikazan primjer slojevite organizacije. Najniži sloj, nulti, je **sloj digitalne logike**. Sastoji se od kola koja omogućavaju rad sa jednostavnim logičkim operacijama kao što su: AND, OR, NOT, XOR, NAND, NOR, itd. Naravno, svaki put kada nečemu dodijelimo status "nultog" nivoa ispostavi se da i ispod njega postoje dodatni nivoi. U ovom slučaju se radi o nivoima na kojima baratamo individualnim poluprovodničkim komponentama kao što su tranzistori i gejtovi, pa čak i nižim nivoima koji uzimaju u obzir fizičke fenomene koji za nas, srećom, nisu bitni. Kako se bavimo računarstvom i volimo slojevitost, imamo luksuz da ignorišemo sve probleme koji postoje ispod nultog nivoa i bavljenje njima ostavimo elektroničarima i fizičarima. Pogodnim spajanjem pojedinačnih komponenti možemo da pravimo prvo jednobitnu memoriju, zatim da spajamo više (npr. 64) takvih ćelija u grupe koje onda zovemo **registri**. Osim memorije, možemo da pravimo kola koja će izvršiti neku jednostavnu operaciju nad ulaznim podacima. Tako polako pravimo sve više logike koja će nam na kraju omogućiti da napravimo kompletan procesor i snimamo klipove mačaka koje sviraju klavir.

Naredni sloj je **sloj mikroarhitekture**. Na ovom sloju organizujemo registre i pravimo malo kompleksnije blokove za obradu podataka (sabirači, komparatori, itd) koje obično nazivamo **aritmetičko logička jedinica (ALU)** i na koju su povezani registri (jer joj služe i kao ulazni podaci i kao mjesto gdje može da smjesti rezultat operacije). Ovdje dolazimo do prve bitne podjele: da li se operacije izvršavaju direktno pod kontrolom **hardvera** ili pod kontrolom **mikroprograma**. U slučaju mikroprograma samo izvršavanje je interpretacija instrukcija i vrijedi sve što je ranije rečeno po tom pitanju. Kontrola hardvera omogućava bolje teoretske performanse, ali i mnogo manju slobodu realizaciji koja se na kraju manifestuje i boljim stvarnim performansama. I u ovom slučaju se danas koriste hibridna rješenja koja kombinuju najbolje od oba svijeta. Vrijedi napomenuti da mi kao programeri nemamo pristup ovom sloju (osim ako ne radimo za Intel ili AMD i naš posao je da pišemo mikrokod).

Naredni sloj je prvi kojem možemo da imamo pristup i on definiše arhitekturu skupa instrukcija ili **instruction set architecture (ISA)**. Na ovom nivou su definisane sve instrukcije koje možemo zadati procesoru, svi registri koje vidimo, svi tokovi i načini rada koji su nam dostupni. Kada u normalnom govoru koristimo izraz "mašinski kod" obično mislimo na kod koji je pogodan za izvršavanje na ovom nivou i sastoji se od instrukcija koje čine ISA. Važno je napomenuti da je vrlo moguće da određene instrukcije postoje u neizmijenjenom obliku na više nivoa (INC je INC, nema tu mnogo filozofije).

Naredni nivo je hibridni nivo koji koristi jezik nastao kombinovanjem ISA jezika i dodatnih instrukcija koje nam omogućavaju lakši rad sa memorijom, paralelizmom ili nečim trećim. Ovaj sloj je obično pod kontrolom **operativnog sistema** (zavisi, vidjećemo kasnije kada nije). U klasičnim podjelama je ovo **sloj sistemskog softvera** i sadrži i kompajlere i interpretere potrebne za rad na višim slojevima.

Sada prelazimo na nivoe koje i obični smrtnici programeri koriste u svom svakodnevnom poslu. Na četvrtom nivou nas čeka **asembler**ski jezik koji je malo više od ljepše zapakovanog nižeg nivoa sa manje ružnim simbolima. Omogućava nam da pišemo kod koji je lako prevesti na niže nivoe i koji nam daje više kontrole. S druge strane, moderni kompajleri i interpreteri su u stanju da optimizuju kod na način koji je van dohvata ljudi. Dobar inženjer zna kada koristiti koje rješenje.

Na petom sloju se nalaze **namjenski programski jezici** sa kojima ste već navikli da radite, od programskog jezika C koji kombinuje fleksibilnost asemblera sa elegantnošću asemblera, preko

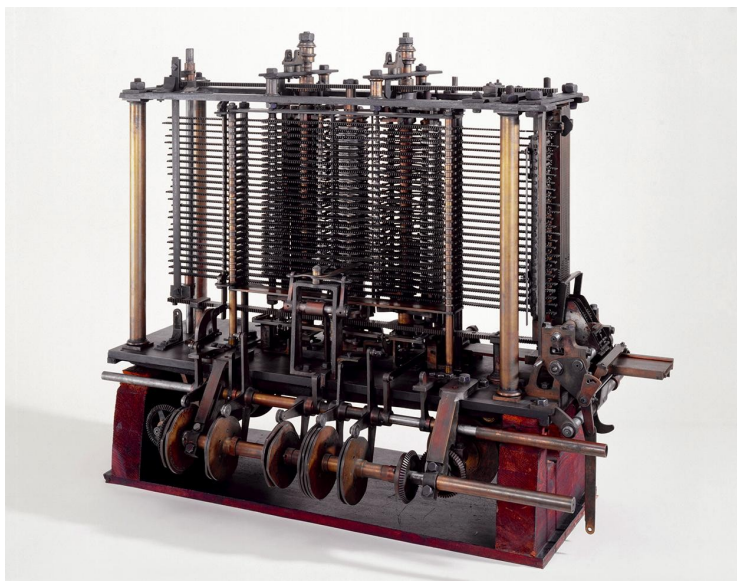
Java do JavaScripta, ujedno i najružnijeg i najkorisnijeg programskog jezika na svijetu. Iako su dugo ovi slojevi bili **prevođeni** prije izvršavanja, danas je prisutan trend primjene hibridnih rješenja. Primjeri su brojni, od **just-in-time (JIT)** prevođenja Java bajtkoda u mašinski kod do **just-in-time (JIT)** prevođenja JavaScript izvornog koda u mašinski kod. Drugim riječima, primjer su savremeni JIT interpreteri koji na osnovu praćenja načina izvršavanja koda mogu da na optimalan način balansiraju potrebu da neki dio koda interpretiraju (sporije izvršavanje, ali bez čekanja na prvu instrukciju) ili prevedu (brže izvršavanje, ali čekamo da se završi prevođenje). Očigledno je da prvi pristup biramo za kod koji se izvršava jednom, a drugi za petlje i slične višestruke upotrebe istog koda. Čak je moguće na osnovu očekivanog broja izvršavanja odabrati i nivo optimizacije pri prevođenju, ali i keširati određena prevođenja kako bi uštedjeli na vremenu po cijenu većeg zauzeća radne memorije. Nije na odmet napomenuti i da na ovako visokom nivou imamo i luksuz da interpreter može da presloži naš kod na takav način da rezultat ostane isti, ali da se pojedinačne instrukcije ne izvršavaju originalnim redoslijedom, može da nam razmota petlju da bi je bolje optimizovao, da unaprijed pogodi da li na kraju petlje ide još jedna iteracija ili se iskače iz iste, itd. To je ljepota softvera - interpreter kontroliše izvršavanje, naš kod su samo podaci.

Sve što je upravo ispričano za jezike višeg nivoa i interpretiranje i hibridna rješenja vjerovatno zvuči poznato, ali to je samo zato što je ispričano i na prethodnoj strani. Moderni procesori koriste sve trikove optimizujućih JIT interpretera, samo spuštenu par slojeva niže i naš "mašinski" kod u stvarnosti posmatraju više kao listu lijepih želja koje će oni isjeckati, presložiti i skuhati na način koji će što je bolje moguće iskoristiti sve resurse procesora koji ni mi, ni operativni sistem ni kompajler nismo svjesni. Kao što smo ranije napomenuli, inženjeri vole šablone i ponovnu upotrebu dobrih rješenja.

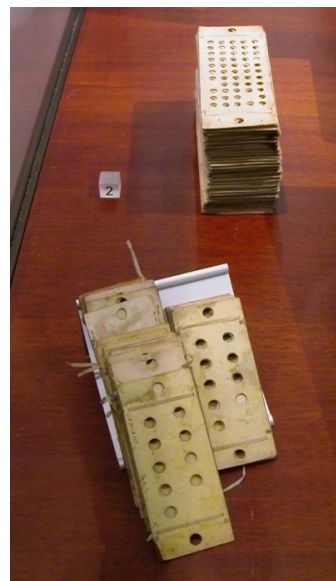
2. Kratka istorija, generacije i redovi veličine

Iako ste već vjerovatno upoznati sa osnovama istorije razvoja računara, nije na odmet ponoviti gradivo i organizovati na način pogodan za polaganje ispita.

Razvoj računara je obično predstavljen kao prolazak kroz porodično stablo pri čemu svaki bitan tehnološki proboj izaziva pojavu nove generacije. Posmatrano na ovaj način, razvoj počinje **nultom generacijom** računara koji su bili mehaničke prirode i više su predstavljali rješenja frustracije brilijantnih umova koji su željeli uštedjeti vrijeme potrebno za ručno izvršavanje matematičkih operacija. Počevši od Blaise Pascala koji je 1642. godine (sa 19 godina) prvi mehanički kalkulator da bi pomogao ocu, porezniku, da lakše radi svoj posao i sabira i oduzima automatski. Na sličan način je i Gottfried Wilhelm von Leibniz napravio prvi uređaj koji je mogao obavljati četiri osnovne matematičke operacije. Situacija je bila mirna duže od jednog vijeka do Charlesa Babbagea koji je smislio sistem koji se sastojao od komponenata za čuvanje podataka, računanje, ulaz sa kog je čitao komande i izlaz (zvuči poznato?) i nazvao ga **analytical engine**¹. Kako je sad trebalo i smisliti neki način jednostavnijeg programiranja, zaposlio je Adu Lovelace koja je postala prvi programer na svijetu. Nažalost, mehaničke komponente su onemogućavale razvoj punog potencijala ideje.



Slika 2: Analytical engine



Slika 3: Bušene kartice

John Atanasoff je kreirao prvu mašinu koja je koristila binarnu aritmetiku, ali i kondenzatore kao memoriju (što znači da morate da ih periodično osvježavate - praktično je izmislio DRAM memoriju, vidi poglavlje 17 za detalje). I ovdje dolazimo do klasične sudbine tipične za inženjerstvo: Atanasoff je napravio superioran sistem koji nije u potpunosti radio, dok je George Stibbitz napravio primitivnije rješenje koje su ljudi mogli koristiti što je inspirisalo Johna Mauchleya (kreatora ENIACa) što nas na lijep način uvodi u **prvu generaciju** računara. Ako zanemarimo engleski COLOSSUS² (u razvoju učestvovao i Alan Turing, korišćen je za razbijanje ENIGMA šifrovanja) koji je bio vojna tajna i nije uticao na razvoj računarstva na bitan način, najvažniji je ENIAC³ koji je inspirisao veliki broj istraživača da počnu da se bave razvojem

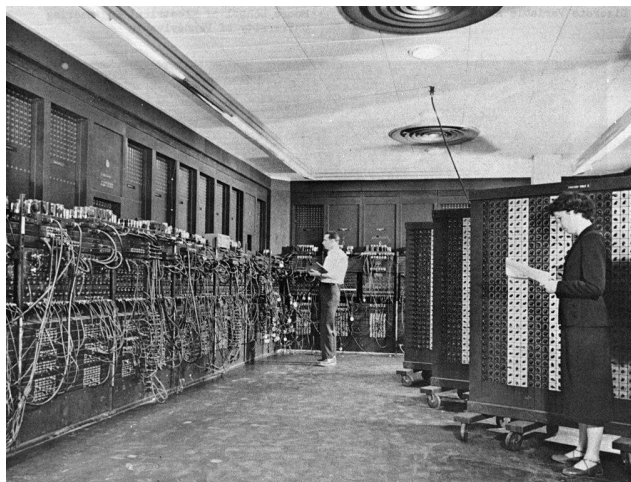
1 [https://en.wikipedia.org/wiki/File:Babbages_Analytical_Engine,_1834-1871._\(9660574685\).jpg](https://en.wikipedia.org/wiki/File:Babbages_Analytical_Engine,_1834-1871._(9660574685).jpg)

<https://en.wikipedia.org/wiki/File:PunchedCardsAnalyticalEngine.jpg>

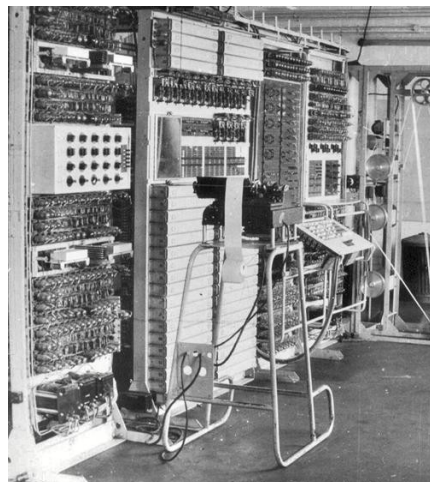
2 https://en.wikipedia.org/wiki/File:Wartime_photo_of_Colossus_10.png

3 <https://en.wikipedia.org/wiki/File:Eniac.jpg>

računara (jer su sada vidjeli da je nešto takvo i moguće). **Vrlo važno:** Mauchley i Eckhart su pokušali da patentiraju digitalni računar, ali je sud odlučio da ga je izmislio Atanasoff i da nije patentiran što je značilo da sada svi mogu slobodno da prave svoje računare. Alternativa je teško zamisliva. Jedan od zainteresovanih naučnika bio je i John von Neumann. Originalna ideja Von Neumanna je da je upotreba brojnih prekidača i kablova sa ciljem programiranja računara (ENIAC) fundamentalno pogrešan pristup, kao i da je upotrebu serijske decimalne predstave moguće zamijeniti efikasnijom binarnom (izvorna ideja je Johna Atanasoffa). To je dovelo do pojave Von Neumannove organizacije računara (vidi poglavlje 3).



Slika 4: ENIAC



Slika 5: COLOSSUS

Druga generacija računara je zasnovana na upotrebi tranzistora (pronalazači: John Bardeen, Walter Bratain i William Shockley) kao osnovnog elementa pri čemu je prvi računar takve vrste bio TX-0 a jedan od tvoraca je oformio kompaniju Digital Equipment Corporation (DEC) koja je napravila PDP-1, prvi "pristupačan" računar (stotine hiljada umjesto miliona dolara). Još jedan izvor inspiracije, naslijeđen PDP-8 računarom (gotovo deset puta jeftiniji, prodan u preko 50 000 primjeraka) koji je vrlo bitan jer je prvi računar koji je koristio **magistralu**.



Slika 6: CDC 6600



Slika 7: CDC 6600 - Šematski prikaz

1964. godine, mala kompanija pod imenom Control Data Corporation (CDC) je napravila računar CDC 6600⁴ koji je naprosto pomeo ostale računare po pitanju performansi - jer je imao više nezavisnih jedinica i mogao je da interno **paralelno izvršava** zadatke, ali i pomoćne procesore koji su omogućavali da centralni procesor posveti svoje vrijeme i resurse bitnim stvarima. Dizajner ovog

4 https://en.wikipedia.org/wiki/CDC_6600

revolucionarnog rješenja je bio Seymour Cray, često nazivan bog super-računarstva (jer je prvi pravio superračunare).

Razvojem integrisanih elektronskih kola (Jack Kilby i Robert Noyce) je omogućilo da spakujemo više tranzistora na isti čip. IBM je razvio liniju proizvoda pod nazivom System/360 koja se sastojala od više modela koji su bili međusobno kompatibilni (mogli su izvršavati isti kod i radili su na istim principima) ali sa različitim performansama po različitim cijenama. Ovaj koncept je fundamentalno promijenio ekonomsku stranu računarstva, a samim tim i računarstvo u cjelini. Još jedan revolucionaran koncept je postojanje više programa u memoriji u isto vrijeme (multiprogramiranje). Kako ovo nije bilo dovoljno, System/360 je i prvi računar koji je mogao raditi kao emulator sa ciljem obezbjeđivanja kompatibilnosti sa starijim, izvorno nekompatibilnim, modelima. Ovo je omogućeno upotrebom mikroprograma (interpretera u procesoru). Na polju računara koje su i smrtnici mogli priuštiti, DEC je kreirao PDP-11⁵ (16-bitni nasljednik 8-bitnog PDP-8⁶) koji je postao izuzetno popularan u akademskim institucijama.



Slika 8: DEC PDP-8



Slika 9: DEC PDP-11

Pojavom **Very Large Scale Integration (VLSI)** je omogućeno da na isti čip sada stavimo zaista veliki broj tranzistora što je dovelo do obaranja cijena, poboljšanja performansi i povećanja resursa. Samim tim, računari su postali pristupačni prosječnim ljudima, a ne samo institucijama, što je dovelo do apsolutne eksplozije popularnosti računara, ali i fundamentalno novih načina da se oni koriste. Na primjer, niko nije pravio igrice za računare kada ih je bilo stotine i kada su koštali milione dolara. S druge strane, ako računari koštaju stotine, a ima ih na milione, pravljenje programa za razonodu postaje ozbiljan posao. Ovo je doba nastanka Intelovih 8080, 8086 i 8088 procesora, prodaje računara u dijelovima, ali i pojave Apple računara, a zatim, kao reakcije, i IBM PC računara. U nevjerovatnom potezu, IBM je objavio kompletne planove za IBM PC u knjizi i to na način da bilo ko može da kupi knjigu i pravi dodatne kartice za IBM PC. Ovo je dovelo do pojave **klonova** (računara koji su bili jednostavna reimplementacija originala) ali i naprednijih računara koji su sada svi bili kompatibilni. Ovaj potez je doveo do pojave računarskog tržišta kakvog ga danas poznajemo. Apple Lisa⁷ je propao na tržištu, ali je upoznao mase sa pojmom **grafičkog korisničkog interfejsa (GUI)**. Malo manje loše je prošao i prvi "prenosni" računar Osborne-1⁸. Svi promašaji su služili i kao marker drugima šta je moguće i kuda možemo ići. Intel je nastavio da razvija procesore (80286, 80386, 80486, itd) koji su postali poznati kao **x86**

5 <https://en.wikipedia.org/wiki/File:Pdp-11-40.jpg>

6 [https://en.wikipedia.org/wiki/File:PDP-8_\(1\).jpg](https://en.wikipedia.org/wiki/File:PDP-8_(1).jpg)

7 https://en.wikipedia.org/wiki/Apple_Lisa

8 https://en.wikipedia.org/wiki/Osborne_1

arhitektura a došlo je i do pojave proizvođača kompatibilnih procesora (AMD, Via, Cyrix, itd). Cijene su nastavile da padaju a računari da napreduju, IBM je predstavio prvi procesor sa dva jezgra (POWER4), procesori su postali dominantno 64-bitni. Gordon Moore, jedan od osnivača Intela, je formulisao Mooreov zakon po kojem se broj tranzistora uvećava dva puta svakih 18 mjeseci. I danas uživamo u blagodatima Mooreovog "zakona" koji nam daje sve više resursa po istoj cijeni.



Slika 10: Apple Lisa



Slika 11: Osborne 1

Peta generacija se više odnosi na sveprisutnost računara, a ne na neki ogroman napredak u performansama. Kako su veliki i brzi procesori postajali sve brži, tako su i mali i spori procesori napredovali i omogućili da računari postanu dio skoro svega čega mogu. Od malih, prenosnih uređaja poznatih kao **personal digital assistant (PDA)** uređaji, preko mobilnih i pametnih telefona (IBM Simon - prvi smartphone iz 1990. godine), do ogromnih superračunarskih klastera sastavljenih od stotina hiljada procesora, ali i do minijaturnih računara koji sviraju melodiju kad podignete šolju kafe.

Podjela prema upotrebi i redovima veličine

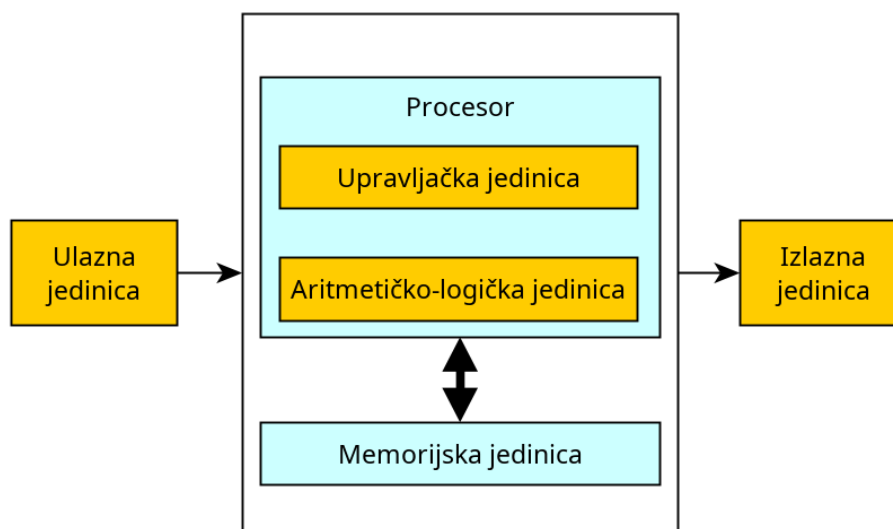
Između različitih kategorija postoje i razlike u redovima veličine i u cijeni i u performansama. Očigledno je da su namjene ovih fundamentalno različitih računara uzrokovale i potpuno drugačiji pristup razvoju datih sistema. Svaki pojedinačni superračunar koji košta približno stotinu miliona dolara je namjenski proizvod, pravljen u jednom primjerku, dok su najmanji procesori od desetog dijela dolara pravljeni serijski u stotinama miliona primjeraka. Na osnovu toga možemo napraviti grubu podjelu na: potrošne procesore, mikrokontrolere, mobilne procesore, procesore za personalne računare, servere, te procesore za superračunare. Iako je ova podjela elegantna, granice između pojedinih kategorija su sve tanje, a u nekim slučajevima i potpuno izbrisane. **Potrošni procesori** su u ranije pomenutim šoljama, čestitkama, platnim karticama, radio frequency identification (RFID) karticama i tagovima, itd. **Mikrokontroleri** su prisutni u mnoštvu uređaja, od časovnika, preko računarskih periferija, kućanskih uređaja i igračaka do vojne tehnike i oružja. Predstavljaju namjenske računarske sisteme prilagođene kontroli spoljnih aktuatora, prikupljanja podataka, itd, gotovo svi rade u realnom vremenu i podložni su značajnim fizičkim ograničenjima.

Kada govorimo o granicama između procesora koji se koriste u **mobilnim uređajima**, **personalnim računarima**, **serverima** i **superračunarima**, sve je teže pronaći granice. S jedne strane je velika većina, čak i najskupljih, superračunara sastavljena od komponenti koje su prisutne i u standardnim serverima ili jačim radnim stanicama (uz možda izuzetak specijalizovane mrežne infrastrukture). S druge strane je Apple lansirao seriju personalnih računara i laptop uređaja koji koriste snažnije verzije procesora koje koriste i u mobilnim telefonima (jer u računaru imaju na raspolaganju više energije i veći termalni budžet - koliko toplote mogu da ispuste u okolinu u jedinici vremena). Granice se ne brišu samo po pitanju procesora, vrhunski modeli mobilnih telefona imaju, na primjer, 12 GB ili više radne memorije i 256 GB ili više sekundarne memorije, što je potpuno uporedivo sa "srednjom" klasom laptopa. Savremene konzole za igre i jesu prilagođeni personalni računari i koriste čak i iste osnovne komponente. Na primjer, Sony Playstation 5 i Microsoft Xbox X su vrlo slični klasičnom personalnom računaru, oba koriste AMD Zen 2 APU sa 8 jezgara i grafičkim procesorom, imaju po 16 GB radne memorije, koriste NVMe SSD kao sekundarnu memoriju, imaju klasične mrežne kartice, itd. Gotovo da možete da napravite PC istih karakteristika.

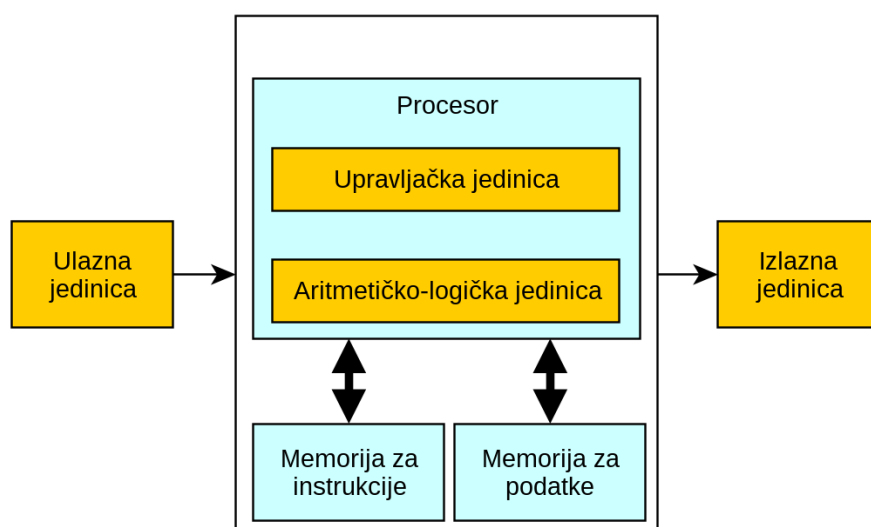
3. Procesor - organizacija

Originalna ideja Von Neumanna je da je upotreba brojnih prekidača i kablova sa ciljem programiranja računara fundamentalno pogrešan pristup, kao i da je upotrebu serijske decimalne predstave moguće zamijeniti efikasnijom binarnom (izvorna ideja je Johna Atanassoffa).

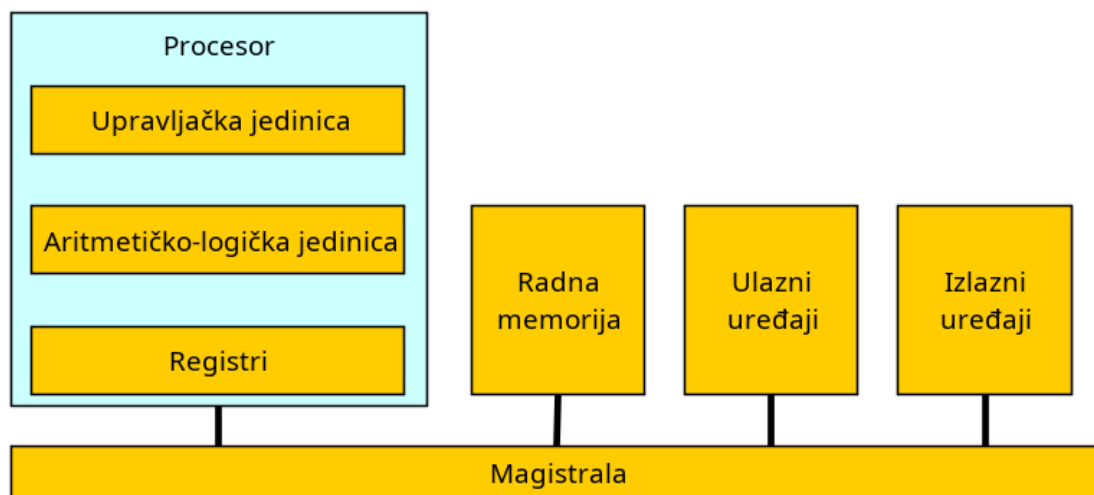
Osnova je računar koji se sastoji od upravljačke jedinice (CU), aritmetičko-logičke jedinice (ALU), zajedničke memorije (koja sadrži i podatke i program), te ulaznih i izlaznih jedinica koje pristupaju **akumulatoru** (dijelu ALU). Danas se CU i ALU grupišu u centralnu procesorsku jedinicu (CPU) i relaksiran je pristup ulaznih i izlaznih jedinica koje ne moraju da pristupaju akumulatoru (slika 12 za Von Neuman i 13 za Harvard), dok je pojednostavljena organizacija računara sa jednim procesorom prikazana na slici 14.



Slika 12. Von Neumann organizacija računara

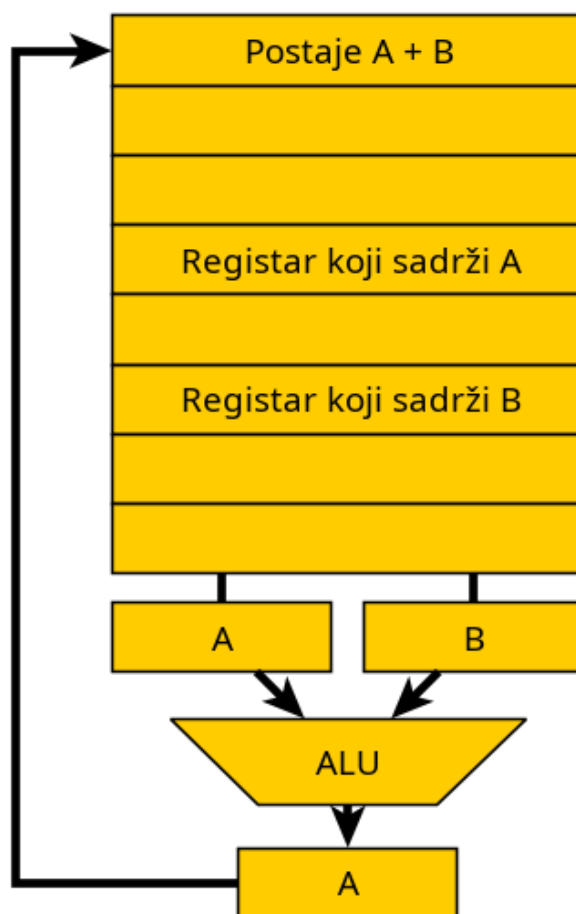


Slika 13: Harvard organizacija računara



Slika 14. Organizacija jednostavnog računara sa jednim procesorom

Osim akumulatora, sada u procesoru postoji i skup brzih memorijskih lokacija pod imenom registri. Svaki registar može da čuva određeni broj bita i ima tačno određenu funkciju (npr. **Program Counter (PC)** pokazuje na narednu instrukciju koju treba izvršiti, dok **Instruction Register (IR)** sadrži instrukciju koju trenutno izvršava, itd).



Slika 15. Put podataka (data path)

Kao što je ranije rečeno, registri su povezani sa ALU koji ih koristi i kao ulaze (jer se operandi nalaze u registrima) i kao izlaz (jer se rezultat operacije smješta u registar, najčešće akumulator, ali utiče i na statusne registre, itd.) što je prikazano na slici 15. Ukoliko se podaci ne nalaze u registrima nego u memoriji, prvo ih je potrebno pribaviti i smjestiti u blok registara pa na ulazni port ALU (**load**). Slična je situacija i sa rezultatom koji je potrebno pohraniti u memoriju - rezultat na putu do memorije prvo prolazi kroz registar (**store**) pa onda u memoriju. Iz ovoga je vidljivo da je broj, veličina, organizacija i način rada sa registrima izuzetno bitna osobina svakog procesora i računara.

Postoje dva načina kako možemo da ostvarimo bolje performanse: ubrzavanje jednog toka (praktično podizanje takta što nije moguće unedogled) i paralelizacija izvršavanja (kad je moguće). Što se tiče procesora, paralelizam je moguće ostvariti na dva načina: paralelizam na nivou više procesora (od poglavlja 6) i paralelizam na nivou instrukcije (od poglavlja 10).

4. Procesor - izvršavanje instrukcija

Procesor ne izvršava instrukcije kao monolitnu i kompleksnu operaciju, samo izvršavanje je podijeljeno u seriju jednostavnijih i relativno nezavisnih operacija (što će nam kasnije biti od velike koristi). Osnovni, klasični, redoslijed je sljedeći:

1. Pribavi instrukciju koju treba izvršiti iz memorije (PC pokazuje na nju) i smjesti je u IR
2. Uvećaj PC da pokazuje na narednu instrukciju
3. Odredi tip instrukcije koja je u IR
4. Ukoliko su potrebni parametri iz memorije dohvati ih i smjesti u registre
5. Izvrši instrukciju
6. Ukoliko je potrebno rezultat smjestiti u memoriju - smjesti ga
7. GOTO 1

Kako je rad procesora moguće napisati na ovaj način, moguće je rad prevesti i u neki programski jezik, a samim tim nam nije ni potrebno da imamo pravi, hardverski CPU da bi mogli da ga koristimo (ukoliko imamo na raspolaganju softver koji može da shvati i izvrši instrukciju i koji nazivamo **interpreter**). Na ovaj način nam je olakšano projektovanje i razvijanje novih procesora, ali su nam otvorena i vrata nekim vrlo korisnim stvarima (simulacija, emulacija i virtuelizacija). Još jedna vrlo bitna stvar koja proizilazi iz ovoga je da nam nije potrebno da baš sve operacije koje može da izvrši procesor budu realizovane u hardveru, dio možemo realizovati tako što će sam procesor dijelom biti i **interpreter** za instrukcije koje nisu realizovane u hardveru. Praktično pravimo mini procesor u procesoru (čak ne mini, bolje reći "mikro"). U početku želimo da jednostavno omogućimo izvršavanje i instrukcija koje se rijetko koriste tako da nam nije isplativo da ih realizujemo u hardveru. Kasnije shvatimo da nam ovakav način izvršavanja otvara mogućnost i da vršimo brojne optimizacije kratko opisane u poglavlju 1 a detaljnije obrađene u narednim poglavljima. Još jedna važna osobina procesora sa interpreterima je i mogućnost izmjene načina funkcionisanja procesora nakon isporuke izmjenom koda interpretera (što se pokazuje kao vrlo bitna osobina prethodnih godina, vidjeti Spectre, Meltdown i slične sigurnosne probleme).

Važno je shvatiti da je osnovni motivator kompanija novac. Ukoliko nije isplativo nešto realizovati u hardveru (jer se radi o relativno jeftinijem uređaju) to neće biti realizovano u hardveru, jer hardver košta. Slično i za skupe sisteme kod kojih su performanse ključni razlog postojanja, sve što možemo realizovati u hardveru, a kasnije i naplatiti - biće realizovano u hardveru. Na taj način pravimo segmentaciju prema vrsti korisnika. Već tokom pedesetih godina prethodnog vijeka, IBM je shvatio da je mnogo isplativije segmentaciju praviti prema performansama, a ne i prema načinu programiranja jer nas to dovodi u situaciju da je za bolje performanse potrebno promijeniti ne samo računar, nego i kod. Definisanjem i podržavanjem određenog načina upotrebe i programiranja sistema na **kompatibilan** način omogućeno je da korisnici sami biraju odgovarajuće performanse prema sopstvenim potrebama, ali i finansijskim resursima, bez potrebe za bitnom izmjenom samog koda. Ovaj nivo kompatibilnosti je nazvan **arhitektura**. Brzina na kojoj procesor radi naziva **takt procesora** i obično se izražava u Hz, odnosno u MHz i GHz, ali i u **milijunima instrukcija u sekundi (MIPS)**, dok je vrijeme izvršavanja instrukcije **latency**. Važno je i voditi računa o tome o kakvim instrukcijama govorimo, tako da možemo napraviti razliku između Dhrystone rezultata za Integer operacije, te Whetstone za Floating Point performanse. Takođe je važno shvatiti da svaki benchmark testira samo uzak skup mogućnosti i rezultate treba uzeti s rezervom.

5. Procesor - CISC i RISC

U situaciji u kojoj nije neophodno razviti i realizovati stvarni hardver da bi postojala podrška da određenu instrukciju koju želimo ili je žele neki naši korisnici, lako je upasti u zamku beskrajnog povećavanja broja instrukcija. Klasičan primjer je serija DEC VAX računara koji su u jednom trenutku podržavali (u množini) stotine instrukcija i stotine načina specifikacije operanada. Kako ovo zvuči kao kompleksan skup instrukcija, tako je i nazvan - **complex instruction set computer (CISC)**. Ovakav pristup nam omogućava da, na primjer, u jedan registar stavimo adresu sa koje kopiramo, u drugi adresu na koju kopiramo, u treći koliko riječi kopiramo i jednom instrukcijom pustimo procesor da sam odradi kopiranje cijelog bloka memorije.

Iako su korisnici jedno vrijeme bili vrlo srećni zbog fleksibilnosti, uskoro su došli u situaciju da nema bitnih pomaka u performansama i da se ispostavlja da su isti efekat koji dobijaju kupovinom novog sistema mogli postići i sami boljim pisanjem koda (u životu često vrijedi Paretovo ili 80/20 pravilo - tako se i ovdje pokazalo da od oko 300 instrukcija svega 20% instrukcija koristimo gotovo 80% vremena, a preostalih 80% instrukcija tek 20% vremena).

Alternativa tom pristupu se zasnivala na upotrebi smanjenog broja jednostavnijih instrukcija raspoloživih korisnicima, ali uz bolju optimizaciju izvršavanja i veći stepen realizacije u hardveru. Dva divna primjera su DEC MicroVAX 3400 i HP 835 Top Gun računari predstavljeni u isto vrijeme (1988. godina) i po gotovo identičnim cijenama (\$45000) ali uz bitno bolje performanse računara koji se zove "Top Gun". Taj alternativni pristup zasnovan na redukovanom skupu instrukcija je nazvan **reduced instruction set computer (RISC)**. Redukcija u broju instrukcija je varirala od ekstremne (samo 50 instrukcija) do ne baš dramatične (više od 100 instrukcija). Ono što je bitnije je činjenica da je u ovom slučaju veći broj instrukcija ili operacija bio realizovan sa direktnom hardverskom podrškom, a ne isključivo kroz sloj interpretiranja. Kao i svaka velika promjena, dovela do gotovo vjerskih ratova u oblasti računarstva uporedivih sa ratovima kao što su EMACS protiv vi editora, Apple protiv PC ili iOS protiv Androida.

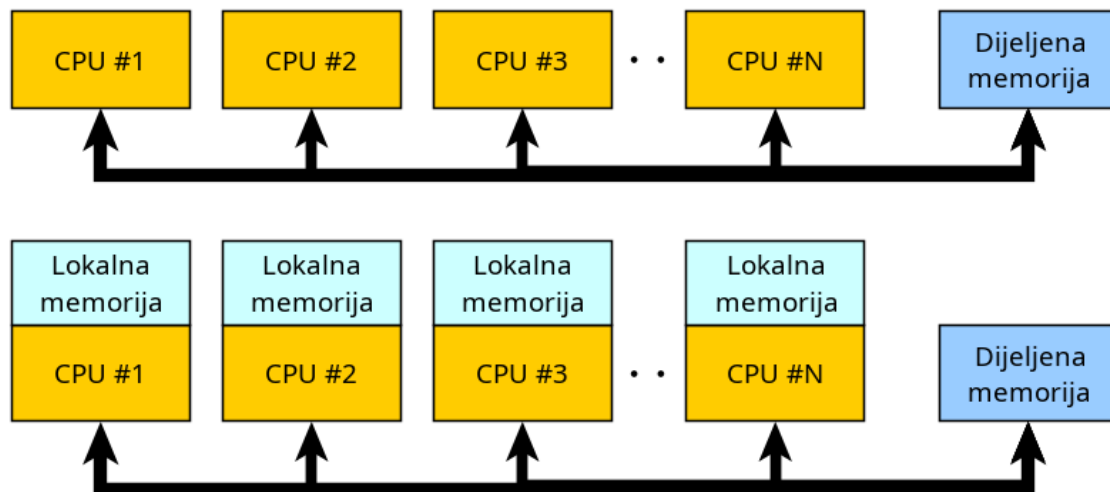
S jedne strane je CISC zagovarao eleganciju pisanja programa u manje instrukcija, s druge je bio RISC kamp sa tvrdnjom da nije problem ako nam za istu stvar koju realizuje jedna CISC instrukcija treba pet RISC instrukcija jer se one izvršavaju deset puta brže tako da na kraju dobijemo duplo bolje performanse. Ono što je važno shvatiti je da se gotovo ništa ne dešava u vakuumu i da je za bolje performanse RISC pristupa u velikoj mjeri zaslužan i veliki napredak u razvoju memorija što je omogućilo da se u velikoj mjeri eliminišu problemi vezani za dohvaćanje većeg broja instrukcija iz memorije (ali i kapaciteta memorije potrebne za programe jer je CISC kod mnogo "gušći" i zauzima manje prostora u memoriji), ali i smanjuje jaz između brzine izvršavanja interpretera u procesoru (CISC) ili direktnog izvršavanja instrukcija u procesoru (RISC).

Danas je jedina preživjela CISC arhitektura Intelova, ali ta arhitektura je apsolutno dominantna u mnogim segmentima računarstva i ima ogroman ekonomski uticaj što u velikoj mjeri garantuje i prilično sigurnu budućnost. Vrijedi napomenuti da su i moderni Intel kompatibilni procesori "u duši" RISC procesori sa slojem kompatibilnosti koji im omogućava da izgledaju kao kompatibilni CISC procesori. Intelov inženjer Bob Colwell, koji je i dizajnirao prvi takav procesor - Intel P6, je prilično "alergičan" na takve izjave jer to i nije baš RISC jezgro i nisu instrukcije, već više liči na VLIW pristup i govorimo o mikrooperacijama, a ne instrukcijama. Takođe, i moderni RISC procesori rade na praktično identičan način. ㄟ_(ツ)_ㄟ

6. Multiprocesori

Ako je računarski sistem baziran na više potpuno samostalnih procesora (od kojih svaki ima svoju upravljačku jedinicu) možemo govoriti o multiprocesorskim sistemima. **Multiprocesorski sistem** sadrži više od jednog samostalnog procesora koji dijeli radnu memoriju sa drugim procesorima u sistemu. Kako sada memorija predstavlja dijeljeni resurs, neophodno je postojanje mehanizma koordinacije pristupa i upotrebe memorije. Ovakav način bliske saradnje više procesora se naziva **snažna sprega**. Komunikacija između procesora se obavlja preko dijeljene memorije, a jedan od primjera je OpenMP.

Realizacija opisanog pristupa je moguća na više načina, od kojih je najjednostavniji sistem u kojem svi procesori ravnopravno pristupaju dijeljenoj memoriji, što je prikazano na slici 16 (gore). U ovakvoj situaciji brzo dolazimo do problema kada više procesora pokušava da radi sa memorijom jer nije moguć istovremeni pristup što dovodi do čekanja na resurs.



Slika 16. Primjer realizacije multiprocesorskog sistema

Na slici 16 (dolje) je prikazano jedno moguće rješenje koje je zasnovano na ideji da svaki procesor ima sopstvenu lokalnu memoriju kojoj ne mogu pristupati drugi procesori. U ovoj memoriji se mogu nalaziti instrukcije ili podaci koji su namijenjeni samo jednom procesoru. Kako je pristup direktan, performanse su odlične. Više riječi o ovom problemu će biti u narednim poglavljima. Postavlja se pitanje kako su procesori zaista povezani i kako komuniciraju. Tradicionalno, dva CPU-a komuniciraju preko dijeljene memorije (kojoj pristupaju preko FSB magistrala ili Front Side Bus-a), što radi na sljedeći način: CPU0 upiše u RAM, CPU1 pročitava, pa upiše, pa ... Ovakav pristup komunikaciji je prilično neefikasan, naročito ako postoji potreba za čestom razmjenom velike količine podataka ili ako radimo sa većim brojem procesora (jer je RAM bitno sporiji od samog procesora). Savremeni računari gotovo svi koriste point-to-point veze umjesto magistrale što znači da postoji čvor koji je srce zvijezde koja povezuje sva jezgra i omogućava bržu komunikaciju između njih. Intel-ov pristup se zove QuickPath Interconnect (QPI), dok je slična tehnologija u upotrebi od strane AMD-a nazvana HyperTransport (HT), mada se u novijim generacijama naziva Infinity Fabric (IF).

Ako ste se zapitali kako je došlo do toga da i Intel i AMD (a i gotovo svi drugi moderni procesori) koriste vrlo slične pristupe, odgovor leži u najvažnijem procesoru za koji vrlo vjerovatno

nikad niste čuli - DEC Alpha 21x64. Riječ je o familiji procesora napravljenoj od strane firme DEC (tvorci PDP-8 i PDP-11 iz prethodnih poglavlja) koja je u svakom koraku bila revolucionarna i donijela nešto što se od tog trenutka smatralo za pravi pristup:

- 21064 (EV4) - prvi CMOS CPU iste frekvencije kao ECL. Tradicionalno su CMOS procesori bili spori uz malu potrošnju energije, a za performanse ste morali imati ECL kola koja su vrlo brza i vrlo rastrošna. Svi moderni procesori visokih performansi su CMOS.
- 21164 (EV5) - prvi sa velikim L2 kešom na čipu. Tradicionalno su procesori imali vrlo malu količinu keša (najčešće samo jednog sloja) na čipu. Svi moderni procesori imaju velike količine višenivovskog keša na čipu.
- 21264 (EV6) - prvi visoke frekvencije i naprednim izvršavanjem instrukcija van originalnog redoslijeda (OOOE). Tradicionalno ste ili birali visok takt ili izvršavanje van redoslijeda. Svi moderni procesori visokih performansi rade izvršavanje van redoslijeda.
- 21365 (EV7) - prvi sa ugrađenim memorijskim kontrolerom na čipu. Tradicionalno su procesori koristili FSB za pristup memoriji kroz memorijski kontroler. Svi moderni procesori imaju ugrađen memorijski kontroler na čipu.
- 21464 (EV8) - trebao biti prvi sa podrškom za Simultaneous Multi-Threading (predstavljanje jednog procesora kao dva sa ciljem poboljšanja performansi). Brojni moderni procesori podržavaju ovakav rad.

DEC je kao firma propala (česta priča za one koji budu previše ispred svog vremena i bez dobre poslovne i marketinške podrške) i prodana je prvo firmi Compaq, a zatim Intel-u. Dio tima koji je ostao u firmi je razvio HyperThreading i QPI, dok je dio tima koji je napustio DEC i otišao u AMD razvio HyperTransport. Sve navedene tehnologije i pristupi danas predstavljaju osnovu savremenih procesora, višeprocorskih mašina i, uopšte, podrazumijevanog načina rada računara.

7. Višejezgreni procesori i HyperThreading

Kao što smo ranije naveli, da bi zaista ostvarili multitasking (istovremeno izvršavanje više programa) potrebno nam je i više jezgara. Kod višejezgrenih (multicore) procesora svaki se procesor se sastoji od više samostalnih, međusobno identičnih, jezgara i taj pristup se naziva Symmetric multiprocessing (SMP). Riječ je o čvrsto spregnutim multiprocesorima koje čini skup homogenih jezgara što znači da nije bitno na kojem jezgru se izvršava naš niz instrukcija (jer su sva ista). Praktično možemo i da mijenjamo jezgra u toku izvršavanja (ili između vremena izvršavanja) i ništa se bitno neće desiti (osim malog gubitka performansi izazvanog kopiranjem podataka između jezgara). Sva jezgra dijele zajedničke resurse od kojih je najvažniji radna memorija. Sama veza između jezgara može biti magistrala, zvijezda, crossbar ili neki drugi pristup.

Postavlja se pitanje da li jezgra moraju biti identična i šta ako nisu? Heterogenost predstavlja komplikaciju za operativni sistem i raspoređivač, jer sada moramo da vodimo računa i o tome smijemo li prebaciti proces na *drugačije* jezgro. Jedan od prvih procesora sa različitim jezgrima je bio NVidia vSMP (Tegra 3) koji se sastojao od dva skupa jezgara: 4 ista jezgra visokih performansi i 1 posebno jezgro za smanjenu potrošnju električne energije (riječ je o procesoru za prenosne uređaje). Dodatno jezgro se obično ne koristi za aktivne programe nego za pozadinski rad ili reprodukciju multimedijalnih zapisa koji su u stanju da iskoriste namjenske sklopove za dekodovanje tako da samo jezgro nije opterećeno.

Danas je u mobilnim uređajima vrlo popularan ARM big.LITTLE (DynamIQ) pristup koji je zasnovan na podjeli na brza i efikasna jezgra koja mogu biti vrlo različita jezgra. Ovaj pristup je u praksi više "ideja vodilja" nego čvrsto pravilo. Na primjer, Google Tensor procesor se sastoji od dva vrlo brza X1 jezgra, dva umjereno brza A76 jezgra i četiri efikasna (spora) A55 jezgra. Postoje i procesori koji se sastoje od jezgara koja podržavaju različite dužine procesorskih riječi, pa tako kombinuju ARM Cortex A710 (brzo jezgro koji podržava i 64b i 32b izvršavanje) i A510 (efikasno jezgro koje podržava samo 64b izvršavanje) što je naročito "vesela" situacija za operativni sistem koji sada mora da vodi računa i tome kakav proces na kom jezgru može izvršavati.

Danas je upotreba heterogenih jezgara gotovo nezaobilazna u mobilnim uređajima i teško je naći napredniji procesor koji nema bar dva tipa jezgara. Apple Silicon predstavlja sličan pristup, ali uniformniji jer su jezgra spolja vrlo slična (gotovo identična sa strane izvršavanja programa) ali bitno različitih performansi. Iako je pristup potekao od mobilnih uređaja, čak i Intel x86-64 nije ostao imun na ove procese što je vidljivo od prvih pravih koraka sa Intel Alder Lake serijom procesora koji imaju P(erformance) i E(fficiency) jezgra tako da je prilično izvjesno da ovaj pristup predstavlja budućnost i u x86 svijetu.

Simultaneous Multithreading / HyperThreading

Simultaneous multithreading (SMT) ili HyperThreading (HT) predstavlja pristup u kom CPU "laže" operativni sistem da ima dva "virtuelna" jezgra za jedno "fizičko" koje zaista ima. Zašto? Kakva je korist od toga da lažemo raspoređivač ako nemamo zaista dva jezgra? Cijedimo suvu drenovinu?

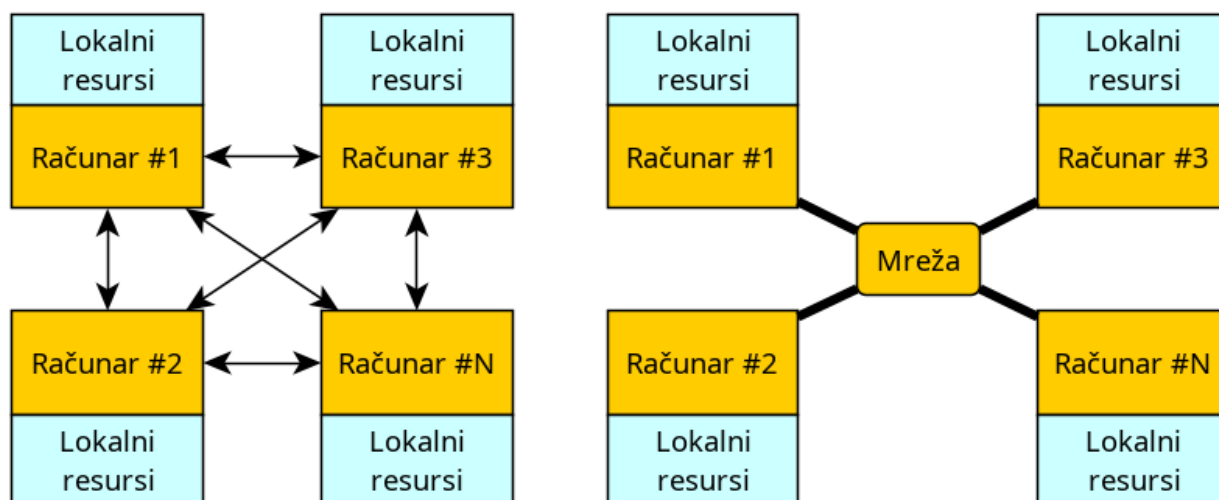
Prednosti ovog pristupa je najlakše ilustrovati na praktičnom primjeru: Neka procesor ima jedno jezgro koje ima po jedan INT i jedan FP blok (vrlo pojednostavljeno, ali dovoljno dobro za objašnjenje). Neka jedan program ima dugačak niz FP množenja, a drugi program dugačak niz INT

sabiranja (oba vrte petlje i računaju nešto od velike važnosti za korisnika). Dok se prvi program izvršava (raspoređivač može da dozvoli samo jednom da se izvršava jer ima samo jedno jezgro) čeka da FP blokovi odrade posao (što je vrlo često sporo jer su FP operacije kompleksne) i čeka i čeka i čeka, a za to vrijeme INT blokovi stoje neiskorišćeni. Ovdje ne pomaže ništa od tehnika koje se inače koriste za ubrzavanje izvršavanja jer se u jednom trenutku izvršava samo jedan program (jer imamo samo jedno jezgro). Ako CPU "slaže" da ima *DVA* jezgra, raspoređivač će dozvoliti da se *OBA* programa izvršavaju istovremeno (jer ima *DVA* jezgra). Na ovaj način će procesor moći da iskoristi INT blok za drugi program dok prvi čeka na završetak rada FP blokova. Sve dok im ne trebaju isti resursi unutar CPU-a mi dobijamo bolje performanse. Za visoko optimizovan paralelni kod neće biti bitne razlike, čak možete dobiti i gore performanse (jer postoje troškovi emuliranja postojanja dva jezgra). Bolje performanse nećemo dobiti jer se petlja vrti na isti način i u istom trenutku su za obje niti potrebni isti resursi (a i kompajler zna da složi instrukcije tako da resursi procesora budu bolje iskorišćeni, a i sam procesor pomaže u tome).

Cijeli opisani proces nije jednostavan i zahtijeva dosta dodatne logike u procesoru što košta i troši električnu energiju (moramo imati duplirane SVE kontrolne resurse da bi JEDNO jezgro moglo da izvršava DVA niza instrukcija bez neželjenih efekata). Ipak, često je poželjna upotreba SMT-a, naročito u slučaju opšte upotrebe računara za uobičajeni korisnički softver.

8. Multiračunari

Multiračunari su **slabo spregnuti** sistemi koji ne koriste dijeljenu memoriju i na taj način zaobilaze probleme simultanog pristupa memoriji od strane velikog broja procesora (mijenjajući ih potpuno novim skupom problema). Sada svaki računar ima sopstvenu memoriju kojoj samo on pristupa, a komunikacija se dominantno obavlja razmjenom poruka (jer ne postoji zajednička dijeljena memorija) putem mrežne infrastrukture, a primjer je **message passing interface (MPI)**. Ovakav način realizacije je prikazan na slici 17, prvo kao veza svakog sa svakim (full mesh) a zatim i u topologiji zvijezde.



Slika 17. Primjer dva načina povezivanja multiračunara

Mrežna infrastruktura i njena topologija definišu moguće načine upotrebe ovakvih sistema jer su kašnjenja u odnosu na dijeljenu memoriju mnogo veća, ali je moguće i povezati pojedine čvorove na proizvoljne načina. Iz tog razloga je od izuzetnog značaja odabrati pravu mrežnu topologiju i odgovarajuće algoritme za postizanje željenih performansi.

Kada govorimo i topologiji **zvijezde** postoji središte sistema koje povezuje sve ostale tačke. To središte je najčešće mrežni svič sa po linkom do svakog računara (čvora). Ovo je najjednostavnije rješenje za relativno manje klastere jer nam treba samo jedan svič kojim omogućavamo povezanost. Sam klasičan mrežni svič može imati 24 ili 48 ili čak i stotine portova (o tome ćete više učiti na drugim predmetima). Problem kod ovakvog pristupa je u tome što je brzina prenosa određena brzinom tog jednog linka. Takođe, ne možemo da šaljemo podatke na više čvorova punom brzinom istovremeno (osim u posebnim situacijama), a ne možemo da primamo podatke sa više čvorova punom brzinom (ograničeni smo tim našim jednim linkom bez obzira preko koliko linkova podaci na svič dolaze za nas). Jedno očigledno rješenje je da "poduplamo" linkove. Ili stavimo po 4 linka sa svakog čvora. Ovo je moguć pristup, ali sada smanjujemo broj čvorova u zvijezdi (umjesto 24 računara sada možemo povezati samo 12 ili 6). Još jedna stvar o kojoj moramo voditi računa je to što grupna komunikacija postaje problematična (iako je rješiv problem, nije trivijalan), jer moramo organizovati razmjenu između svih čvorova što često dovodi to čekanja na podatke i "praznog hoda" u radu pojedinačnih čvorova.

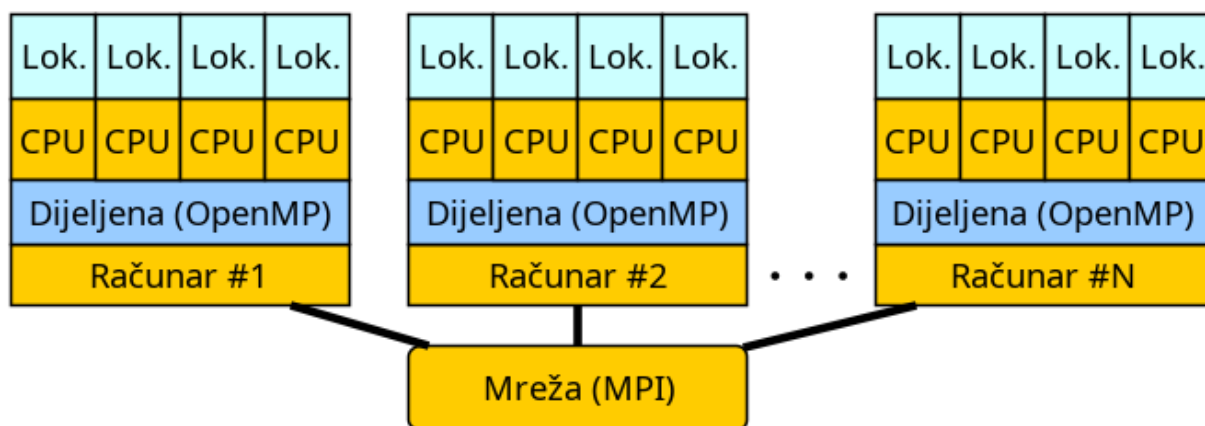
Kada govorimo o **full mesh** topologiji svaki čvor je povezan sa svakim drugim čvorom što nam omogućava da možemo sa bilo kim da komuniciramo punom brzinom (jer imamo direktan link

do svakog drugog čvora). U ovom slučaju je grupna komunikacija bez problema jer možemo direktno da komuniciramo sa svima i možemo da šaljemo i primamo u isto vrijeme. Jedini *mali* problem je broj linkova i portova koji raste sa kvadratom broja čvorova prema formuli $N*(N-1)/2$. Tako nam za klaster sa 4 čvora od kojih svaki ima po 4 porta dobijamo 16 portova i 6 linkova. Za klaster sa 32 čvora u kojem svaki čvor ima po 32 porta dobijamo 1024 porta i 496 linkova. U tom slučaju nam trebaju i 32-portne mrežne kartice ako uopšte postoje na tržištu. Postavlja se i pitanje da li uopšte možemo da obradimo toliko saobraćaja, a i da ne pominjemo da je sve ovo vezano samo za međusobne veze i potpuno smo zaboravili da nam treba i bar još jedan port da bi mi pristupili čvoru (i on ostatku mreže).

Naravno, na raspolaganju nam je i opcija da pravimo namjensku topologiju za naš klaster što nam omogućava da svaki čvor povežemo sa manjim brojem drugih, ali na način koji nama omogućava kvalitetan rad. Na primjer, ako je organizacija čvorova kao 2D matrica i svaki čvor je povezan samo sa susjedima gore, dolje, lijevo i desno. Krajnje čvorove povežemo i sa parovima na suprotnim krajevima (tako da dobijamo prsten kako god da pogledamo). Na ovaj način dobijamo jednu od brojnih varijacija na tu temu: grid, cube, torus, hypercube, hypertorus, itd. Možemo i sami da napravimo sopstvene topologije prema konkretnim potrebama, ali se postavlja pitanje: Zašto?

Ako simuliramo vremenske prilike, bitni su nam samo susjedi koji direktno utiču na nas, ne i čvorovi koji simuliraju vrijeme osam ćelija daleko od nas. Ovo je čest slučaj za superračunare i pokazuje se da ovakav pristup radi i za simuliranje velikog broja drugih fizičkih procesa (jer su većinom prostorno ograničeni) uključujući simulacije fluida, sagorijevanje, nuklearne reakcije, itd. Na ovaj način dobijamo performanse slične kao kod potpune veze, ali sa mnogo manje linkova i jednostavnijom mrežnom infrastrukturom. Važno je razumjeti da ovo rješenje nije univerzalno rješenje i da pravimo novu topologiju za svaku klasu problema. Postoji i problem pristupa nesusjednim podacima koji se obično rješava upotrebom **softverskog prosljeđivanja** - zamolite susjedni čvor da proslijedi dalje što jeste sporije, ali radi i često je dovoljno dobro rješenje.

Kao i uvijek, danas su u upotrebi **hibridna rješenja** koja se zasnivaju na slaboj sprezi računarskih čvorova, pri čemu je svaki čvor snažno spregnuti multiprocesorski sistem. Ovakav pristup je prikazan na slici 18. Isti pristup postoji i u slučaju hijerarhijske organizacije čvorova pri čemu dobijamo topologije iz klase stabla (samo na slici umjesto CPU zamisliti cijeli računar).



Slika 18. Primjer hibridne organizacije

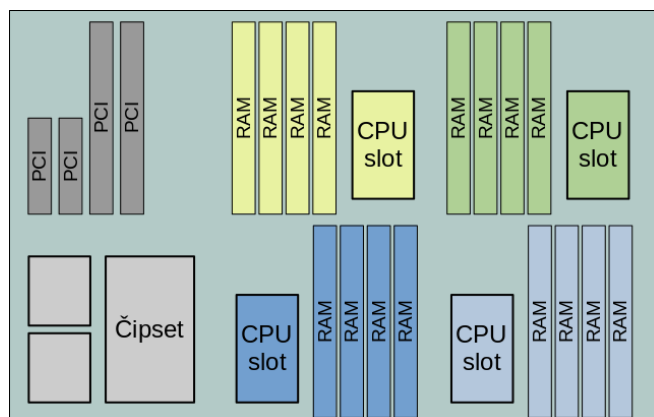
9. Masovno multiprocesorske mašine

Savremeni računari visokih performansi su često masovno multiprocesorske (MMP) mašine i imaju zaista veliki broj procesora (jezgara). U tom slučaju moramo da vodimo računa i o tome šta je lokalno za naš procesor (pa je brže dijeljenom memorijom), a šta nije (pa mora da se koristi komunikacija porukama). Kako želimo da radimo efikasno, bolje je da to radi neko drugi za nas ;)

Sada imamo sve probleme iz prethodnih poglavlja, a savremeni server je upravo ovakvog kompleksnog dizajna. Tipično je mnogo jezgara raspoređenih u čiplete (chipllets) - blokove od po 4 ili 8 pojedinačnih jezgara i brža je komunikacije između jezgara na istom čipletu nego na različitim čipletima, a još je sporija ako postoje dva procesora sa po 4 čipleta sa po 4 jezgra u jednom računaru pa komuniciraju dva jezgra sa različitim procesora (iako su i dalje u istom računaru).

Koliko procesora staje u jedan računar? Za komercijalna rješenja zasnovana na AMD procesorima, možemo posmatrati sljedeći primjer: AMD EPYC procesor sa 64 jezgra (128 niti) i dva procesora u jednom računaru omogućava 128 jezgara (256 niti) na jednoj matičnoj ploči, te do 8 TB RAM. Za Intel Xeon Scalable (Ice Lake-SP) procesore moguće je na jednu matičnu ploču smjestiti do 8 CPU sa po 40 jezgara (80 niti) što nam daje 320 jezgara (sa 640 niti) i do 32 TB RAM. Ovo su vrijednosti za jedan čvor koji dalje možemo da povezujemo u prethodno opisane topologije. Na primjer, kao što smo vidjeli, potpuno povezana topologija sa 4 čvora nije problematična, a ovdje nam omogućava rješenje sa do 1280 jezgara (2560 niti) što se često ispostavlja kao sasvim dovoljno.

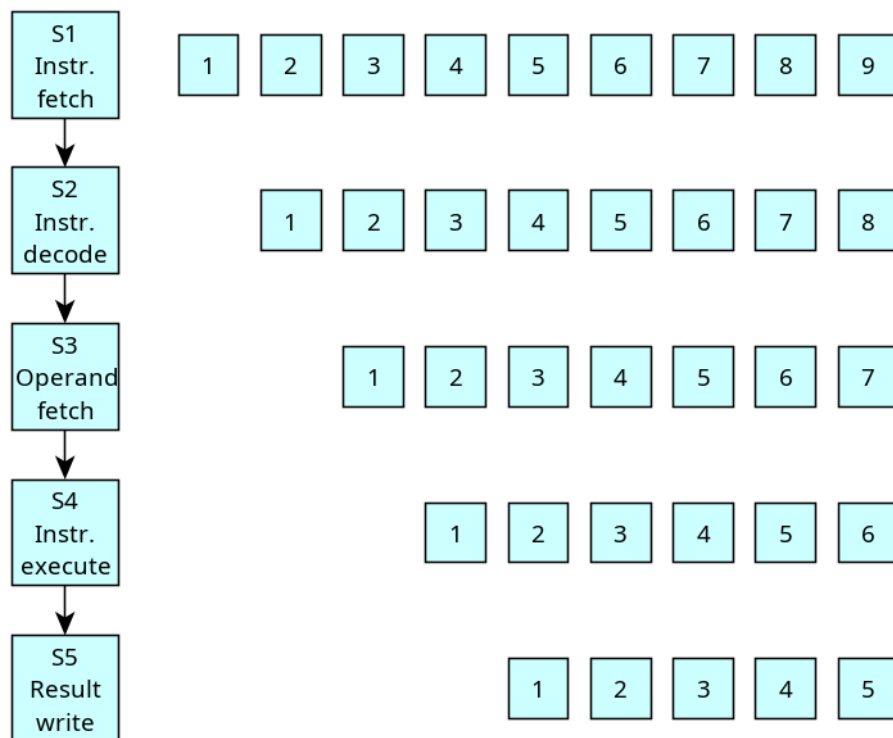
Naravno, očigledno je da je sada pristup memoriji sve samo ne uniforman, tako da se, vrlo neočekivano, naziva Non-Uniform Memory Access (NUMA). Nekada je ovaj problem bio izražen samo kod multiračunara, kasnije se pojavio i kod multiprocesorskih računara, a danas su gotovo svi procesori multiprocesori. Mi želimo da podaci koje često koristimo budu što bliži procesoru koji ih najčešće koristi jer tako dobijamo bolje performanse. Gotovo svi savremeni operativni sistemi podržavaju ovakav rad, a često postoji i podrška u bibliotekama (kao što je OpenMP). U ovom slučaju poseban problem predstavlja keš jer je moguća situacija da imamo više kopija istog podatka (jer ga koristi više jezgara i svaki ga čuva u svom kešu) i moramo da vodimo računa o svakoj kopiji. Ovaj problem se rješava upotrebom ccNUMA (Cache Coherent NUMA) tehnologije koja narušava performanse u odnosu na idealan slučaj, ali i dalje nudi mnogo bolje performanse u odnosu na odsustvo takvog pristupa (kompromis je često jedino smisljeno rješenje u praksi). Čak i sam izgled matičnih ploča za više procesora jasno pokazuje da su procesori bliži "svojoj" memoriji.



Slika 19: Ilustracija izgleda matične ploče (NUMA)

10. Pipelining

Pipelining predstavlja vid paralelizma na nivou instrukcije. Osnovna ideja je da se koraci opisani u poglavlju 4 ne izvršavaju 1-6 za jednu instrukciju, pa 1-6 za drugu, itd, nego da se pojedini koraci izvršavanja (kojih može biti i mnogo više od navedenih 6) izvršavaju istovremeno, ali pomjereno za jedan korak. Ovo je moguće u slučajevima kada različiti koraci koriste različite jedinice u procesoru, na primjer, jedinica koja dobavlja instrukciju iz memorije nakon završenog posla nema nikakvu aktivnost do izvršavanja naredne instrukcije. Možemo tu pauzu da iskoristimo da dobavimo narednu instrukciju tako da kad dođe do potrebe za pribavljanje naredne instrukcije - posao je već urađen i ostvarili smo uštedu u vremenu. Ovaj koncept se proširuje i na ostale korake (dekodovanje instrukcije, pribavljanje operanada, itd), kao na slici 20.



Slika 20. Pipeline od 5 koraka, prikazano 9 vremenskih ciklusa

U toku prvog ciklusa, prva instrukcija se pribavlja. U toku drugog, prva instrukcija se dekodira, a druga pribavlja. U toku trećeg, dobavljamo operande za prvu, dekodujemo drugu i pribavljamo treću. U četvrtom izvršavamo prvu, dobavljamo operande za drugu, dekodujemo treću i pribavljamo četvrtu. U petom upisujemo rezultat prve, izvršavamo drugu, dobavljamo operande treće, dekodujemo četvrtu i dobavljamo petu.

Na ovaj način smo bez povećanja takta procesora uspjeli da ubrzamo izvršavanje pet puta. Naravno, sve navedeno vrijedi u idealnim uslovima u kojima su svi koraci istog trajanja, sve što nam treba je dostupno i slobodno u datom trenutku, ništa ne čekamo, ništa ne preskačemo, nema instrukcija koje uopšte nemaju operande, itd. U stvarnosti nije moguće ostvariti takve direktne dobiti, pogotovo kod vrlo dubokih pipelinea, ali su i dalje ostvarena poboljšanja performansi značajna.

Koji problemi postoje u slučaju upotrebe pipelinea? Zašto procesori nemaju pipeline koji se sastoji od stotina koraka? Prije svega, postoje procesori koji imaju vrlo dubok pipeline (nekoliko stotina koraka) ali oni predstavljaju samo istraživanje, ne i komercijalne proizvode.

Problemi nastaju iz mnogo razloga, ali najčešće su vezani za izvršavanje instrukcija koje međusobno zavisne? Npr. neka je potrebno izvršiti sljedeće dvije operacije:

$$A = A + 42$$

$$B = A$$

U ovom slučaju nije moguće izvršiti drugu operaciju ($B=A$) dok ne izračunamo A u prethodnom koraku (ne možemo da pribavimo operand A jer ga još nema). Postoje tri osnovna rješenja za ovaj problem:

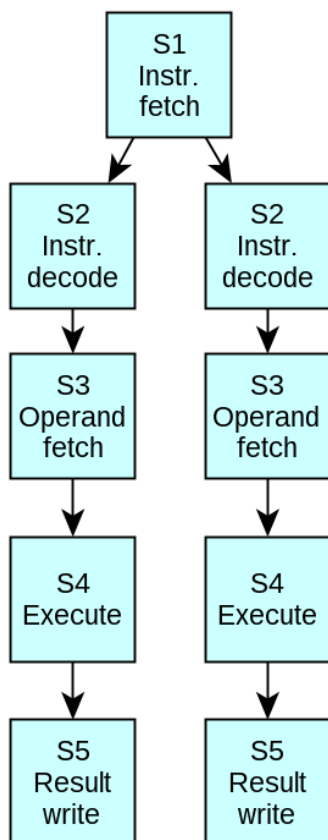
1. Možemo da čekamo (bubbling) - jednostavno čekamo da A postane dostupno i da onda nastavimo izvršavanje. Ovo je najjednostavniji način ali je prilično neefikasan jer procesor mnogo vremena provodi ne radeći ništa, a i samo čekanje može biti dosta kompleksnije od prostog čekanja na jedan operand.
2. Možemo da pravimo povratne veze (operand forwarding) - procesor ima posebne povratne veze koje omogućavaju da je vrijednost A dostupna narednim operacijama i prije nego što je njegova operacija *potpuno* završena (A postaje dostupno čim je izračunato). Sličan pristup se često koristi u vektorskim procesorima (poglavlje 12).
3. Možemo da tražimo šta možemo da izvršimo u međuvremenu (izvršavanje van redoslijeda) - ovaj pristup je najkompleksniji i zahtijeva prilično naprednu kontrolnu logiku koja je u stanju da pouzdano izračuna šta smijemo da izvršavamo i kada, kao i da iz skupa mogućih instrukcija odabere tačan redoslijed koji će optimalno opteretiti sve elemente pipelinea.

Očigledno je da što je dublji pipeline to je veća vjerovatnoća problema u pipelineu (hazard), a kako i samo razrješavanje problema traje duže (jer je često u takvim slučajevima neophodno isprazniti cijeli pipeline) i same performanse su lošije. Iz tog razloga se u praksi ne koriste vrlo duboki pipelinei (Intel Pentium 4 ima dubinu 20, a Pentium D čak 31 i oni su predstavnici starijih pristupa povećanja dubine). Savremeni CPU rade drugačije i pametnije (i mnogo kompleksnije) upotrebom naprednih tehnika izvršavanja van redoslijeda (Out-of-Order Execution).

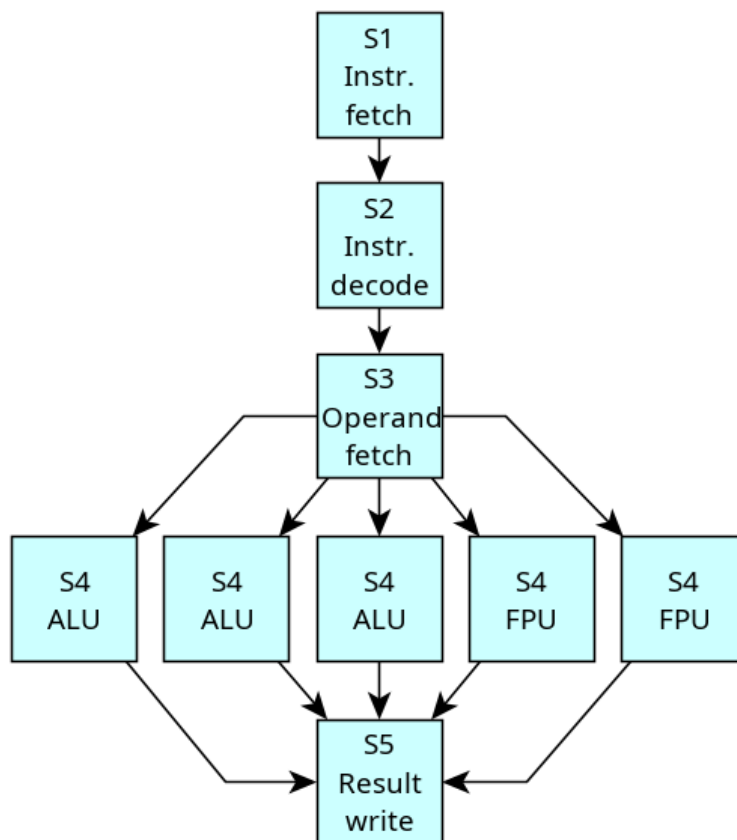
11. Superskalarnost

Često se obični RISC procesori nazivaju i **skalarni RISC procesori** jer su u stanju da u svakom ciklusu počnu izvršavanje tačno jedne instrukcije. Kako je naša želja da imamo što bolje performanse, mi želimo da budemo **superskalarni**. Moderna **definicija superskalarnog procesora** je procesor koji može da započne izvršavanje više od jedne instrukcije u svakom ciklusu.

Kako smo već realizovali pipeline, zaključili smo da bi bilo još bolje kada bi imali više izvršnih jedinica na raspolaganju pa bi mogli dobavljati dvije po dvije instrukcije i izvršavati ih istovremeno što je prikazano na slici 21. Iako to ima smisla, na taj način bi možda dobili dvostruko bolje performanse, ali po cijenu dvostruko više hardvera (i to u slučaju da možemo da izvršavamo instrukcije u paraleli). Kako svi koraci nisu isto zahtjevni i ne traju isto vremena, pokazuje se da je bolje i isplativije ne duplirati cijeli tok nego se koncentrisati na ono što će nam donijeti najviše uz najmanje dodatnog truda i hardvera. Taj kritični korak, koji obično traje značajno duže od jednog ciklusa, je izvršavanje instrukcije. Na slici 22 je prikazan primjer superskalarnog procesora koji ima pet izvršnih jedinica umjesto jedne prisutne u prethodnim primjerima.



Slika 21: Dvostruki pipeline



Slika 22. Primjer procesora sa pet izvršnih jedinica

Prvi superskalarni procesor u širem smislu je bio u CDC 6600 računaru i sadržavao je 10 izvršnih jedinica. Danas su gotovo svi procesori (osim onih vrlo specifične namjene) superskalarni i koriste asinhrono dekodovanje i redove čekanja instrukcija o čemu će više riječi biti u poglavljima 13 i 14. Iako se kao alternative superskalarnim procesorima navode VLIW procesori, višejezgreni procesori i SMT, zapravo nije riječ o alternativama i u praksi se koriste svi navedeni pristupi u kombinaciji.

12. SIMD i vektorski procesori

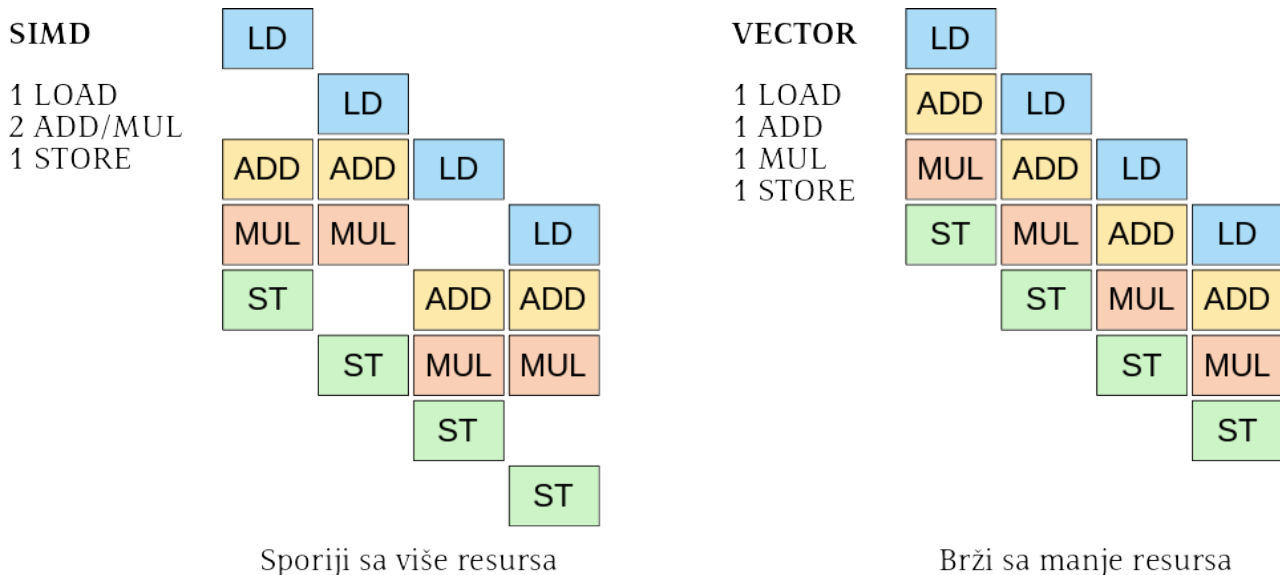
Većina numerički zahtjevnih poslova koje obavljaju procesori su obično vezani za petlje, vektore i matrice. Da bi procesor intenzivno zaposlili na duže vrijeme, potrebno je da obezbijedimo veliku količinu podataka nad kojom može da izvršava određene sekvence instrukcija (alternativa bi bila da pišemo ogromne količine koda što se kosi sa našom prirodom). Podaci na ulazu i transformacije koje želimo činiti na njima su obično strukturisani i često se svode na ponavljanje operacija na nizu ulaznih podataka. Ovo nam otvara priliku da izvršimo paralelizaciju i da ostvarimo bolje performanse. Dva osnovna pristupa koja se koriste su: SIMD procesori i vektorski procesori. Iako postoje velike sličnosti, često se SIMD posmatra kao paralelni računar dok se vektorski pristup posmatra kao proširenje jednog procesora. Vrijedi napomenuti da savremeni procesori koriste oba pristupa prema potrebi i često je teško povući jasnu granicu između njih.

Single instruction multiple data (SIMD) procesor se interno sastoji od velikog broja identičnih procesorskih jedinica koje mogu da obavljaju istu sekvencu instrukcija na različitim podacima. Svi moderni procesori koji se koriste u okviru grafičkih kartica su bazirani na SIMD pristupu iz jednostavnog razloga što su poslovi koji se očekuju od procesora grafičke kartice izuzetno repetitivni (obrađivanje velikog broja piksela, vektora, tekstura, itd). Važno je napomenuti da pri ovakvom izvršavanju postoje određena ograničenja, prije svega vezana za činjenicu da su sa jedne strane ograničene mogućnosti svake procesorske jedinice, a s druge strane da sve jedinice u bloku moraju da izvršavaju istu sekvencu instrukcija (drugim riječima, nema grananja). Jedan eksperimentalni fizičar je uporedio upotrebu grafičkih kartica sa oranjem njive, ne pomoću jednog vola, nego pomoću 1024 pileta. Iako je svako pojedinačno pile značajno slabije, brojnost više nego kompenzuje razliku u snazi.

Opisani pristup je našao veliku primjenu u modernim procesorima pod nazivom Streaming SIMD Extensions (SSE), a zatim i kao Advanced Vector Extensions (AVX) koji podržava paralelne vektorske operacije. Osnovna primjena je u oblasti multimedije (npr. kodovanje i dekodovanje video zapisa) i za matična izračunavanja. Važno je primijetiti da opisana rješenja ne predstavljaju potpuno nezavisne procesore, već su bazirana na višestrukim ili namjenskim izvršnim jedinicama ograničene namjene pod kontrolom jedne upravljačke jedinice.

Vektorski procesor je spolja vrlo sličan SIMD procesoru, ali su razlike u načinu implementacije. Slično SIMD procesoru, vektorski procesori su izuzetno efikasni u izvršavanju operacija nad nizovima podataka. Razlika leži u načinu kako se ta "efikasnost" realizuje. Kod SIMD pristupa, procesor ima, na primjer, dovoljan broj sabirača da sabere sve elemente, vektorski procesor koristi **vektorske registre**. Oni se učitavaju serijski iz memorije i nakon toga, na primjer, se sabiraju po parovima jednim vektorskim sabiračem (koji je *izuzetno* pipelineovan). Izlaz je moguće ili smjestiti u vektorski registar ili koristiti kao ulazni vektor za narednu operaciju, čak i prije potpunog završetka izračunavanja tog koraka (slično jednom od predloženih rješenja za pipeline pristup što ima smisla jer su vektorski procesori dubokih pipelinea).

U čemu su osnovne razlike između ova dva pristupa? Jedan jednostavan primjer koji ilustruje kako je moguće da vektorski procesor ostvari bolje performanse od SIMD procesora koji ima više resursa je prikazan na slici 23. Primjer ilustruje četiri ciklusa LOAD-ADD-MUL-STORE operacija što je čest primjer i u praksi. Vektorski procesor ima po jednu ADD i MUL jedinicu, a SIMD ima po dvije ADD i MUL jedinice, dok oba imaju po jednu LOAD i STORE jedinicu.



Slika 23: Poređenje vektorskog i SIMD pristupa

Ako želimo malo formalnije da objasnimo razliku, možemo to uraditi na sljedeći način:

- SIMD rade sa fiksnim dužinama podataka (mogu imati različit broj blokova podataka, ali najčešće su ograničeni na broj blokova koji je stepen broja 2), dok vektorski mogu da postavljaju dužinu vektora na bilo koju potrebnu.
- SIMD ne mogu da koriste međurezultate izračunavanja, dok jedinice u vektorskim procesorima gotovo uvijek koriste međurezultate prethodnih operacija (praktično operacija koje su još u toku).
- SIMD mora da učitava sve podatke potrebne za prvi korak, pa da ih obradi, pa da pređe na drugi korak, ... praktično moraju imati višestruke elemente da bi bili efikasni što ima smisla ako razumijemo da bolje performanse izvedu iz paralelne obrade podataka i izvođenja iste operacije u istom trenutku na svim elementima (ne može jedan raditi ADD, a drugi MUL), dok su vektorski pipelineovani interno tako da ne moraju imati višestruke elemente.

Važno je napomenuti da, iako ime sugerise drugačije, procesori sa Intel AVX (Advanced Vector eXtensions) i ARM SVE (Scalable Vector Extensions) ekstenzijama **nisu vektorski procesori** već samo proširenja SIMD modela, ali ne prave od procesora vektorske procesore. Ove ekstenzije omogućavaju maskiranje određenih SIMD operacija pa izgleda kao da rade sa vektorima različitih dužina, što nije formalno tačno, taj pristup se formalno naziva "predikatski SIMD" i omogućava da za određeni korak "preskočimo" rad jednog od elemenata u zavisnosti od trenutnog stanja i operanda. Tako praktično možemo da maskiramo neki element ili dio funkcionalnosti za određene podatke pa *izgleda* kao vektorski procesor i omogućava bolje performanse, što je često dovoljno dobro u praksi. Kao što smo ranije vidjeli, ako smo u stanju da nešto radimo dovoljno dobro i bez bitnih negativnih efekata, taj pristup postaje dominantan i to se i desilo sa AVX i SVE proširenjima koja su danas izuzetno popularan pristup.

13. Izvršavanje instrukcija van redoslijeda

Iako je u poglavlju 4 opisan klasični proces izvršavanja instrukcija, važno je razumjeti da moderni procesori ne rade na opisan način. Iako se vrlo grubo posmatrano može reći da nema većih odstupanja od osnovne ideje, sama praktična realizacija je bitno drugačija. Osnovni koraci su prikazani na slici 24, dok tačna realizacija u velikoj mjeri zavisi od konkretnog procesora i tipično predstavlja najstrože čuvanu poslovnu tajnu proizvođača.



Slika 24: Primjer izvršavanja instrukcije kod modernih procesora

Osnovna izmjena u odnosu na klasični model je sadržana u postojanju Reorder Buffera (ROB) koji omogućava da procesor ne izvršava instrukcije redoslijedom kojim ih dobija nego redoslijedom koji će na najbolji način opteretiti sve izvršne elemente *bez mijenjanja rezultata obrade*. Dubina ovog bafera je značajna i za savremene procesore iznosi nekoliko stotina unosa (npr. Intel i9 12. generacija ima dubinu od 256 unosa, AMD Zen4 ima dubinu od 320 unosa, dok Apple M1 ima dubinu od približno 620 ali je kod njega u pitanju egzotičniji pristup tako da je teško odrediti tačnu dubinu).

Zašto nam je uopšte potrebno izvršavanje van redoslijeda (Out-of-Order Execution - OOOE)? Zar ne možemo jednostavno da imamo što dublji pipeline i bolje performanse? Ne baš, rizikujemo sve više i čekamo sve duže (vidjeti poglavlje 10). Na primjer, neka nam dolaze instrukcije u "CPU" sa jednim sabiračem i jednim množačem sljedećim redoslijedom:

- $A=A+3$ pa $B=4*8$ - u ovom slučaju možemo da ih izvršimo istovremeno
- $A=A+3$ pa $C=A*4$ - sada ne možemo jer C zavisi od A (i nema jednostavnog rješenja)
- $A=A+3$ pa $B=A*4$ pa $C=C+4$ pa $D=4*8$ - ne možemo da ih izvršavamo paralelno, ali mogli bi da su drugačije složene, npr: A, D, B, C ili A, D, C, B

Rješenje je da presložimo instrukcije tako da ne zavise jedna od druge i da ne koriste iste izvršne jedinice, jer u protivnom čekamo da se stvore uslovi za izvršavanje, a mi ne volimo čekanje. Instrukcije koje dolaze na izvršavanje čuvamo u "Reorder buffer" strukturi, a rezultate upisujemo u

originalnom redoslijedu (jer ne smije izgledati da su izvršene drugim redoslijedom). Zašto ovo ne može kompajler da uradi sam? Ne može uvijek jer ne zna stanje u toku izvršavanja, inače bi mogao da optimizuje kod (što i može i svi moderni kompajleri to i rade ako statički mogu da izračunaju zavisnost).

Da bi mogli da izvršavamo instrukcije van redoslijeda, potrebno je da je zadovoljeno nekoliko uslova:

- Da možemo da pribavljamo i dekodiramo više od jedne instrukcije u paraleli (ovo nije problem jer gotovo svi moderni procesori to mogu i to 2 do 8 instrukcija)
- Treba nam i predikcija grananja (predviđanje petlji) jer u linearnom kodu jednostavno nema dovoljno instrukcija da ostvarimo bitno bolje performanse. Većinu vremena provodimo u petljama, samo treba znati kad je kraj petlje da ne računamo više nego što je potrebno.
- Moramo moći preimenovati registre da bi se riješili problema sa parametrima (jer je sve u registrima). Na ovaj način imamo mnogo više fizičkih registara (praktično radimo sa blokovima skupova registara) i preimenovanje se svodi na tabelu pokazivača na fizičke registre, tako da svaka instrukcija radi sa svojom kopijom registara. Na ovaj način čak ubrzavamo i neke operacije koje nemaju veze sa redoslijedom izvršavanja, npr. kad uradite MOV X0, X1 - svede sa na preimenovanje (što je brzo i lako).
- Moramo i **spekulativno izvršavati** - ovo je veliki izvor i performansi i beskrajne rijeke suza. Izvršavamo instrukcije za koje još ne možemo znati da će se izvršiti (možda dobitak), ali nam je bolje i to nego da čekamo bez izvršavanja ičega (garantovan gubitak), a ako pogriješimo, jednostavno odbacimo *te* rezultate (zahvaljujući preimenovanju registara) *bez ikakvih negativnih posljedica*. Brojni sigurnosni propusti zbog tog "bez ikakvih negativnih posljedica" (Spectre, Meltdown, Foreshadow, ...) jer se ispostavlja da je sve u životu kompromis i da bolje performanse ne plaćamo samo brojem tranzistora i količinom utrošene električne energije, nego i potencijalnim sigurnosnim problemima.

Koliko tačnost predviđanja grananja utiče na izvršavanje i performanse?

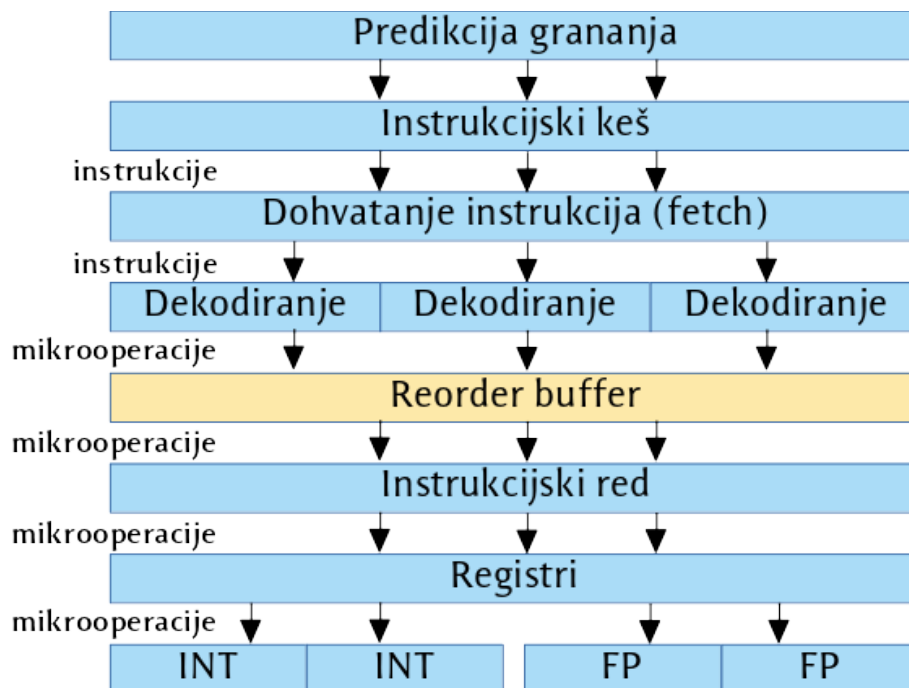
Neka CPU ima bafer za 20 instrukcija i neka je grananje u prosjeku na svakih 10 instrukcija, što znači da u baferu imamo 20 grananja. Ako je tačnost predviđanja 90%, to znači da je ukupna tačnost $0,90^{20} = 0,12$. Tačnost 95% $\Rightarrow 0,95^{20} = 0,36$. Tačnost 98% $\Rightarrow 0,98^{20} = 0,67$. Tačnost 99% $\Rightarrow 0,99^{20} = 0,82$.

Svaki put kad promašimo moramo da "počistimo" i često da ispraznimo bafer (jer smo otišli na pogrešan niz instrukcija i uzalud ih izvršavali). Ovakva tačnost nije moguća bez napredne analize izvršavanja petlji koja predstavlja "tamnu magiju" od koje uveliko zavise performanse procesora. Apple M1 procesor ima gotovo dva puta dublji bafer, a vjerovatno i toliko bolju predikciju grananja u odnosu na savremenike iz Intel-a i AMD-a. Vodite računa da nam je na 40 grananja potrebna tačnost od 99,5% za gotovo iste vrijednosti kao 20 grananja i tačnost od 99%.

Sve analizirano potpuno ignoriše postojanje prekida i drugih nepredviđenih događaja što praktično garantuje da su stvarne realizacije čak i mnogo kompleksnije u odnosu na opisani, već kompleksan, pristup.

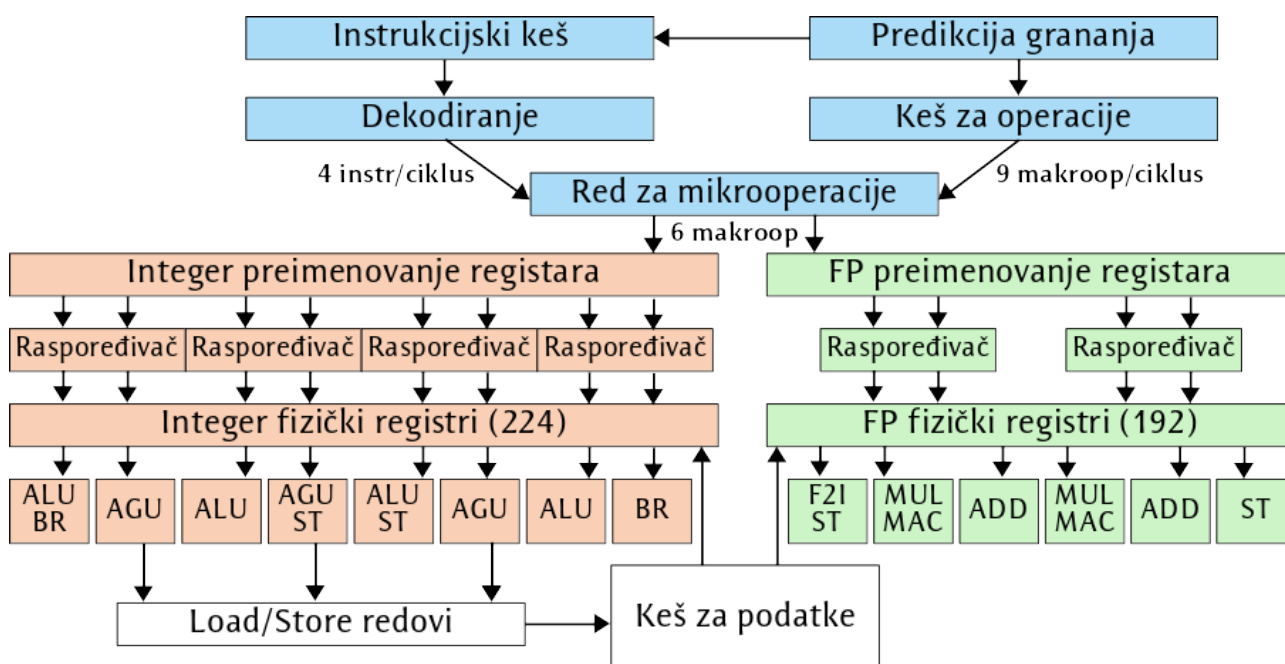
14. Arhitektura jezgra modernog procesora

Na slici 25 je prikazana nešto realnija arhitektura jezgra modernog procesora koja uključuje višestruke jedinice za dekodovanje, ako i za INT i FP operacije. Još realniji prikaz uključuje i dodatno pre-dekodovanje koje na izlazu daje makro-operacije koje mogu činite dvije mikrooperacije, ali je u stvarnosti sve još mnogo komplikovanije i zavisi od konkretnog jezgra.



Slika 25: Realnija arhitektura jezgra modernog procesora

Na slici 26 je kao primjer data pojednostavljena arhitektura jednog AMD Zen 4 jezgra. Vidimo da je cijeli dio zadužen za obradu prije samog izvršavanja prilično kompleksan, dok sami izvršni blokovi posjeduju višestruke raspoređivače mikrooperacija i brojne izvršne elemente (BR Branch, AGU Address Generation Unit, F2I Float-to-Integer, MAC Multiply-ACcumulate, itd).

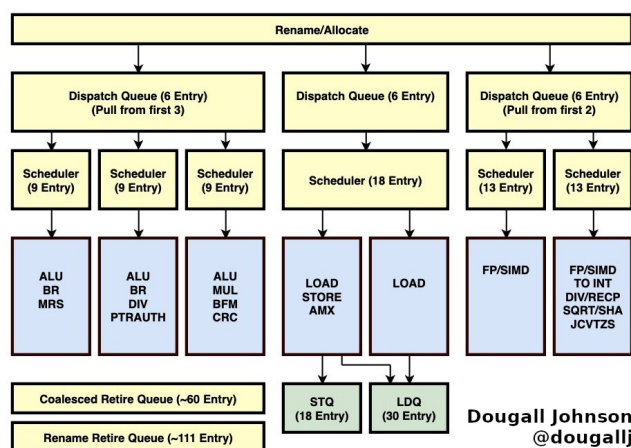


Slika 26: Pojednostavljena arhitektura AMD Zen 4 jezgra

Ono što možemo da vidimo je da većina izvršnih elemenata ima višestruke funkcije (u jednom trenutku izvršavaju samo jednu od njih), što omogućava veću fleksibilnost u radu (i kompleksniji izbor redoslijeda izvršavanja iz reda instrukcija). Vidimo i da je broj fizičkih registara mnogo veći nego logičkih kojima imamo pristup.

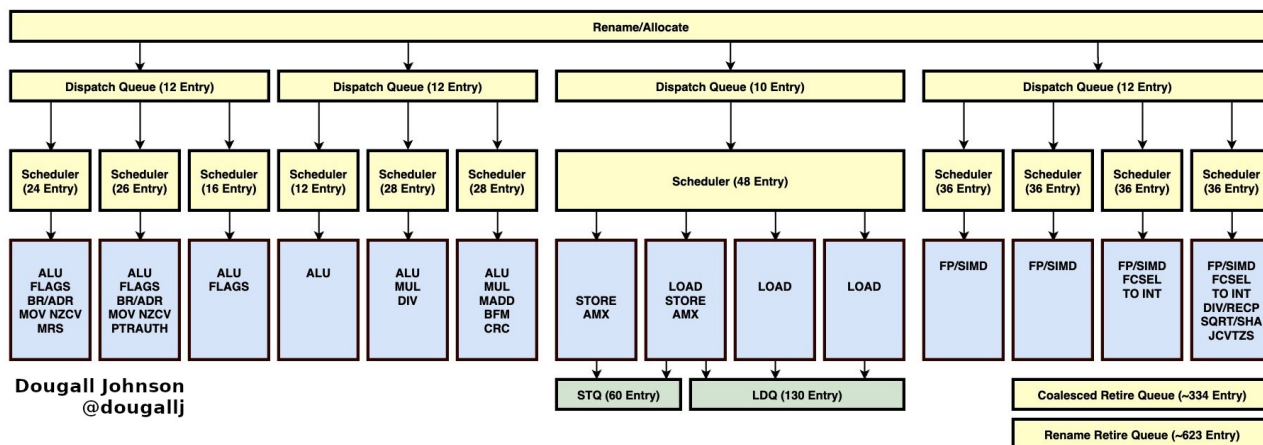
Trenutno je apsolutni rekorder po kompleksnosti Apple M1 procesor koji ima dvije vrste jezgara: Icestorm (jednostavnija efikasna jezgra) i Firestorm (kompleksnija i brža). Na slikama 27 i 28 su prikazane vjerovatne organizacije napravljene od strane Dougalla Johnsona.

Icestorm



Slika 27: Apple M1 Icestorm jezgro - vjerovatna organizacija

Firestorm



Slika 28: Apple M1 Firestorm jezgro - vjerovatna organizacija

Vidimo da su i "jednostavna" jezgra prilično kompleksna, a brza jezgra krasi sljedeći atributi (vodite računa da su ovo jezgra originalno razvijena za mobilne uređaje male potrošnje električne energije): 8 mikrooperacija po ciklusu, 1 grananje po ciklusu, ~620 dubina reorder bafera (ali je komplikovano), ~390 integer registara (fizičkih), ~432 floating point registara (fizičkih), ~128 flags registara (fizičkih), ~144 grananja u obradi, ~130 dubina LOAD bafera i ~60 dubina STORE bafera. Iz navedenog je očigledno da je danas teško govoriti o bitnim razlikama između procesora koji se koriste u prenosnim uređajima i procesorima za servere jer se većina razlika svodi na više jezgara, veće količine keš memorije i, vjerovatno najbitnije, količini električne energije koju imamo na raspolaganju (kao i količini toplote koju smijemo otpustiti u okolinu bez da se procesor istopi).

15. Primarna memorija

Iako se procesor posmatra kao napredak koji je izazvao revoluciju i stvorio računarstvo (a samim tim ga i najviše proučavamo na ovom predmetu), činjenica je da je ponovno upisiva memorija jednako zaslužna. Bez takve memorije, računari bi bili osuđeni na fiksne funkcije bez mogućnosti programiranja u smislu kakvim ga mi danas shvatamo. Ni sami procesori ne bi bili ni izbliza slični današnjim jer su gotovo sve funkcionalnosti vezane za memorijske lokacije unutar procesora, kao što je objašnjeno na prethodnim predavanjima. Kada govorimo o memoriji, u ovom poglavlju ćemo se ograničiti na primarnu radnu memoriju koja se ne nalazi u procesoru, dok će keš memorija biti obrađena zasebno. Memorija je izuzetno važna jer sadrži i program koji procesor izvršava i podatke koje obrađuje, a predstavlja i jedini način komunikacije procesora sa drugim komponentama (jer im pristupa i posmatra ih kao memoriju).

Kao što vam je već poznato, sama digitalna memorija digitalnog računara je organizovana kao skup **bita**, pri čemu bit može da čuva samo vrijednost 0 ili 1. U drugim poglavljima će biti više riječi o načinu čuvanja sadržaja bita, ali i o tome da li je neophodno da jedan elementarni sklop može čuvati samo jedan bit ili možda i više. Memorija je podijeljena u **ćelije** (lokacije) pri čemu svaka ćelija čuva dio informacije (svaka ćelija sadrži isti broj bita) i čija lokacija je određena brojem koji se naziva **adresa**. Kao što znate, mi rijetko radimo sa bitima i mnogo češće koristimo skup od osam bita koje nazivamo **bajt** (byte, oktet). I bajtove grupišemo u **riječi** (word) pri čemu je dužina riječi vrlo važna za svaki računar jer ona definiše veličinu opštih registara i bitnost računara (32-bitni ili 64-bitni). Kako postoji više od jednog bajta u riječi, očigledno je da ih je moguće složiti na više načina tako da je došlo do pojave različitih pristupa od strane različitih proizvođača. U zavisnosti od toga da li će bajtovi nekog hipotetičkog 32-bitnog računara biti poredani sekvencom 0,1,2,3 ili 3,2,1,0, govorimo o **big-endian** (IBM z) ili **little-endian** (Intel, ARM AArch64, RISC-V, POWER, ...) sistemima. Da bi stvar bila još malo komplikovanija, ARM procesori mogu da rade kao big-endian na dva načina. Kako sve to nije još dovoljno komplikovano, vrijedi napomenuti da postoje procesori koji mogu da rade i kao big- i kao little-endian, a čak i kod striktno little-endian procesora (Intel Core novijih generacija) postoje instrukcije koje će automatski izvršiti konverziju između big- i little-endian podataka (MOVBE).

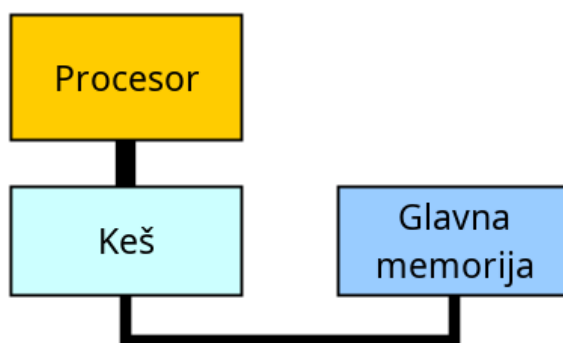
Velika većina implementacija primarne (glavne, radne) memorije je realizovana kao privremena (*volatile*) memorija što znači da je neophodno obezbijediti kontinualno napajanje električnom energijom da bi memorija čuvala podatke i bila upotrebljiva. O alternativa će više riječi biti kasnije. Kada danas govorimo o primarnoj memoriji, često je nazivamo RAM (*random access memory*) što označava memoriju sa slučajnim pristupom ili direktno adresibilnu memoriju - moguće je pristupiti bilo kojoj memorijskoj lokaciji bez obzira na to kojoj smo lokaciji prethodno pristupali (alternativa je sekvencijalni pristup). Iako nije neophodno, danas su sve primarne memorije (od interesa za nas) RAM i privremenog tipa. Dakle, ROM (*read only memory*) nije alternativa RAM-u, nego privremenoj memoriji, ali je ROM neophodan dio računara jer računar koji sadrži samo RAM pri uključivanju nema šta da izvršava, što je problem koji se rješava postojanjem ROM-a koji sadrži inicijalni program.

Primarna memorija je direktno ili indirektno povezana na CPU putem memorijske magistrale (*memory bus*) koja se sastoji od dvije magistrale - jedne za adrese (*address bus*) i druge za podatke (*data bus*). Memorijom upravlja **MMU** (*memory management unit*) koji je zadužen i za rad sa virtuelnom memorijom, keš memorijom, upravljanje memorijskim magistralama, itd.

16. Keš memorija

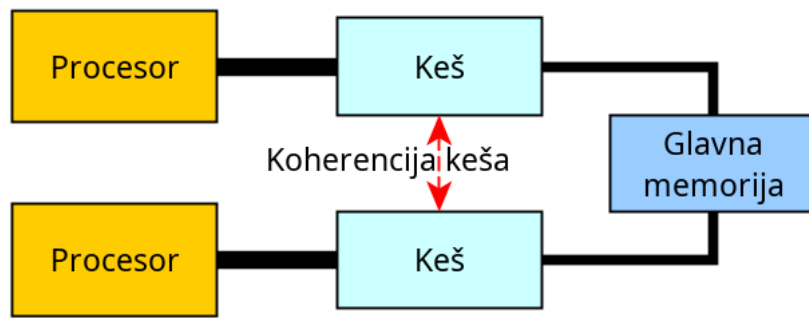
Primarna memorija je brza, ali je procesor brži. U zavisnosti od konkretnog slučaja, procesor može biti i mnogo brži od memorije, što je priličan problem jer se svi programi i svi podaci nalaze u primarnoj memoriji i procesor ne može da radi ništa dok ih ne učita. Već smo obradili protočnost i superskalarnost, kao i postojanje više jezgara u jednom procesoru, što snažno sugerije da, iako memorije vremenom postaju brže, ovaj problem nije lako rješiv. To je laž. Ovaj problem je lako rješiv i svakako je moguće napraviti memoriju koja će raditi istom brzinom kao i procesor, problem je što bi takva memorija bila preskupa i neisplativa. Sile ekonomije su neumoljive i mi moramo da pronađemo rješenje koje će sa jedne strane omogućiti procesoru da "uvijek" ima na raspolaganju sve instrukcije i podatke koji mu trenutno trebaju, a s druge strane nećemo otjerati firmu u bankrot.

Pitanje da li je bolje imati manje brže memorije ili više sporije memorije ima jednostavan odgovor - zašto ne obje? Mala, brza memorija se naziva **keš** (*cache*) i osnovni zadatak je da ona čuva najčešće korišćene podatke i/ili instrukcije sa idejom da ako smo ih često koristili u prošlosti, postoji solidna šansa da ćemo ih koristiti i u budućnosti pa je dobro što se nalaze u brznoj memoriji iz koje procesor može efikasno da ih pročita. Iako je prva riječ u skraćenici RAM - random, to ne znači da je pristup memoriji potpuno nasumičan. Postoje pravilnosti u pristupu (npr. većina instrukcija se čita redom uz izuzetak grananja, ali i tu možemo da predvidimo koja će grana biti odabrana, vektorske ili matrične operacije pristupaju elementima na dobro poznat način, itd). Jedan od efikasnih načina da ubrzate izvršavanje vašeg koda je da optimizujete upotrebu keš memorije. Kada govorimo o tome da li je podatak pronađen u kešu ili ne, mi govorimo o **cache hit** i **cache miss** događajima koji su nam važni jer na osnovu ovih vrijednosti možemo pravilno projektovati i optimizovati keš. To je važno jer rad kroz keš unosi kašnjenje za svaki promašaj (npr. umjesto da odmah nešto učitamo iz memorije, mi prvo provjeravamo da li postoji u kešu, a to nas košta).



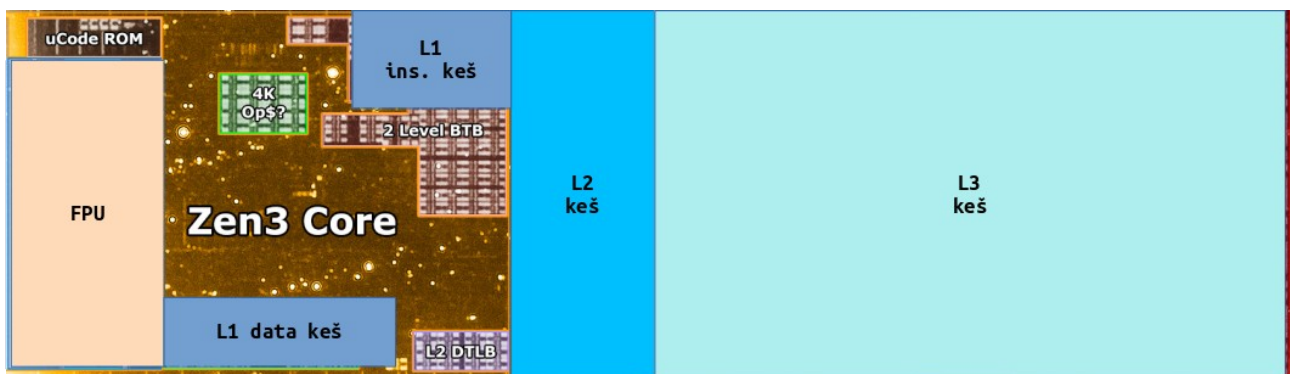
Slika 29: Keš memorija

Sav problem leži u dva naizgled jednostavna izbora: šta i kako staviti u keš i šta i kad izbaciti iz keša. Postoje brojne strategije i nije moguće definisati šta je najbolji izbor u opštem slučaju. Kada govorimo o punjenju keša, očigledno je da je neophodno ono što pohranimo u keš u nekom trenutku upisati i u memoriju. Prvi pristup se naziva **write-through** i funkcioniše tako što sve što upisujemo u keš odmah upisujemo i u memoriju. Drugi pristup je **write-back** i upisi u memoriju se izvršavaju tek kad podatak izbacujemo iz keša. Naravno da postoje i hibridna rješenja, a moramo da vodimo i računa o tome šta se dešava sa podacima u našem kešu kada neka periferija (ili drugi procesor) izmijene podatke u radnoj memoriji koje mi držimo u kešu. Keševi koji mogu da sinhronizuju izmjene između više entiteta se nazivaju koherentni (*coherent cache*).



Slika 30: Koherencija keša

Jedna zanimljivost vezana za keš je upotreba odvojenih keševa za instrukcije i podatke čime, praktično, pravimo računar po Harvard arhitekturi. Gotovo svi moderni procesori imaju odvojene keševe, ali i hijerarhiju keševa. Naime, isti princip koji postoji u odnosu brzina-veličina između keša i primarne memorije, postoji i između nivoa keša. Što je keš brži i bliži procesoru, to je skuplji i, samim tim, manjeg kapaciteta. Na taj način dolazimo do, danas uobičajenih, troslojnih keševa. Na primjer, AMD Zen 3 procesori imaju L1 keš od 32KB za instrukcije i 32KB za podatke (po jezgru), L2 keš od 512KB (po jezgru) i L3 dijeljeni keš od 32MB za osam jezgara. Tek kada instrukcije ili podaci ne mogu biti pronađeni u kešu, procesor će pristupiti glavnoj memoriji. Ekstrem po pitanju količine L1 keša je Apple M1 procesor koji ima 192 KB L1 instrukcijskog keša i 128 KB data keša za svako od *high-performance* jezgara (ukupno za svih 8 jezgara 2 MB L1 keša). Kod određenih procesora postoji i L4 keš većeg kapaciteta (npr. 64 ili 128 MB), najčešće u slučajevima kada je sa procesorom kolociran i grafički procesor, te se na taj način poboljšavaju performanse. Na slici 31⁹ je prikazana gruba anotacija fotografije jednog AMD Zen 3 jezgra. Na slici je očigledno da u stvarnosti kupujemo keš na koji je dodano jezgro, a ne jezgro koje ima i keš.



Slika 31: Gruba anotacija fotografije jednog AMD Zen 3 jezgra

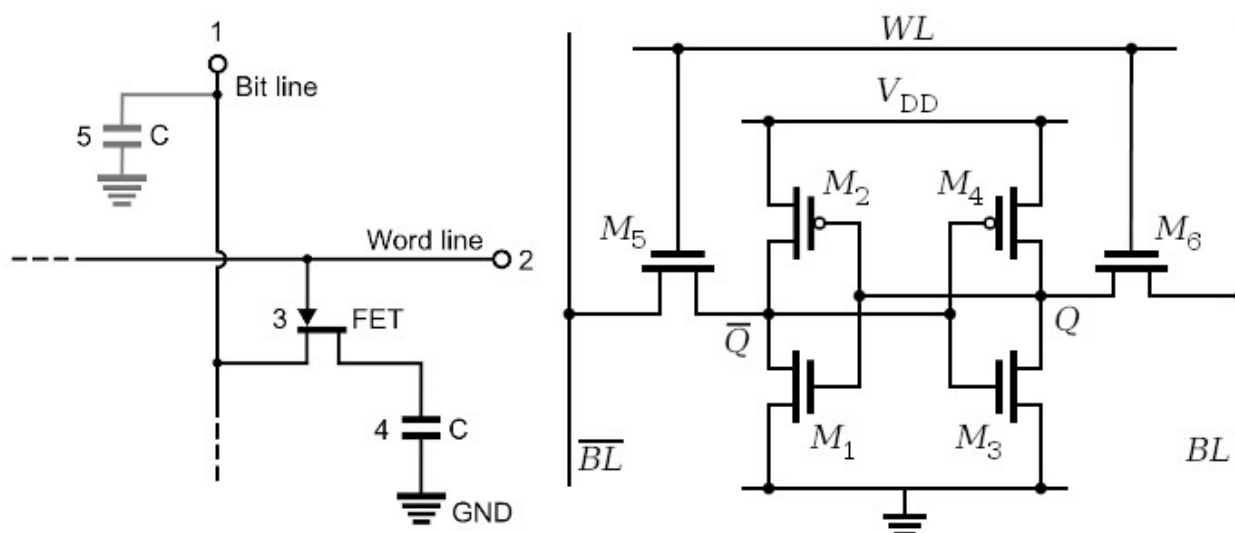
Ako sada pogledamo sliku 16 možemo da primijetimo da je isti princip koji se koristi kod višeprocorskih sistema prisutan i kod multi-računarskih sistema (slika 17) ali i unutar jednog procesorskog modula i procesora. Na drugim predmetima će biti obrađeni problemi potpuno drugačije prirode (web aplikacije, serveri baza podataka) koje rješavamo istim pristupom - keširanjem.

Naravno, nisu samo L1, L2, L3, L4 keševi - keševi. Struktura pod nazivom Translator Lookaside Buffer (TLB) čuva nedavne translacije virtuelna-fizička memorija i predstavlja vrstu keša, kao i μop (micro-op) keš koji kešira mikrooperacije (izlaz dekodera instrukcija), dok prediktori grananja (poglavlje 13) takođe koriste poseban "keš" u radu (history buffer).

⁹ https://www.reddit.com/r/Amd/comments/jqjg8e/quick_zen3_die_shot_annotations_die_shot_from/

17. Statička i dinamička memorija (SRAM i DRAM)

Kada govorimo o memoriji koja se koristi u računarima, generalno je možemo podijeliti u dvije velike grupe: statička i dinamička memorija. Statička memorija sa slučajnim pristupom (**SRAM** - static random access memory) je privremena memorija koja koristi flip-flobove za pohranu jednog bita. SRAM, kao što je vidljivo na slici 32¹⁰, ima relativno kompleksnu strukturu što znači da je za izradu potrebno više elemenata i da zauzima više prostora. Na slici je prikazana standardna organizacija sa šest tranzistora, ali su moguće i druge varijante (npr. četiri tranzistora i dva otpornika, itd) koje mogu zauzimati manje prostora po cijenu veće kompleksnosti proizvodnog procesa. Kako sve navedeno uzrokuje visoku cijenu SRAM-a, najčešće se koristi za memoriju skromnog kapaciteta (reda veličine stotina kilobajta) koja se nalazi u procesoru i koja mora da radi istom brzinom kao i sama logička kola u procesoru (registri, keš, itd). Potrošnja električne energije zavisi i od toga koliko često i na koji način vršimo pristup.



Slika 32: DRAM (lijevo) i SRAM (desno) ćelije

Alternativa SRAM-u je **DRAM** (*dynamic random access memory*) koji ima mnogo jednostavniju strukturu koja je vidljiva na slici 32. Postoji više varijanti, a prikazana je organizacija zasnovana na jednom tranzistoru i jednom kondenzatoru, iako su mogući čak i pristupi zasnovani na samo jednom tranzistoru. Kao i SRAM i DRAM je privremena radna memorija i neophodno je obezbijediti električnu energiju za funkcionisanje. Osnovna funkcionalna razlika u odnosu na SRAM leži u činjenici da DRAM vremenom zaboravlja sadržaj (posljedica jednostavne strukture), a i samo čitanje je destruktivno. Da bi DRAM uopšte mogli koristiti na smislen način, neophodno je da ga periodično osvježavamo za što se koriste eksterna kola koja unose kompleksnost i smanjuju efikasnost ovog tipa memorije. Kako je neophodno konstantno osvježavanje, a i sam rad je kompleksniji, DRAM ima nezanemarljivu potrošnju energije u radu koja je značajna čak i ako ga uopšte ne koristimo u radu. To je razlog zašto apsolutno želite da koristite svu raspoloživu memoriju sistema, ako ništa drugo, onda kao keš za sporiju sekundarnu memoriju. Međutim, sve te negativne strane su višestruko kompenzovane mnogo manjom cijenom (i u novcu i u prostoru) koju ovaj pristup donosi tako da je praktično sva radna memorija računara DRAM tipa. Čak se i u slučajevima veće količine procesorskog keša koristi DRAM (embedded DRAM - eDRAM), kod Intel procesora kao L4 keš, a kod određenih IBM z procesora i na nižim nivoima.

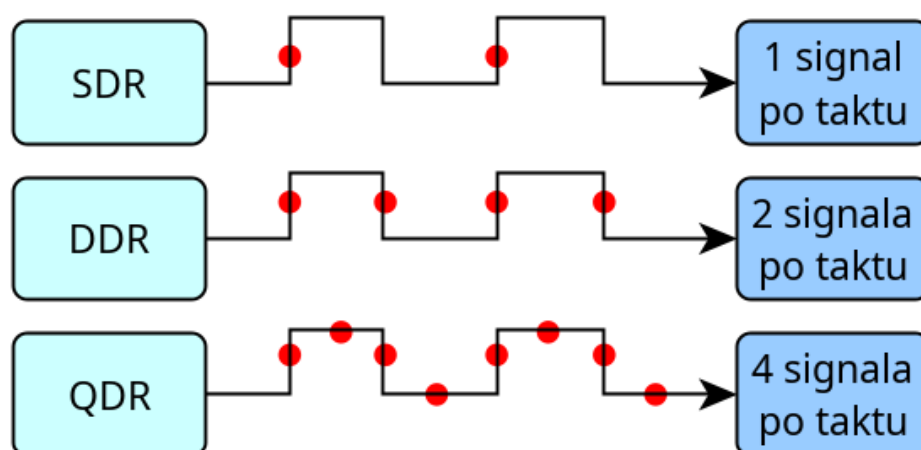
¹⁰ [https://en.wikipedia.org/wiki/File:DRAM_Cell_Structure_\(Model_of_Single_Circuit_Cell\).PNG](https://en.wikipedia.org/wiki/File:DRAM_Cell_Structure_(Model_of_Single_Circuit_Cell).PNG)

18. DRAM moduli

Prve verzije DRAM memorije su radile asinhrono, ali su vremenom u potpunosti potisnute i danas praktično sva memorija radi kao sinhrona (SDRAM - *synchronous dynamic random-access memory*) što podrazumijeva da postoji neki eksterni izvor takta i da se sve operacije izvršavaju u skladu s njim. Kada govorimo o načinu fizičkog pakovanja memorijskih čipova na kartice, moguća su dva osnovna načina: sve sa jedne strane i dvostrano. Jednostrano pakovanje se naziva SIMM (*single in-line memory module*) i svi kontakti se nalaze na jednoj strani pločice ili su ponovljeni sa obje strane (ali je uvijek riječ o strogom preslikavanju istih kontakata). Ovaj način pakovanja memorije danas ima samo istorijski značaj jer je zamijenjen upotrebom DIMM (*dual in-line memory module*) standarda kod kojeg su električni kontakti sa različitih strana različiti i imaju svi svoju funkciju.

Originalna verzija DRAM-a je SDRAM (ili SDR) i memorija može da primi jednu komandu i izvrši jedan transfer podataka po svakom ciklusu. Specifičnost ove memorije je da se upisi vrše u istom ciklusu kada memorija dobije komandu i podatke, ali čitanje kasni dva do tri ciklusa iza komande.

Da bi poboljšali performanse, umjesto da radimo samo na uzlaznu ivicu signala, možemo da čitamo/pišemo i na silaznu ivicu i na taj način poboljšamo performanse (**DDR** - *double data rate*). Ako radimo sa dva takta, pri čemu drugi kasni za prvim za jednu četvrtinu takta, i radimo na obje ivice oba takta - dobijamo QDR (*quad data rate*). Razlog za upotrebu **QDR** nije uvijek da se poveća propusni opseg već i da se riješi problem zauzeća magistrale na taj način što se sada mogu raditi upisi i čitanja u paraleli (ako magistrala podržava takav rad).



Slika 33: Prikaz rada SDR, DDR i QDR

DDR memorija ima više generacija (DDR1 - originalno DDR ali se stavlja i 1 da bi izbjegli konfuziju, DDR2, DDR3, DDR4) pri čemu svaka nova generacija donosi bolje performanse povećanjem memorijskih jedinica kojim radimo (umjesto dvije riječi, radimo sa minimalno četiri, itd) i podizanjem takta (sa početnih 133-200 MHz kod DDR1, preko 200-400 MHz i 400-800 MHz, do 800-1600 MHz kod DDR4). Da bi kontrolisali potrošnju električne energije (i oslobođenu toplotu) DDR4 koristi niži napon (1,2 V) u odnosu na starije verzije (1,5 V). DDR5, koji je predstavljen 2020. godine, donosi još niži napon (1,1 V) i još brži takt.

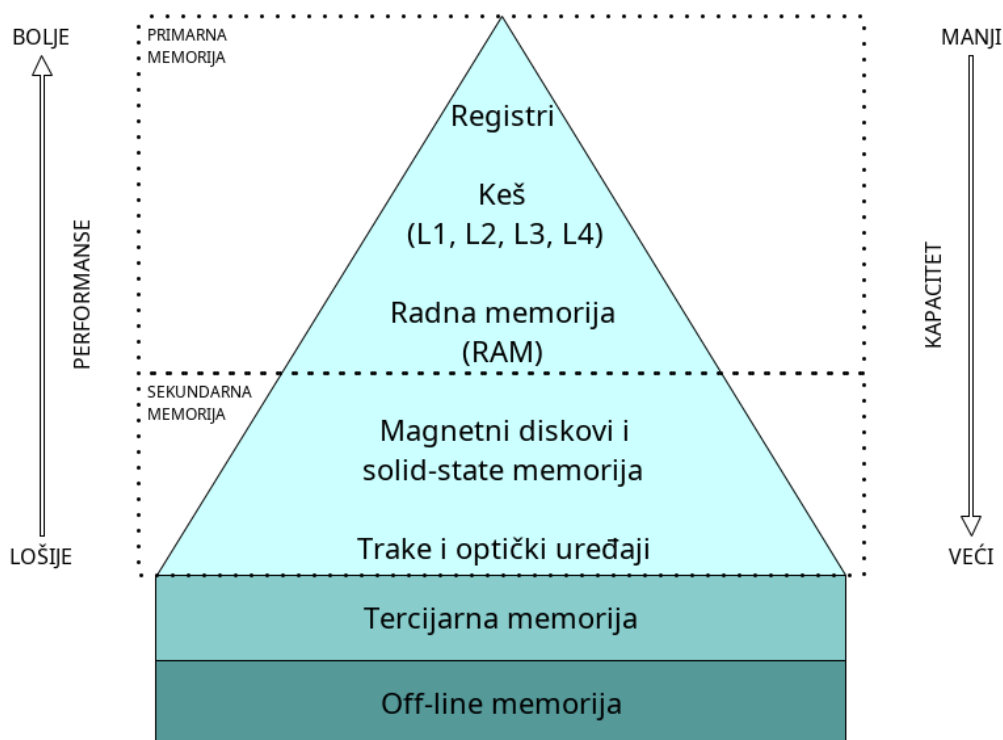
Osim opisanih tehnologija, pri komunikaciji sa memorijom nismo ograničeni na jedan kanal. Ukoliko sistem posjeduje dva memorijska modula (i sve komponente podržavaju ovaj način rada) moguće im je nezavisno i istovremeno pristupati. Obično su parovi slotova za memorijske module posebno označeni na matičnoj ploči i neophodno je da su oba modula u uparenim slotovima ista (isti kapacitet, poželjno i brzina). Trenutno aktuelni procesori vrhunskih performansi i Intela (npr. Core i7/i9 i Xeon) i AMD-a (npr. ThreadRipper i Opteron) podržavaju rad sa četiri kanala. Procesori namijenjeni za upotrebu u višeprocorskim serverima mogu podržavati veći broj kanala, na primjer osam (npr. Intel Scalable Xeon i AMD EPYC).

Memorije koje se koriste na grafičkim karticama su danas dominantno DDR tipa i obično dijele dosta zajedničkog sa prethodno opisanim memorijama, ali postoje specifičnosti. Na primjer, kod grafičkih kartica je moguće kvalitetnije hlađenje što znači da su termalna ograničenja relaksirana te memorija može da radi na većem taktu i obezbijedi bolje performanse. Takođe, kako tačno znamo gdje i kako će memorija biti korišćena, možemo je optimizovati na načine koji nisu izvodivi kod generičkih RAM modula. Najčešći korišćeni standardi su GDDR3 (zasnovan na DDR2), GDDR5 (zasnovan na DDR3, ali koristi tri takta, jedan za komande i dva za podatke, itd), GDDR5X (koji može da radi i u DDR i u QDR režimu i koristi se u npr. Nvidia 1080 i skupljim karticama), GDDR6 (QDR, Nvidia 2080) i GDDR6X (QDR, kompleksnija signalizacija, manje potrošnja po bitu).

Alternativa novijim GDDR standardima je upotreba HBM (*high bandwidth memory*) tehnologije koja RAM pločice postavlja vertikalno jednu na drugu (i memorijski kontroler) dok su veze raspoređene po površini (jer nismo više ograničeni na ivice). Širina magistrale je mnogo veća nego kod GDDRa, na primjer, GDDR5 omogućava 512 bita, dok je kod HBMA riječ o 4096 bita ili čak i više. Kada se sve sabere i oduzme, dolazimo do teoretskih vrijednosti od preko 400 GB/s (nije samo teoretski, postoje komercijalni proizvodi poput Nvidia Tesla P100/V100 ili AMD XFX kartica).

19. Hijerarhijska organizacija memorije

U prethodnom tekstu smo obradili pretežno primarnu, privremenu memoriju i to u vidu registara (i drugih internih struktura u procesoru), više nivoa keš memorije (takođe u procesoru), te radne memorije (RAM). Kako je riječ o privremenoj memoriji, pri isključivanju ili ponovnom pokretanju računara ta memorija ne čuva prethodni sadržaj te ne neohodno postojanje i malog dijela trajne memorije neophodne za inicijalizaciju rada računara (npr. BIOS, UEFI, firmware i slični koji služe prvenstveno da učitaju drugi kod i predaju mu kontrolu nad izvršavanjem). Zajednička osobina svih ovih vrsta memorije je da su **direktno adresibilne od strane procesora**. Ovo vrijedi i za prethodno pomenutu trajnu memoriju, posebno kod jednostavnijih uređaja koji sav kod često izvršavaju iz trajne memorije bez potrebe da ga kopiraju u radnu memoriju.



Slika 34: Hijerarhijska organizacija memorije

Pomenuti pristup nije praktičan kod kompleksnijih uređaja iz mnogo razloga: kod se često mijenja, trajna memorija je relativno spora i ograničenog kapaciteta, operativni sistem često treba da ima kontrolu na kodom i načinom izvršavanja, itd. Standardno rješenje je postojanje sekundarne (trajne, eksterne, pomoćne) memorije koja čuva kod i podatke i u slučaju restartovanja računara ili prestanka napajanja električnom energijom. Očigledno je da ta memorija (**sekundarna**) treba da ima **veći kapacitet** od radne memorije, ali i da može imati **lošije performanse** jer se kod ne izvršava direktno iz nje što znači da ova memorija (sekundarna) **ne mora biti direktno adresibilna od strane procesora**. Ispod sekundarne memorije se mogu nalaziti "slojevi" tercijarne i off-line memorije koje se formalno mogu posmatrati kao posebni slučajevi sekundarne memorije, ali su način i razlozi upotrebe drugačiji.

Konkretno govoreći, najbrža memorija su registri koji rade brzinom procesora uz minimalno kašnjenje i kapacitet reda veličine KB, nakon njih keš memorija koja ima kapacitet od nekoliko stotina KB do nekoliko desetaka MB i kašnjenje od nekoliko do nekoliko desetaka ciklusa, te radna

memorija kapaciteta reda veličine GB, propusnog opsega reda veličine desetak GB/s i kašnjenje reda veličine 10 ns.

Kada govorimo o sekundarnoj memoriji, kapaciteti su reda veličine TB i kašnjenja reda veličine ms za magnetne i solid-state uređaje, te desetaka TB i kašnjenja reda veličine sekundi, čak i minuta, za trake i uređaje koje koriste optičke medije. Kako je rastao kapacitet, a padala cijena po jedinici memorije magnetnih uređaja, dolazi do opadanja popularnosti i praktične upotrebljivosti traka i optičkih uređaja. Danas je vrlo teško povući jasne granice po pitanju kapaciteta između magnetnih diskova, SSD uređaja i magnetnih traka. Svi se kreću u istim grubim okvirima do nekoliko desetaka TB (18 TB za trake, trenutno 20 TB za HDD i 100 TB za SSD). Razlike su, prije svega, u prirodi rada. Klasični HDD uređaji nude nisku cijenu po TB i prihvatljive performanse, SSD velikog kapaciteta su izuzetno skupi, ali su im i performanse proporcionalno izuzetne, dok su pojedinačne trake relativno spore, rade sekvencijalno, ali su uklonjive i koštaju nekoliko puta manje nego HDD uporedivog kapaciteta i imaju mnogo duži životni vijek što ih čini pogodnim za dugoročno čuvanje podataka i bekapa.

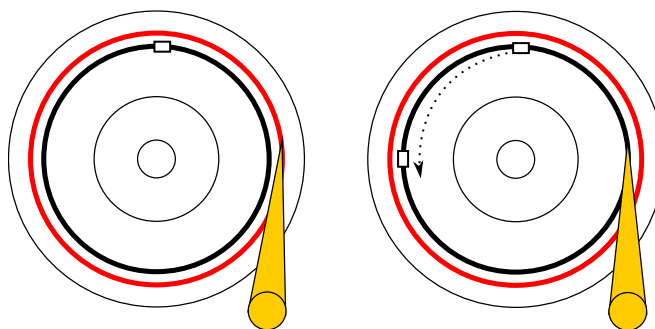
Tercijarna memorija se obično može podijeliti u dvije široke kategorije: **nearline** i **cloud**. Nearline sistemi tipično obuhvataju sisteme koji nisu uvijek aktivni i dostupni ali se mogu relativno brzo aktivirati i učiniti dostupnim. Moguća rješenja se kreću od sistema za pohranu koji se sastoje od velikog broja diskova koji većinu vremena provode u "idle" stanju (praktično isključeni svi mehanički i pokretni dijelovi, najčešće i velika većina elektronike) i iz kog se mogu "probuditi" u roku od nekoliko sekundi, kao i od robotskih sistema koji automatski upravljaju "bibliotekom" traka i na taj način omogućavaju velike kapacitete uz vrijeme aktiviranja od nekoliko desetaka sekundi. Cloud bazirani sistemi su potpuno drugačije namjene, nude vrlo fleksibilnu pohranu podataka i praktično gotovo beskonačan kapacitet (proširiv po potrebi) uz relativno malo kašnjenje, ali i vrlo skroman propusni opseg koji direktno zavisi i od brzine veze prema internetu, ne samo do internet provajdera, nego do provajdera usluge pohrane podataka. Na primjer, čak i ako imate 1 Gbps link rezervisan za ovu namjenu, efektivna brzina je približno 100 MB/s što znači da je za prenos jednog terabajta potrebno gotovo tri sata. Velika prednost je mogućnost postojanja geografske distribucije (u slučaju prirodnih katastrofa) i upotreba od strane više poslovnica, koje takođe mogu biti geografski distribuirane. Cijena nije nezanemarljiva i često se troškovi računaju ne samo po iznajmljenom kapacitetu, nego i po količini prenesenih podataka.

Off-line podaci, striktno govoreći, nisu dio hijerarhije, ali su bitni za razumijevanje kompletne slike. Često je poželjno, čak potrebno, da određeni podaci **ne budu pod kontrolom sistema** na kojem su zapisani u trajnu memoriju. Ideja je da se podaci zapisuju u sekundarnoj ili tercijarnoj memoriji i nakon toga fizički uklanjaju na takav način da to nije moguće izvršiti automatski bez fizičke ljudske intervencije. Očigledna primjena je za pravljenje rezervnih kopija podataka koje se čuvaju na udaljenim lokacijama i koriste u slučaju potrebe za rekonstrukcijom podataka (jer su podaci oštećeni višom silom, malicioznim aktivnostima, itd). Osim navedenog, na ovaj način je moguće izvršiti i prenos velike količine podataka na udaljene sisteme u određenim slučajevima (npr. za prenos 100 TB podataka je potrebno oko 11 dana pomoću 1 Gbps namjenskog linka ili par sati vožnje kutije sa 10 diskova od po 10 TB).

20. Magnetna memorija

Magnetna memorija je u sekundarnoj memoriji najčešće predstavljena kroz hard diskove (HDD - hard disk drive) i uređaje sa trakama (TD - tape drive).

HDD radi po principu rotirajućih ploča sa magnetnim medijem koji je podijeljen u koncentrične staze. Da bi pročitali ili upisali neki podatak, potrebno je prvo glavu za čitanje/upis pozicionirati iznad odgovarajuće staze, a nakon toga i sačekati da se rotiranjem odgovarajuće mjesto pojavi ispod aktivnog dijela glave (vrh). Proces je prikazan na slici 35 pri čemu je crvena staza staza na kojoj se trenutno nalazi glava, crna staza je staza na kojoj se nalazi željeni podatak koji je predstavljen pravougaonikom. Ukoliko HDD čita ili piše podatke u dugačkoj sekvenci, tako da su svi podaci smješteni na istoj stazi ili na određenom broju uzastopnih staza, performanse mogu biti vrlo dobre i moguće je ostvariti brzinu prenosa podataka od nekoliko stotina megabajta u sekundi. Ova veličina zavisi direktno od brzine rotacije diska, a standardne brzine se izražavaju u broju okreta u minutu (rpm) i uobičajene vrijednosti su 4200 za štedljive diskove, 5400 i 7200 za standardne diskove, 10.000 i 15.000 za brze serverske diskove.

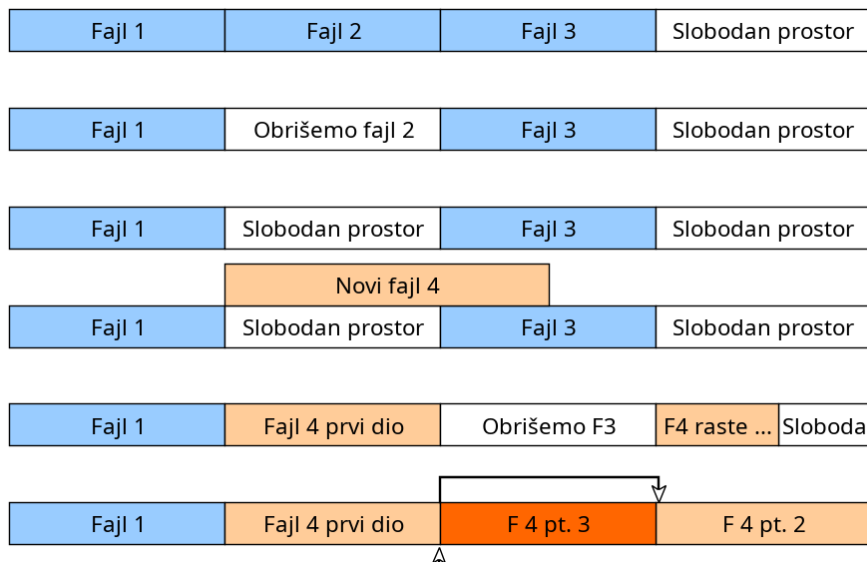


Slika 35: Primjer rada HDD-a

Kako je HDD, prije svega, mehanička komponenta (jer u potpunosti zavisi od mehaničkih kretanja diskova i glave) performanse su vrlo loše u "slučajnom" pristupu podacima. Srednja vrijednost pronalaska podatka na disku (*average random seek*) je reda veličina 10 ms, dok je vrijeme neophodno za prelazak između dvije susjedne staze manje i obično iznosi ~ 0,5 ms. Srednje vrijeme čekanja da podatak rotiranjem bude pozicioniran ispod glave zavisi od brzine rotacije i kreće se između ~7 ms za 4200 rpm diskove i ~2 ms za 15000 rpm diskove.

Još jedan od problema vezanih za HDD je i fragmentacija koja je prikazana na slici 36. Fragmentacija nastaje tako što podaci koji su dio iste logičke sekvencijalne strukture (fajla) ne budu smješteni sekvencijalno na HDD već, usljed problematičnog rasporeda slobodnog prostora, fajlovi budu smješteni po dijelovima na različite fizičke lokacije, tako da je pri čitanju sekvence neophodno vršiti brojne pokrete glave. Na slici je ilustrovan primjer u kojem je, nakon brisanja Fajla 2, snimljen Fajl 4, ali kako nije bilo dovoljno slobodnog prostora u kontinuitetu, Fajl 4 je podijeljen u dva dijela i smješten na disk. Nakon toga, brišemo Fajl 3, a Fajl 4 nastavlja da raste (jer dopisujemo sadržaj u njega) te dio fajla dopijeva u prostor koji je ranije zauzimao Fajl 3. Sada pri čitanju Fajla 4 sa diska, iako sam fajl zauzima neprekinutu sekvencu blokova, dolazi do više bespotrebnih pokreta glave za čitanje: nakon čitanja prvog dijela, glava preskače treći i čita drugi dio, pa se vraća na treći dio, pa onda opet skače na neki četvrti i tako dalje. Kako je svaki od ovih "skokova" reda veličine 10 ms, očigledno je da je fajl koji je izdijeljen u stotinu dijelova nemoguće

pročitati za manje od 1 s, bez obzira na njegovu veličinu i tako posmatrano dobijamo vrlo lošu brzinu prenosa podataka od čak manje od 1 MB/s. Jedno od rješenja je defragmentacija što je proces ponovnog slaganja fajlova u kontinuitetu. Nažalost, i diskovi interno mogu remapirati pojedine oštećene blokove na rezervna mjesta i na taj način napraviti probleme i kod nefragmentisanog fajl sistema.



Slika 36: Primjer fragmentacije

I u slučajevima u kojima ne postoji fragmentacija, često je neophodno pristupati različitim dijelovima disk u sekvenci (npr. rad sa bazom podataka, višekorisnički pristup sistemu, kompajliranje velikog broja fajlova, itd) i u tom slučaju dolazimo do istih problema kao i kod fragmentisanih fajlova - umjesto da čitamo podatke sa diska, mi čekamo da se glava i ploče pozicioniraju na pravi način. Mjera za ovakav način rada je obično broj UI operacija koje možemo da izvršimo u sekundi i označava se sa IOPS (Input-output OPERations per Second) i za HDD uređaje iznosi između nekoliko desetaka i manji broj stotina operacija u sekundi (u zavisnosti od brzine rotacije, broja staza, kvaliteta uređaja, tipa opterećenja, itd).

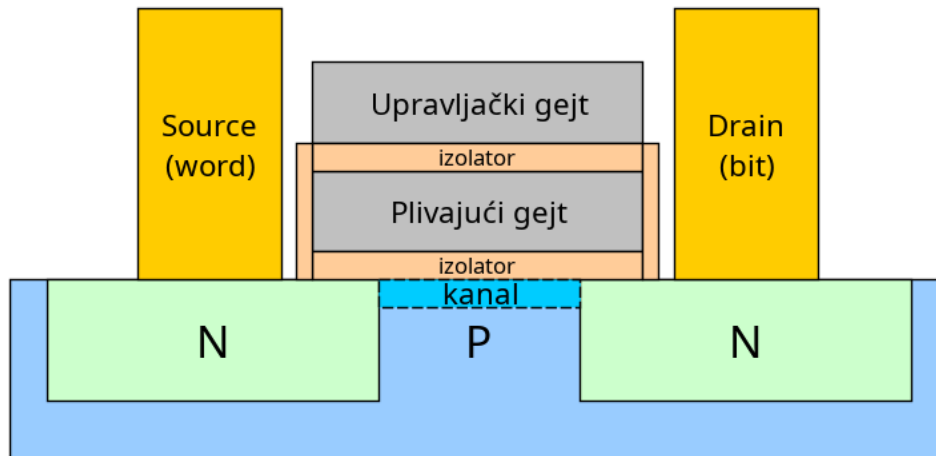
Da bi donekle ublažili ove negativne efekte, većina diskova sadrži integrisani keš i kontrolere koji mogu izvršiti zadatke mimo reda a sa ciljem da poboljšaju performanse (slično kao *out-of-order* izvršavanje instrukcija u procesoru). Sve to, naravno, dovodi do veće kompleksnosti i cijene sistema, ali alternative za ovakav tip uređaja teško da ima. Godinama je primjetan pad broja prodatih jedinica i novčanog udjela na tržištu, iako ukupan kapacitet prodatih jedinica polako raste.

Kada govorimo o trakama, važno je napomenuti da je ovdje pristup i ekonomska računica drugačija. Trake se praktično nikad ne koriste kao osnovno skladište za podatke i obično predstavljaju dio rješenja za bekap ili prenos podataka (fizičkim putem). Jedan uređaj za trake može koristiti proizvoljan broj traka i na taj način možemo ostvariti vrlo fleksibilan sistem sa stanovišta povećanja kapaciteta bez izmjene uređaja (što nije moguće kod HDD jer u tom slučaju možemo dodavati nove uređaje, što ne može ići u nedogled, ili moramo mijenjati uređaje onima većeg kapaciteta). Što se tiče cijene po kapacitetu, trake imaju najnižu cijenu, ali i ograničenu upotrebu i pristup je sekvencijalan. Kvaliteti trake su mogućnost višegodišnjeg čuvanja podataka (i preko 10 godina) i mogućnost fizičkog uklanjanja trake na drugu lokaciju ili čuvano mjesto što može biti izuzetno važno za arhivske upotrebe. HDD uređaji mogu zadržati podatke relativno dug period, ali zbog kompleksne mehanike često dolazi do otkaza uređaja koji nisu u upotrebi duži niz godina.

21. Solid State Drive - SSD

SSD uređaji za pohranu podataka se zasnivaju na upotrebi trajne memorije realizovane elektronskim kolima. Danas su gotovo svi SSD uređaji bazirani na upotrebi *flash* (fleš) tehnologije, iako postoje i drugi pristupi o čemu će više riječi biti u poglavlju 22. Vrijedi napomenuti, kao kuriozitet, da postoje i SSD uređaji koju se sastoje od DRAM modula i pomoćnog baterijskog napajanja (jer je i to jedan način da sačuvate podatke). Kada govorimo o konkretnim načinima realizacija SSD uređaja i načina upotrebe pojedinačnih ćelija, moguća su dva pristupa: NOR i NAND. Svaki od pristupa ima određene prednosti i mane, ali je činjenica da je NAND moguće napraviti jeftinije što znači da je gotovo sva memorija u SSD uređajima NAND tipa.

Osnovna ćelija je prikazana na slici 37 i možemo primijetiti da je vrlo slična klasičnom MOS FET tranzistoru, ali je gejt drugačije realizovan. Riječ je o "plivajućem" gejtu koji je sa svih strana električno izolovan (okružen izolatorom i nema direktne kontakte) tako da teoretski ne možemo da mu pristupimo. Praktično je to moguće jer možemo dovesti dovoljno intenzivno električno polje na upravljački gejt i na taj način injektovati "vruće" nosioce naelektrisanja u njega. Jednom kada plivajući gejt naelektrišemo, on teško gubi tako dobijeno naelektrisanje, a dok je naelektrisan može formirati kanal u poluprovodničkoj strukturi ispod gejta i na taj način čuvati podatak. Ovo je primjer kako možemo od katastrofalnog tranzistora (ili, još gore - "mrtvog" tranzistora jer ovakvu injektovanje naelektrisanja ubija klasične tranzistore) možemo da napravimo korisnu stvar. Inženjering se često svodi na pronalaženje najmanje lošeg rješenja ili sagledavanja stvari iz drugog ugla.

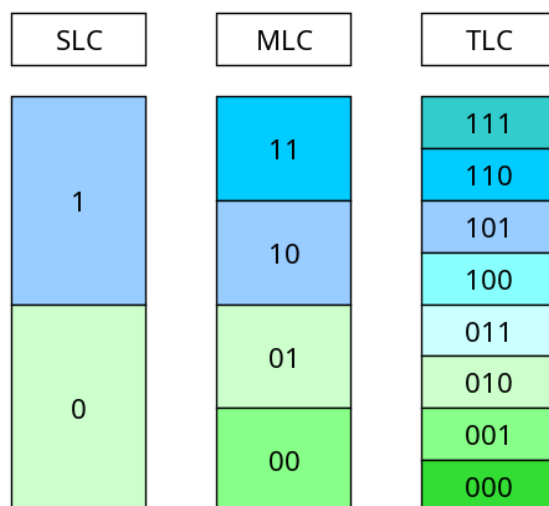


Slika 37: Osnovna ćelija

U okviru memorije se nalazi veliki broj pojedinačnih ćelija koje mogu biti organizovane i složene na različite načine, pri čemu se danas koristi "3D" ili "V-NAND" pakovanje pri čemu se pojedinačni slojevi vertikalno slažu, gotovo kao palačinke (vaferi i jesu okrugli i peku se tako da poređenje i nije mimo mjesta). Na taj način je, osim veće gustine pakovanja i manje cijene po kapacitetu, moguće ostvariti i vertikalno povezivanje kroz Si umjesto da se sve veze ostvaruju na rubovima (što je, opet, jako slično pristupu kod HBM koji je opisan na kraju poglavlja 18).

Uspostavljeni kanal nije "digitalne" prirode i nije ograničen na sve-ili-ništa provodnost nego je u zavisnosti od toga kolika je količina naelektrisanja prisutna u plivajućem gejtu moguće napraviti kanale koji će proizvoditi različite padove napona između sorsa i drejna tranzistora. Ovo

je dovelo do shvatanja da, ako je čuvati jedan bit u jednoj ćeliji dobro, još je bolje čuvati dva bita pri čemu sad pratimo četiri vrijednosti napona umjesto dvije. Tako su nastale MLC (*multi-level cell*) fleš ćelije. Gdje su dva bita, tu mogu i tri i dobili smo TLC (*three-level cell*) sa osam nivoa, a kasnije i QLC (*quad-level cell*) sa 16 nivoa napona i mogućnošću čuvanja četiri bita po ćeliji. Praktično prevedeno, na istu površinu je moguće pohraniti četiri puta više podataka ako možemo dovoljno precizno da upisujemo i čitamo. Ovaj princip je prikazan na slici 38.



Slika 38: SLC, MLC i TLC princip rada

Zašto ne i PLC, HLC? Možda u budućnosti, ali treba voditi računa i o tome da je razlika između SLC i MLC 2 puta veći kapacitet, MLC na TLC je donio još 50%, prelazak sa TLC na QLC još 33%. Vidljivo je da svaki naredni korak donosi sve manje dobiti, uz bitno komplikovanje drugih problema. Jedan od najvažnijih "drugih" problema je činjenica da plivajući gejt gubi naelektrisanje, polako, ali sigurno. Što su manje razlike između naponskih nivoa pojedinih vrijednosti, to je manja i količina naelektrisanja koju ćelija može izgubiti prije nego što postane neupotrebljiva. Navedeno SSD uređaje čini ne baš sjajnim izborom za dugotrajnu pohranu podataka (arhiviranje) jer će nakon nekoliko godina izgubiti naelektrisanje i podatke (zavisi i od tehnologije i od temperature, itd). Ćelije, takođe, imaju i ograničen broj upisa koji direktno zavisi od broja nivoa koje čuvamo u ćeliji pa je tako za SLC broj upisa do 100.000, za MLC 10.000, a za TLC par hiljada upisa. Iako ovo možda ne zvuči strašno, činjenica je da većina proizvođača za stvarne SSD uređaje daje garanciju vezanu za količinu upisanih podataka i da taj faktor nikad ne prelazi 1000 za TLC uređaje (npr. za Samsung 960 EVO 1 TB SSD iznosi 400 TBW - nakon upisa 400 TB, odnosno 400 prepisivanja, nema garancije da će uređaj raditi).

Ovo je očigledno veliki problem jer se ne upisuju svi blokovi ravnomjerno, neki sadrže kernel operativnog sistema ili arhivu starih podataka i mijenjaju se vrlo rijetko, dok su drugi dio baze podataka ili fajl baziranog keša i svakog dana će biti prepisani stotine ili hiljade puta. To znači da će u slučaju TLC uređaja biti ubijeni vrlo brzo. Da bi izbjegli ovu neslavnu sudbinu, SSD uređaji koriste kontrolere koji interno čuvaju mapu sa brojem upisa za svaki blok i kada detektuju da je neki blok blizu opasne zone, jednostavno ga remapiraju na drugi, trenutno slobodni, blok koji ima mali broj upisa. Kako je SSD memorija sa direktnim pristupom i ne pati od problema fragmentacije, nema ni gubitaka performansi pri ovom procesu (jer nam je svejedno koji blok poslije kojeg bloka čitamo, radi se o elektronskim "skokovima", ne mehaničkim). Ovaj pristup ravnomjernog raspoređivanja upisa (i potrošnje fleša) se naziva *wear-leveling*.

Poređenje SSD i HDD uređaja

Osnovne prednosti SSD uređaja u odnosu na HDD uređaje su:

- **Mnogo veće brzine čitanja i pisanja podataka** - i do nekoliko GB/s za potrošačke SSD modele u odnosu na par stotina MB/s za vrhunske serverske HDD uređaje.
- **Mnogo bolje performanse u "slučajnom" pristupu** - kod HDD govorimo o, u najboljem slučaju, par stotina IOPS, dok su potrošački SSD na stotinama hiljada IOPS, a namjenska rješenja omogućavaju i milione IOPS. Ovo je izuzetno važno za rad sa bazama podataka i višekorisnički/višeprocetni rad.
- **Nema efekata fragmentacije**, odnosno, ne interesuje nas fragmentacija jer ne plaćamo bitan danak nesekvencijalnom radu.

Osnovne mane SSD fleš bazirane tehnologije su:

- **Loše performanse.** Pojedinačna ćelija (odnosno blok ćelija) ima neimpresivne performanse (~25μs za čitanje 4KB, ~250μs za upis 4KB, ~2000μs za brisanje 256KB). Da bi došli do visokih performansi, neophodno je SSD uređaj napraviti od velikog broja čipova koji interno raspoređuju čitanja i pisanja (kao RAID-0, vidjeti poglavlje 34). To takođe znači da je često kapacitet vezan sa performansama po principu - što veći kapacitet, to bolje performanse (što i nije loše jer tradicionalno memorija radi na obrnutom principu). Podaci su često kompresovani prije upisa tako da SSD uređaji mogu imati i bolje performanse, ali to u velikoj mjeri zavisi od prirode podataka (tekst se vrlo dobro kompresuje, podaci iz baza podataka takođe, JPEG slike i h.264 video zapisi - ne baš, već su vrlo efikasno kompresovani).
- **Konačan broj upisa po bloku.** Svaki blok možemo upisati samo određeni broj puta nakon čega postaje neupotrebljiv. Naročito izraženo kod TLC i QLC tehnologija što zahtijeva primjenu *wear-leveling* pristupa.
- **Brisanje je sporije od upisa.** Da bi izmijenili jedan bajt, moramo pročitati cijeli blok, izbrisati cijeli blok i zatim ga upisati. Ovo je vrlo spor proces i SSD uređaji obično jednostavno upišu novi sadržaj u neki drugi blok i remapiraju ga da pokazuje na pravi. Stari blok biva izbrisan kada uređaj nije opterećen korisnim poslovima. Problem nastaje kad fajl sistem misli da je neki blok slobodan (jer je obrisani fajl koji ga je zauzimao) ali SSD ne zna za to i uporno vodi računa o podacima u njemu (što ruši performanse, povećava broj upisa, itd). Da bi ostvarili smislenu saradnju između fajl sistema i SSD uređaja koristimo TRIM naredbu kojom SSD uređaju javljamo o kojim blokovima više ne mora da vodi računa jer nas više ne interesuju.
- **Viša cijena za isti kapacitet** u odnosu na HDD. Iako cijene SSD uređaja konstantno padaju, padaju i cijene HDD uređaja i još uvijek je moguće za istu količinu novca kupiti znatno više kapaciteta upotrebom HDDova. Na primjer, trenutno je cijena po TB standardnog HDDa približno pet puta manja u odnosu na standardni SSD.

Praktično, od navedenih mana je jedino bitna cijena za potreban kapacitet, sve drugo su finese ili problemi koje je moguće efikasno riješiti, dok broj IOPS kod HDD nije moguće povećati ni blizu vrijednostima koje je lako moguće ostvariti upotrebom SSD uređaja.

22. Trajna radna memorija - Non-volatile RAM

Jedan od razloga za postojanje primarne i sekundarne memorije leži u činjenici da je gotovo sva primarna memorija privremenog karaktera, dok je praktično sva sekundarna memorija trajnog karaktera (ovo nije jedina razlika). Ako bi postojala memorija koja bi bila direktno adresibilna, trajna i efikasno ponovno upisiva, mogli bi koristiti istu memoriju za sve potrebe i riješiti se problema vezanih za memorijsku hijerarhiju (makar onih o kojima moramo voditi računa i na koje možemo uticati). Ovo nije nova ideja i kroz istoriju razvoja računara su postojale brojne tehnologije koje su u većoj ili manjoj mjeri ispunjavale zadate kriterijume, ali nijedna se nije zadržala ili preuzela ozbiljan dio tržišta od postojećih tehnologija. Veliki broj tehnologija je "u fazi razvoja" i u vječnom stanju "10 godina do komercijalnog proizvoda" (što znači da već dugi niz godina treba još 10 godina). Jedina trenutno komercijalno raspoloživa tehnologija je 3D XPoint.

3D XPoint tehnologiju su razvili Intel (koji je naziva Optane) i Micron (koji je naziva QuantX) i predstavlja vid trajne radne memorije, najvjerovatnije zasnovane na Phase-change principu rada, ali postoji više kontradiktornih izvora informacija, a Intel i Micron ne žele da definitivno odgovore na koji tačno način tehnologija funkcioniše. Ono što je poznato je da se svaki bit čuva kao promjena otpornosti osnovnog materijala (slično kao kanal u FET tranzistoru, ali trajno). Sama ćelija je komplikovanija od osnovne 1T1C ćelije DRAM memorije i 3D XPoint se nalazi i po performansama i po cijeni između DRAM i fleš baziranih memorija (za iste kapacitete).

3D XPoint je moguće koristiti na dva načina. Prvi način je upotreba kao klasičan SSD uređaj, samo ne zasnovan na fleš tehnologiji. Drugi način je interesantniji i omogućava upotrebu kao trajna radna memorija i isporučuje se u DIMM formatu (koji se naziva NVDIMM - *non-volatile DIMM*).

U slučaju upotrebe kao vida sekundarne memorije, u poređenju sa fleš baziranim SSD uređajima, 3D Xpoint uređaji imaju bolje performanse (jer nemaju probleme sa sporim inicijalnim upisom, kao ni probleme vezane za brisanje ili *wear-leveling*) što im omogućava da budu upotrijebljeni ili kao vrlo brzi SSD (po visokoj cijeni) ili kao dio keš sloja iza kojeg se nalazi sporija fleš bazirana memorija (i to QLC tipa da bi cijena bila niža). Iako brzina prenosa podataka možda i nije bitno drugačija u odnosu na fleš SSD uređaje (koji mogu ostvarivati izuzetne vrijednosti upotrebom paralelizacije), najveća prednost leži u činjenici da je latencija ispod 10 μ s što je vrlo važno za određene vrste opterećenja (u kojima fleš SSD nije dobro rješenje). U praktičnim uslovima su izmjerene gotovo dvostruko bolje performanse u odnosu na vodeće fleš bazirane serverske uređaje.

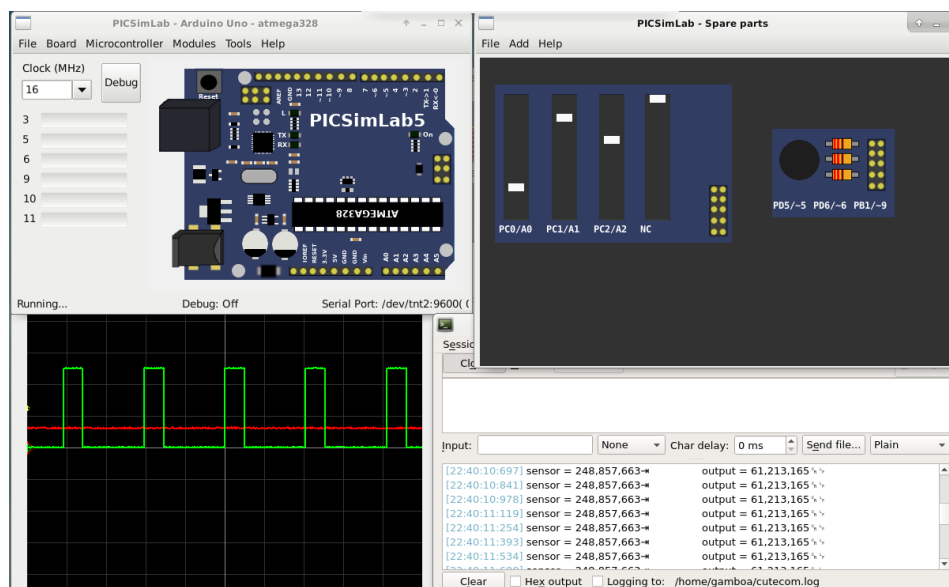
U slučaju upotrebe kao trajna RAM memorija, potrebno je obezbijediti podršku na svim nivoima (procesor, matična ploča, OS) jer čuvanje podataka u RAM-u pri restartovanju računara nije očekivani način rada. 3D XPoint NVDIMM su relativno velikih kapaciteta, ali su i sporiji u odnosu na klasične DRAM module tako da je i sa stanovišta aplikacija potrebno uzeti u obzir specifičnosti kako bi postigli željene rezultate.

Najveći problem ove tehnologije, kao i srodnih tehnologija, leži u činjenici da još nije jasno koje su osnovne praktične prednosti i situacije u kojima postoji očigledna prednost u odnosu na tradicionalne tehnologije. Neki od pristupa se zasnivaju na upotrebi za specijalizovane baze podataka, te kao pomoćna memorija na specijalizovanim procesorskim karticama. Najveći kupci su trenutno Amazon i Alibaba, ali nemamo pouzdane informacije o tome za šta trenutno koriste ovu tehnologiju.

23. Simulatori i emulatori

Iako i simulatori i emulatori imaju isti osnovni zadatak, reprodukovanje ponašanja jednog uređaja na drugom, razlike leže i u načinu rada i namjeni sistema. Iako je moguća simulacija i emulacija velikog broja različitih sistema, od fizičkih procesa do video igrica, mi ćemo se ograničiti na upotrebu u oblasti reprodukovanja ponašanja jednog računarskog sistema na drugom. S jedne strane su simulatori kojima je cilj da korisniku obezbijede što je više moguće informacija o stanju simuliranog uređaja, uključujući i, npr. stanja internih registara kojima u stvarnosti nije moguće direktno pristupiti, pa čak i talasne oblike na magistralama, kod emulacije je naglasak na vjernoj reprodukciji eksternog ponašanja, uključujući i aspekte vezane za performanse emuliranog sistema (koliko je moguće).

Simulatori nam omogućavaju da u bilo kom trenutku zaustavimo izvršavanje, da izvršavamo korak po korak, da se vratimo unazad kroz vrijeme, da direktno mijenjamo interna stanja sistema na načine koji nisu mogući u stvarnoj upotrebi. Tipično su kompletna okruženja koja nam omogućavaju da simuliramo i brojne druge hardverske komponente i ispitujemo njihovo ponašanje u okviru simuliranog sistema (npr. možemo da na mikrokontroler spojimo seriju sedmosegmentnih displeja ili matrični LCD i simuliramo "trčeca slova" koja razvijamo kao info panel za autobuse). Neki od predstavnika su TopView Simulator za 8031/8051 mikrokontrolere, te PICSimLab za PIC porodice mikrokontrolera. Sa slike 39 je očigledno da je cilj okruženja vezan dominantno za fizički svijet (prikaz talasnih oblika, štampane pločice, fizičkih komponenti, itd).



Slika 39: PICSimLab (<https://sourceforge.net/projects/picsim/>)

Emulatori nam, s druge strane, omogućavaju da testiramo funkcionisanje sa eksternog stanovišta i da emuliramo bilo koji procesor i njegovo funkcionisanje u približno realnim uslovima na našem x86 procesoru. To nam omogućava da testiramo kako će naša Android aplikacija kompajlirana za Android OS i ARM procesor raditi bez da moramo da posjedujemo uređaj sa odgovarajućim karakteristikama. I ovdje nam je na raspolaganju mogućnost povezivanja emuliranih uređaja (npr. GPS prijemnika, kamere, itd) sa ciljem detaljnijeg testiranja ponašanja emuliranog sistema. Tipični predstavnici su Bochs i QEMU (iako QEMU uključuje i podršku za VT kroz KVM).

24. Virtuelizacija

Kratka istorija

Virtuelizacija nije nov pojam. Prva demonstracija je izvršena na računaru IBM S/360-67 još davne 1967. godine upotrebom CP-40, a godinu kasnije i upotrebom tadašnjeg "open source" rješenja pod nazivom CP/CMS. Sistem je omogućavao *time-sharing* izvršavanje više programa na jednom računaru (i imao hardversku podršku za virtuelnu memoriju).

Popek i Goldberg su 1974. godine postavili teorijske zahtjeve za efikasnu virtuelizaciju, koji iako su zasnovani na pojednostavljenom modelu i danas imaju bitno mjesto u oblasti virtuelizacije. Prema njima **virtuelne mašine (VM)** su sposobne da virtuelizuju puni skup hardverskih resursa, uključujući procesor, memoriju (primarnu i sekundarnu) i periferije. Ovakva virtuelizacija, ostvarena od strane softvera pod nazivom **Virtual Machine Monitor (VMM)** mora da zadovolji sljedeće uslove:

- **Ekvivalentnost**
 - Program koji se izvršava u okviru VM treba da se ponaša na identičan način kao da se izvršava na stvarnoj mašini.
- **Kontrola resursa**
 - VMM mora u potpunosti kontrolisati sve virtuelizovane resurse.
- **Efikasnost**
 - Statistički značajna većina instrukcija mora biti izvršena bez uplitanja VMM.

Danas je u upotrebi veliki broj različitih definicija virtuelne mašine, od kojih su tri najkraće: izolacija aplikacije od hardvera; stvaranje okruženja za emulaciju operativnog sistema; efikasna, izolovana kopija realnog sistema.

Pri izvršavanju koda u VM možemo razlikovati dva osnovna tipa instrukcija: osjetljive (one koje mijenjaju stanje procesora na privilegovan način, npr. PUSHF i POPF) i neosjetljive (koje to ne rade, npr. JZ i NOP). VMM može da izvrši kod VM i da ostane u potpunoj kontroli resursa na dva osnovna načina. Prvi način podrazumijeva da postoji **hardverska podrška** u ISA koja omogućava da **kritične** instrukcije (osjetljive, a nepriviligovane) obradi procesor na efikasan način (ili da su sve osjetljive instrukcije privilegovane) i obično se naziva **hardverski potpomognuta virtuelizacija**. Drugi pristup se naziva **dinamičko rekompajliranje** i zasniva se na ideji da je ponašanje kritičnih instrukcija (ili većih dijelova koda) moguće reprodukovati zamjenom serijom nekritičnih instrukcija u toku izvršavanja. VMM praktično konstantno nadzire instrukcije koje treba da izvrši u budućnosti i, ako detektuje kritične instrukcije, mijenja ih sekvencom koja mu omogućava da zadrži punu kontrolu nad svim resursima na efikasan način. Ovo je jedini način virtuelizacije moguć na arhitekturama bez hardverske podrške za virtuelizaciju. Ovaj princip se ne koristi samo za virtuelizaciju, npr. JMV i Valgrind koriste ovaj pristup.

Važna osobina VMM je mogućnost privremenog zaustavljanja VM i pravljenja slike svih resursa (**snapshot**) što omogućava savršen bekap ili migraciju VM na drugi server (npr. privremeno zaustavimo VM, iskopiramo njeno stanje na drugi server i ponovo je pokrenemo na njemu).

25. x86 i virtuelizacija

U početku x86 arhitekture bijaše "Realni mod" i nikakva zaštita među procesima, nikakav multitasking, samo prost pristup memoriji i periferijama. To ne bi okuugodno i inženjeri stvoriše 80286 procesor koji imade (prilično prost) "Zaštićeni mod", a nedugo zatim i 80386 koji je pružao osnove podrške u hardveru za virtuelnu memoriju, straničenje i multitasking. Iako je 386 imao podršku za virtuelni "realni" mod, nije postojala podrška za virtuelni "zaštićeni" mod što je jedna od pretpostavki za hardverski omogućenu virtuelizaciju.

Nakon što je postalo očigledno da će 32-bitni sistemi biti nedovoljni, i Intel i AMD su počeli da razvijaju 64-bitne buduće procesore. Intel je napravio veliku grešku razvojem IA64 arhitekture koja je bila jedva unazad kompatibilna, što je dovodilo do vrlo loših performansi pri izvršavanju 32-bitnog koda, dok je AMD razvio AMD64 sa odličnim performansama i u 32 i 64 modu (drugim riječima, nakon kupovine "novog" servera kupci su imali bolje performanse bez obzira na opterećenje, dok kod Intela to nije bio slučaj). Kao najveći šamar u istoriji, Intel je bio primoran da koristi rješenje razvijeno od strane AMD-a za 64-bitne procesore.

Kako je virtuelizacija na 32-bitnim x86 procesorima bila problematična i nepotpuna (obavezna upotreba dinamičkog rekompajliranja), prelazak na novu 64-bitnu arhitekturu je bio pogodan trenutak da se riješi i taj problem i da procesori dobiju smislenu hardversku podršku za virtuelizaciju (koja je već postojala u drugim procesorima, ali ne i u x86 svijetu). Intelov pristup je imenovan Vanderpool i kasnije je dobio marketinški naziv VT-x, dok je AMD-ov pristup nazvan Pacifica, a kasnije preimenovan u AMD-V. Već ovdje je vidljiv uticaj ekonomskih sila na virtuelizaciju, a čak i danas, iako ne postoji stvarni razlog, postoje sistemi koji treba da imaju punu podršku za hardversku virtuelizaciju, ali zbog odluke proizvođača, ona nije prisutna.

Rastom popularnosti i razvojem upotrebe virtuelizacije u x86 svijetu došlo je i do rasta i razvoja podrške za hardverski podržanu virtuelizaciju koja je proširena sa samog procesora i na druge bitne komponente x86 zasnovanog računara. Posmatranjem ponašanja takvog računara možemo lako zaključiti da nisu sve komponente jednako bitne sa stanovišta performansi, kao i da je za virtuelizaciju potreban različit pristup u zavisnosti od namjene sistema. Navedeno se prije svega odnosi na periferije koje su jako često i razlog postojanja računarskih sistema (jer su nam potrebni za obradu ulaza i generisanje izlaza što je ostvareno upotrebom periferija). Emuliranje periferija je i komplikovano i sporo, naročito kada govorimo o brzim periferijama kao što su mrežne i grafičke kartice ili sistemi za pohranu podataka. Bilo kakav gubitak performansi će biti više nego vidljiv krajnjem korisniku, a bilo kakav sigurnosni problem ili nedostatak kontrole će ugroziti stabilnost kompletnog sistema (npr. ako dozvolimo VM da direktno i neograničeno upravlja grafičkom karticom gubimo mogućnost da mi upravljamo sistemom kroz GUI, ako dozvolimo da VM slobodno pristupa disku dolazimo u situaciju da VM može izmijeniti pohranjenu konfiguraciju VMM-a, itd).

Prekidi su izuzetno važni za funkcionisanje računara i prirodno je da su i AMD, i Intel (nažalost, na različite načine) omogućili hardverski podržanu virtuelizaciju prekida koja bitno pojednostavljuje zadatke VMM-a i omogućava bolje performanse. Iako se ova virtuelizacija odvija u samom procesoru, mi je trenutno posmatramo u kontekstu rada sa periferijama (na što nije ograničena).

Kako je moderni rad sa periferijama realizovan, ne samo pomoću prekida, nego i upotrebom DMA (*Direct Memory Access*), logičan naredni korak je i virtuelizacija tog aspekta koja je ostvarena upotrebom IOMMU (*Input Output Memory Management Unit*) jedinice koja omogućava virtuelnim mašinama da direktno komuniciraju sa periferijama (naročito važno kod brzih periferija poput mrežnih kartica, što je, opet, naročito važno jer su danas gotovo svi resursi umreženi). Na ovaj način su poboljšane performanse (jer nema potrebe za stalnim kopiranjem sadržaja po memoriji) ali i poboljšana sigurnost i stabilnost sistema čak i mimo virtuelizacije jer sada problematične periferije imaju manje mogućnosti da neometano pristupaju kompletnoj memoriji. Za upotrebu ove tehnologije je osim, naravno, međusobno nekompatibilnih rješenja AMD-a i Intela potrebno imati realizovanu podršku i na matičnoj ploči (i hardver i firmver) i na samoj periferiji (PCI Express ili, u ograničenom obliku, PCI uređaji). Ovaj pristup se naziva i *PCI-Passthrough* jer praktično predajemo PCI vezu prema periferiji na korišćenje VM-u. Mana IOMMU-a je što je pristup uređaju "sve ili ništa" i ako više virtuelnih mašina želi da koristi isti uređaj, neophodan je *function level reset* između predaje upotrebe. Jedan od načina prevazilaženja ovog problema je upotreba SR-IOV (*Single-root Input/Output Virtualization*) koja omogućava da se jedan fizički uređaj predstavlja kao više virtuelnih uređaja i omogućava VMM-u da svakoj VM kojoj je potreban pristup da "neograničen" pristup putem IOMMU-a po jednoj "virtuelnoj instanci" uređaja. Ukratko, da bi ostvarili dobre performanse, sve bitne komponente sistema (procesor, matična ploča i periferije) moraju podržavati virtuelizaciju upotrebom više tehnologija (AVIC/APICv za prekide, AMD-Vi/VT-d za IOMMU i SR-IOV). Na ovaj način je moguće ostvariti i preko 95% originalnih performansi nevirtuelizovanog sistema (NASA Flexible Cloud Fabric).

Virtuelizacija grafičkih kartica, zbog njihove specifične prirode, je vrlo problematična i nije ograničena na x86 arhitekturu, niti su problemi specifičnost neke određene arhitekture. Najjednostavnija situacija je u slučajevima kada želimo jednoj VM omogućiti nesmetan i neograničen pristup grafičkoj kartici (svodi se na IOMMU) što omogućava odlične performanse, ali i ograničava broj virtuelnih mašina po kartici na jedan. Napredna virtuelizacija koja omogućava da svaka virtuelna mašina ima pristup "svojoj" virtuelnoj instanci grafičke kartice zahtijeva da i same grafičke kartice imaju implementiranu hardversku podršku za virtuelizaciju, što je obično prisutno samo kod namjenskih modela za računarske centre (AMD MxGPU i Nvidia vGPU). Izuzetak je Intel koji omogućava ovakav vid virtuelizacije i na ugrađenim GPU-ovima upotrebom GVT-g tehnologije. Uvijek je moguća emulacija grafičkih kartica ili potpuno u softveru (za potrebe jednostavnog grafičkog izlaza) ili prosljeđivanjem zahtjeva grafičkoj kartici. U ovom slučaju performanse variraju između prihvatljivih (~80%) i loših (~10%), a postoje i brojna ograničenja (npr. podržan samo OpenGL ili Direct3D).

26. Tipovi VMM

U klasičnim pregledima oblasti virtuelizacije, uključujući i Popeka i Goldberga, često je navedena podjela na tipove hipervizora (VMM-a). U ovoj, prilično strogoj, podjeli, **hipervizori tipa 1** su oni koji se izvršavaju direktno na hardveru i služe samo da obezbijede virtuelnim mašinama okruženje za izvršavanje. U modernim okvirima u ovaj tip se obično ubrajaju VMWare ESXi, Microsoft Hyper-V i Oracle VM Server. S druge strane, **hipervizori tipa 2** su hipervizori koji se izvršavaju pod već postojećim operativnim sistemom, uporedo sa drugim aplikacijama i servisima. Moderni predstavnici su VirtualBox, VMWare Player, QEMU, itd.

Problem nastaje kod klasifikovanja hipervizora na Linux i FreeBSD platformama koji su realizovani kao kernel moduli koji praktično od kernela osnovnog operativnog sistema stvaraju hipervizor (znači tip 1) ali je taj osnovni operativni sistem i dalje potpuno funkcionalan i omogućava korisniku da pokreće i druge aplikacije uporedo sa virtuelnim mašinama (znači tip 2).

Tip 1 se još naziva i *Standalone VMM* jer je sam odgovoran za sve. Izvršava se "ispod" svih operativnih sistema (koji su "gosti" u virtuelnim mašinama) i, samim tim, direktno pristupa hardveru. To takođe znači da je za upotrebu bilo koje hardverske komponente potrebno imati i odgovarajuće drajvere za hipervizor. Kako je tržište ovakvih hipervizora mnogo manje od klasičnog tržišta koje pokriva, npr. Windows operativni sistem, samim tim je i proporcionalno manja podrška proizvođača komponenti u vidu fabričkih drajvera. Tim slijedom dolazimo do zaključka da je u slučaju upotrebe hipervizora tipa 1 neophodno pažljivo provjeriti da li su sve komponente podržane, tj. da li postoji odgovarajući drajver za željenu verziju hipervizora. Kako je ovo netrivialan problem, hipervizori tipa 1 su gotovo isključivo serverska rješenja i obično proizvođači nude gotove osnovne konfiguracije već prilagođene određenim konkretnim proizvodima (npr. HP i Dell vam pri specifikaciji servera nude opcije za VMWare, Hyper-V ili Xen).

Hipervizori tipa 2 se još nazivaju i *Hosted* jer se izvršavaju u okviru postojećeg operativnog sistema koji tada ima ulogu "domaćina", odnosno hosta. Kako sada hipervizor ne pristupa direktno hardveru nego koristi usluge osnovnog operativnog sistema, nema ni potrebe za postojanjem drajvera za pojedine komponente (jer je moguće koristiti ih kroz osnovni OS). S jedne strane je to dobra osobina jer gotovo sve što možemo da kupimo ima drajvere za popularne operativne sisteme, a s druge strane je loša jer nemamo nikakvu garanciju da su ti drajveri dovoljno kvalitetni i pouzdani za ozbiljnu virtuelizaciju. Imajući navedeno u vidu, nije iznenađenje što se ovaj vid hipervizora obično koristi kao *desktop* bazirano rješenje i nije pretjerano popularno u serverskoj virtuelizaciji. Treba voditi računa i o tome da je u ovom slučaju određeni dio resursa neminovno dodijeljen i osnovnom operativnom sistemu i te resurse ne možemo da koristimo za potrebe virtuelnih mašina tako da, ako je riječ o namjenskom virtuelizacijskom serveru, mi praktično kupujemo hardver koji neće biti iskorišten.

27. XEN, KVM i paravirtuelizacija

Xen i KVM (*Kernel-based Virtual Machine*) su najpopularnija open-source rješenje za virtuelizaciju. Xen se obično klasifikuje kao predstavnik hipervizora tipa 1, dok je KVM teško kategorisati jer je riječ o modulu za kernel koji od kernela pravi hipervizor koji tada jeste tipa 1 jer se izvršava direktno na hardveru ispod svih drugih operativnih sistema, ali je i tipa 2 jer je taj osnovni operativni sistem i dalje potpuno funkcionalan operativni sistem koji nije namijenjen samo za virtuelizaciju.

Xen ima specifičnu arhitekturu jer sam hipervizor upravlja osnovnim resursima (procesor i memorija), te virtuelnim mašinama (u ovom slučaju se nazivaju "domeni") od kojih jedna ("dom0" ili host domen) ima poseban status jer ima mogućnost direktnog pristupa hardveru, ali i kontrole hipervizora i pokretanja drugih domena. Za razliku od "dom0", ostali domeni ("domU") se ponašaju kao i imaju privilegije klasičnih virtuelnih mašina. U "dom0" se obično izvršava prilagođena verzija Linux operativnog sistema (ili neki BSD derivat), dok se u "domU" mogu izvršavati različiti operativni sistemi, od Linuxa i BSDa do Microsoft Windowsa. Samo izvršavanje virtuelnih mašina je moguće na dva načina: potpunom virtuelizacijom i paravirtuelizacijom.

Potpuna virtuelizacija se oslanja na usluge koje pruža hardverski potpomognuta virtuelizacija i u tom slučaju je moguće koristiti nemodifikovane operativne sisteme u virtuelnim mašinama uz određeni gubitak performansi (jer, npr. procesor mora da presretne sve kritične instrukcije i prepravi ih u prihvatljiv oblik).

Paravirtuelizacija se zasniva na ideji da je sam operativni sistem koji se izvršava u virtuelnoj mašini "svjestan" da se ne izvršava na hardveru i neće ni pokušati da izvrši problematične operacije, nego će "zamoliti" hipervizor da uradi posao za njega ("molbe" su praktično pozivi API-ja koje nudi hipervizor). Na ovaj način je posao hipervizora bitno pojednostavljen i moguće je ostvariti odlične performanse, ali po cijenu neophodnog postojanja prilagođene verzije operativnog sistema za izvršavanje u VM. Pri upotrebi paravirtuelizacije nije potrebna (ne koristi se) hardverska podrška za virtuelizaciju tako da je ovaj pristup moguć i na računarima koji je ne podržavaju (x86 32b i neki 64b sistemi). Xen je takođe poseban i po tome što se u ovom slučaju (x86-32 paravirtuelizacija) zaista koristi veći broj procesorskih prstenova (a ne samo podjela na privilegovan-neprivilegovan) jer se Xen izvršava u nultom prstenu, kernel VM u prvom, a aplikacije u trećem prstenu. Korak na pola puta je upotreba **nemodifikovanog jezgra OS** (mora postojati i hardverska podrška za virtuelizaciju) i **namjenski napravljenih drajvera za virtuelne (emulirane) periferije**. Ovo je često dobar kompromis jer virtuelizacija procesora i memorije nije (obično) problematična, ali periferije su druga priča. Vrijedi napomenuti da ovaj pristup nije vezan za Xen i predstavlja standardan način rada za gotovo sve hipervizore. Ako pokrenete bilo koji hipervizor i VM u njemu primijetićete da su periferije koje VM vidi ni nalik na one koje zaista imate u računaru. Xen trenutno podržava x86-32, x86-64 i ARM ISA.

KVM zahtijeva postojanje hardverski podržane virtuelizacije i trenutno podržava x86-64, S/390, PowerPC, IA64 i ARM ISA. Osim potpune virtuelizacije, za određene operativne sisteme postoji i podrška za paravirtuelizaciju u vidu drajvera za mrežnu i grafičku karticu, disk kontroler, ali i drajver koji omogućava VM da sama poveća ili smanji količinu RAM-a po potrebi (u okviru definisanih ograničenja) što se naziva "*memory ballooning*". Kako je riječ o dijelu zvaničnog Linux kernela, KVM je izuzetno popularan i prisutan u velikom broju besplatnih i komercijalnih rješenja.

28. Virtuelizacija na nivou operativnog sistema

Često nam nije potrebno da u virtuelnim mašinama pokrećemo operativne sisteme koji su drugačiji od osnovnog operativnog sistema (npr. Windows pod Linuxom ili obratno). Često ne želimo da virtuelizujemo kompletan hardver već samo da imamo odvojene operativne sisteme sa ciljem lakšeg upravljanja njima. Često nam je dovoljno da postoji određena logička izolacija procesa i aktivnosti koje se dešavaju u okviru neke "virtuelne mašine" i prihvatljivo nam je da je kernel praktično isti kao i kod osnovnog operativnog sistema. Kako kernel po čitav dan upravo i kontroliše i izoluje procese i aktivnosti (među korisnicima, itd) logično je pokušati iskoristiti isti taj mehanizam na malo napredniji način.

Ovaj pristup se naziva **virtuelizacija na nivou operativnog sistema** i zasniva se na ideji da kernel omogućava simultano postojanje više izolovanih instanci grupa procesa. One se nazivaju kontejneri (*containers*), zone (*zones*), virtuelni privatni serveri (*virtual private servers*), virtuelna okruženja (*virtual environments*), zatvori (*jails*), i tako dalje, u zavisnosti od toga koju konkretnu tehnologiju koristimo (LXC, Docker, Solaris, OpenVZ, BSD, itd).

Sve što je potrebno virtuelnoj mašini već postoji i u osnovnom operativnom sistemu: procesor (kernel ga inače kontroliše), memorija (takođe), fajl sistem (kernel i kontroliše pristup fajl sistemu), periferije (opet kernel kroz drajvere), itd. Ne samo da možemo da iskoristimo osnovni kernel za sve navedeno, nego i dobijamo apsolutno najbolje performanse (gubici ispod 2%) jer se ovaj pristup može posmatrati kao ekstremna paravirtuelizacija u kojoj je svaka funkcija kernela iz VM savršeno usklađena sa hipervizorom (jer je riječ o istom kernelu, ne dva ista kernela nego jedan kernel). Ovo, naravno, nameće očigledno ograničenje da je i u gostujućem OS neophodno koristiti isti kernel, ali zahvaljujući organizaciji Linuxa to ne znači da i distribucije Linuxa moraju biti iste. Tako je moguće pod Ubuntu Linuxom u kontejneru koristiti CentOS ili Arch Linux bez ikakvih problema.

Još jedna specifičnost ovog pristupa je vidljivost gosta iz hosta. Naime, iako je i kod drugih pristupa moguće vršiti nadzor funkcionisanja gosta iz hosta (što je neophodno za klasičnu virtuelizaciju), sada je to potpuno trivijalno. Host vidi sve procese svih kontejnera kao posebne grane (ili imenske prostore, kontrolne grupe, itd) dok je fajl sistemu svakog od kontejnera moguće pristupiti i direktno jer se nalazi negdje u hijerarhiji osnovnog fajl sistema. Kao što je moguće ograničiti korisnike u upotrebi hardverskih resursa, tako je moguće i vrlo precizno ograničiti kontejnere po pitanju utroška procesorskog vremena, memorijskog prostora, broja otvorenih fajlova, mrežnog protoka, itd.

Zahvaljujući svemu navedenom, moguće je na jednom serveru simultano pokrenuti zaista velik broj kontejnera (stotine, hiljade) jer svaki kontejner praktično zauzima onoliko resursa koliko bi zauzimale aplikacije koje se u njemu izvršavaju (uz minimalno dodatno zauzeće).

Danas su kontejneri postali toliko popularni da se o njima i ne razmišlja kao o vrsti virtuelizacije već predstavljaju, u velikom broju okruženja, podrazumijevani način izvršavanja, pa čak i isporuke, aplikacija ili aplikacionih komponenti (npr. jedna aplikacija će pokrenuti veći broj kontejnera, svaki sa specifičnom ulogom, a sve to bez mjerljivog gubitka performansi, samo uz nešto veće zauzeće skladišnog prostora).

29. Matična ploča

Matična ploča je osnovna elektronska štampana ploča u računarskom sistemu i ima ulogu ostvarivanja električnog (i fizičkog) kontakta između drugih komponenata sistema (procesor, memorija, kartice, itd). Osim obezbjeđivanja načina komunikacije između komponenti (što je moguće izvesti i upotrebom magistrale), matična ploča sadrži i aktivne komponente koje implementiraju značajnu funkcionalnost koja nije dio veze između drugih komponenti (BIOS, I/O kontroleri, ugrađene kartice, itd).

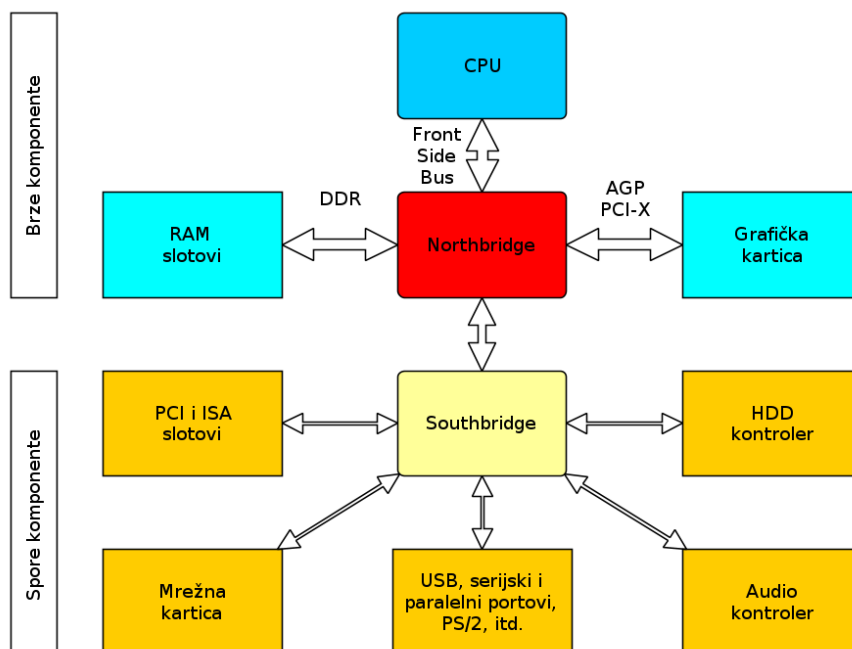
Važna osobina matičnih ploča (*motherboard*), a po kojoj se razlikuju od generičkih osnovnih štampanih ploča u računarskim sistemima (*mainboard*), je mogućnost proširenja funkcionalnosti sistema dodavanjem novih elemenata, realizovanih kao kartice (štampane ploče). U engleskom jeziku se obično koristi igra riječima na temu *motherboard* (za matičnu ploču) i *daughtercard* (za kartice za proširenje funkcionalnosti). Ove kartice mogu imati brojne funkcije, od grafičkih i zvučnih, preko mrežnih do akvizicionih kartica, a neke od njih će biti obrađene u narednim poglavljima.

Savremene matične ploče se tipično sastoje od sljedećih komponenata:

- Čipset - osnovni skup elektronskih kola koji obezbjeđuje povezivanje drugih komponenata, prije svega procesora i memorije.
- Podnožje za procesor - moguće je postojanje jednog ili više podnožja, tipično omogućava jednostavno uklanjanje i zamjenu procesora.
- Memorijski slotovi - namjenski slotovi za proširenje sistema radnom memorijom, danas gotovo isključivo DIMM modulima DDR tipa (konkretna verzija zavisi od procesora i čipseta).
- Osnovni blok ROM-a neophodan za inicijalizaciju sistema (BIOS ili UEFI), može biti integrisan sa čipsetom.
- Slotove za kartice kojima proširujemo funkcionalnost sistema.
- Konektore za povezivanje komponenata, tipično kroz funkcionalnost ugrađenih kartica (npr. USB, SATA, M.2, itd).
- Generator osnovnog takta za rad sistema, te sistem za napajanje koji omogućava prenos električne energije od napojne jedinice do aktivnih komponenata u sistemu.

Klasična logička organizacija matične ploče je data na slici 29. Osnovne komponente su podijeljene u dvije klase: brze i spore. Brze komponente obuhvataju CPU koji je povezan putem FSB-a (*Front Side Bus*) na Northbridge na koji su povezani i memorijski moduli (DDR) te, opciono, grafička kartica upotrebom namjenskog interfejsa (nekada VLB, pa AGP pa PCI-X pa PCIe - svaki je obezbjeđivao bolje performanse). Na Northbridge je povezan i Southbridge koji služi za povezivanje "sporih" komponenti i zadužen je za komunikaciju sa sporijim karticama (audio, HDD, mrežnim, USB, itd). Tipično se koriste sporije magistrale (PCI, prije toga ISA i slične). Kako je procesor "srce" sistema, logično je da je FSB najbrža magistrala jer je sve podatke potrebno prenijeti preko nje, bez obzira na druge magistrale. Ovo je naročito izraženo u slučajevima kada se ne koristi DMA pristup i kada procesor mora da obavlja sav posao oko kopiranja podataka. Ovo je vrlo neefikasan pristup, naročito u komunikaciji između brzih komponenti, tako da je jedan od

osnovnih zadataka Northbridgea bio da upravlja memorijom na efikasan način (MMU - Memory Management Unit je bio dio Northbridgea). Kod višejezgrenih procesora je dolazilo do ozbiljnih padova performansi jer sada veći broj jezgara mora da dijeli performanse jednog FSB-a koji je jedva dovoljan i za jedno jezgro. Kako su rasle performanse komponenti i kako su pojedine "spore" komponente postajale vrlo brze (npr. SSD, mrežne kartice, eksterni diskovi), postajalo je očigledno da ovakav pristup više nije dobar.



Slika 40: Klasična logička organizacija matične ploče (northbridge i southbridge)

Prva bolna tačka, koja je kočila razvoj procesora sa više jezgara, je problem lokacije MMUa van procesora i daleko od jezgara i keša. Rješenje je migracija memorijskog kontrolera u sam procesor. Ova migracija je omogućila i bolju kontrolu pristupa memoriji (koja prethodno nije bila isplativa sa stanovišta performansi) što je dovelo i do bolje podrške za virtualizaciju.

Kako su grafičke kartice postajale sve naprednije, postalo je očigledno da se performanse gube na komunikaciji sa procesorom i memorijom kroz posrednike, tako da su i funkcionalnosti povezivanja grafičkih kartica migrirale u sam procesor. Kod određenih klasa procesora, proces migracije nije stao na tome nego su i kompletne grafičke kartice migrirale u procesor, uz sve prednosti i mane koje taj pristup ima.

Kako su i druge magistrale postajale sve brže, polako je dolazilo i do migriranja kompletne funkcionalnosti PCI Express (vidi poglavlje 31) kontrolera u procesor, a kasnije i drugih, tradicionalno "sporih" kontrolera. Moderne sekundarne memorije, mrežne kartice i USB periferije su u stanju da ostvare prenos podataka reda veličine gigabajta u sekundi i ne mogu se više posmatrati kao "spore", koliko god se tamo svrstavale tradicionalno.

Danas smo u situaciji da je praktično kompletna funkcionalnost koja je tradicionalno bila zadatak Northbridge komponente implementirana direktno u CPU-u. Ekstremni oblik konvergencije funkcionalnosti predstavlja SoC (*System-on-a-Chip*) pristup popularan (ne samo) u ugrađenim i mobilnim uređajima koji objedinjuje svu funkcionalnost (procesor, primarna i sekundarna memorija, mreža, grafika, kontroler za periferije, itd.) u jednoj komponenti.

30. Magistrale

U oblasti računarstva, **magistrala** (*bus*) predstavlja komunikacioni sistem koji obezbeđuje vezu i prenos podataka između komponenti računarskog sistema ili računarskih sistema, te uključuje i hardver i softver i komunikacione protokole potrebne za funkcionisanje.

Brze promjene u oblasti računarstva predstavljaju ozbiljan problem za formiranje smislenih i dugoživućih podjela magistrala. Iako je tradicionalno *magistrala* imala značenje koje sa sobom nosi i kvalifikaciju topologije veze, danas se isti termin koristi i za topologije koje koriste paralelne ili ulančane veze, pa čak i za topologiju zvijezde (koja predstavlja suprotnost originalnom značenju).

Slična situacija je i sa podjelom na interne (lokalne) i eksterne magistrale. Ono što je nekad bilo posmatrano kao interna magistrala koja povezuje interne komponente velikom brzinom, danas se koristi i za povezivanje eksternih komponenti (npr. PCI Express i Thunderbolt, a InfiniBand i HyperTransport su od početka fleksibilni po pitanju upotrebe).

Kako su se računarski sistemi razvijali, tako su rasle performanse i mogućnosti komponentata koje su ih činili, a, samim tim, i zahtjevi koje su morale ispuniti magistrale koje ih povezuju. Prva, nama interesantna, magistrala je ISA (*Industry Standard Architecture*), razvijena od strane IBM-a za potrebe prvih PC računara, tada široko prihvaćena od strane drugih proizvođača. Zbog ograničenih performansi (16-bitna magistrala, unazad kompatibilna sve do 8088 procesora), proizvođači su pokušavali da problem riješe uvođenjem modifikovanih verzija (EISA) ili upotrebom više magistrala na istoj matičnoj ploči kombinujući VLB (VESA Local Bus) za grafičku karticu i ISA za druge periferije. Iako danas predstavlja istoriju, ISA je ostala u internoj upotrebi u brojnim drugim standardima (PCMCIA, ATA/IDE i SuperIO koji se koriste i u modernim računarima za rad sa zaista sporim komponentama). Od navedenih je, nama, naročito interesantan podatak da je ISA poslužila kao osnova za ATA, što je kasnije pretvoreno u Serial ATA, odnosno SATA standard koji i danas koristimo za povezivanje sekundarne memorije.

Pojavom PCI standarda se činilo kao da je problem konačno riješen jer su PCI kartice sada u potpunosti zamijenile prethodne ISA i VLB kartice. Problem sa kapacitetom je taj što čim slobodan kapacitet postane primijećen, desi se neka promjena koja omogućava upotrebu i tog kapaciteta. Tako su se pojavile kartice (najčešće grafičke) koje su obećavale još bolje performanse, ali po cijenu upotrebe još bržeg vida komunikacije. Tako je došlo do pojave AGP (*Accellerated Graphics Port*) standarda koji je predstavljao proširenje PCIa koje je postalo popularno u kućnoj upotrebi (desktop računari) dok su serveri počeli da koriste (naravno nekompatibilan) pristup pod imenom PCI-X (PCI eXtended) koji možemo posmatrati kao PCI ali na (mnogo) većem taktu.

Vremenom je došlo do standardizacije na pristup zasnovan na upotrebi PCI i PCI Express (vidi poglavlje 31) što je umnogome pojednostavilo situaciju jer osim vrlo sličnog imena koje često zbunjuje ljude (PCI Express i PCI-X) više nema dileme oko toga koja magistrala se nalazi u računaru.

31. PCI i PCI Express

Priča o PCI (*Peripheral Component Interconnect*) je interesantna jer ponavlja prethodno naučene lekcije iz oblasti poslovanja i razvoja računarstva. Kako je postajalo izvjesno da je neophodna zamjena za starije interfejsa (ISA, VLB, itd) Intel se posvetio razvoju novog standarda. Netipično za industriju, a vrlo slično potezu IBM-a pri kreiranju prvog PC računara, Intel je objavio specifikaciju i dozvolio neograničenu i besplatnu upotrebu novorazvijenog standarda od strane svih zainteresovanih proizvođača. Kako je PCI bio veliki iskorak u odnosu na tadašnja rješenja, a i nije bio opterećen problematičnim licencama i patentima, trenutno je postao izuzetno popularan na serverskom tržištu (jer je tu postojala potreba za mnogo boljim performansama raznorodnih kartica). Popularnost u potrošačkom segmentu (desktopi) je malo kasnila jer je dio funkcionalnosti bio pokriven još neko vrijeme postojećim rješenjima na prihvatljiv način.

PCI može raditi kao 32-bitni sa taktom od 33 MHz, ali i kao 64-bitni ili na 66 MHz taktu. Iako se prelaskom na 64 bita i 66 MHz ostvaruju četiri puta bolje performanse, u praksi su takva rješenja primijenjena gotovo isključivo na serverskoj strani, dok su desktop računari gotovo isključivo podržavali 32b i 33 MHz varijantu ograničenu na 133 MB/s.

Važna osobina PCI je da je riječ o **paralelnom** načinu komunikacije koji funkcioniše na takav način da se sa stanovišta uređaja priključenog na PCI slot ostvaruje direktna veza u kojoj uređaj upravlja magistralom. Na taj način se ostvaruje topologija zvijezde i nema kolizije između više uređaja koji istovremeno rade u jednom računarskom sistemu. PCI definiše i robustan sistem automatske konfiguracije koja olakšava rad sa povezanim komponentama koje su dostupne i kao memorijski mapirani IO uređaji (MMIO) što olakšava i rad sa softverske strane (PCI podržava i klasičan preko I/O adresa ali je taj način bitno sporiji). Da bi procesor mogao da komunicira sa uređajima (makar na elementarnom nivou) i prije učitavanja drajvera, PCI uređaji mogu posjedovati i ROM sa kodom kojim se ostvaruje ta funkcionalnost.

Nažalost, PCI standard se pokazao kao nedovoljan za, u tom trenutku, najbrže grafičke kartice što je dovelo do razvoja i kratkotrajne popularnosti AGP (*Accelerated Graphics Port*). AGP, koji je u realnosti nadskup PCI standarda, ne koristi PCI magistralu već posebnu vezu prema procesoru koja je jednostavnija i optimizovana za grafičke kartice što obezbjeđuje bitno bolje performanse (jer je namjenski razvijen za grafičke kartice i njihove specifičnosti).

Kao i u mnogim drugim slučajevima (vidi poglavlje 33) nakon paralelnog rješenja, na red je došlo **serijska** veza u vidu PCIe (PCI Express) standarda. Zahvaljujući jednostavnoj i dobro osmišljenoj arhitekturi koju je bilo lako primijeniti za različite potrebe, PCIe je postao dominantna tehnologija za proširenje funkcionalnosti računara. Danas se koristi za povezivanje grafičkih kartica, HDD kontrolera, SSD uređaja, mrežnih kartica, itd. Naravno, kako nije realno da jedna serijska veza ostvari radikalno bolje performanse od paralelne veze iste generacije, PCIe radi sa konceptom linija. Svaka linija je serijske prirode, a komponente kojima jedna linija nije dovoljna mogu ostvariti progresivno bolje performanse povećanjem broja linija tako da postoje x1, x2, x4, x8 i x16 varijante PCIe. PCIe kartice podržavaju izmjenu u radu (*hot-swap*) i omogućavaju podršku za hardversku virtuelizaciju (vidi poglavlje 25). PCIe je poslužio i kao osnova za veliki broj drugih interfejsa kao što su ExpressCard, SATA Express, M.2 i U.2.

Razlike između PCI i PCIe

	PCI	PCI Express
Vrsta linka	0	0
Topologija	Paralelna zvijezda do procesora	Serijska zvijezda do kontrolera (sviča)
Propusni opseg	133 MB/s (32b @ 33 MHz) 533 MB/s (64b @ 66 MHz)	V1.x x1 250 MB/s V1.x x16 4,00 GB/s V3.1 x1 985 MB/s V4.0 x16 31,5 GB/s V6.0 x16 126 GB/s
Izmjena u radu (<i>hot-swap</i>)	Ne	Da
Broj pinova	124 (32b) 188 (64b)	36 (x1) 164 (x16)
Napajanje	+3.3V/7.6A , +5V/5A , +12V/500mA	+3.3V/3A +12V/500mA (x1) +12V/5,5A (x16) 300W (dodatni konektori)

32. USB

USB (*Universal Serial Bus*) je široko prihvaćen industrijski standard koji obuhvata specifikacije konektora, kablova, napajanja i protokola za povezivanje računara i periferija ili drugih računara.

USB je nastao u vrijeme u kojem su za povezivanje različitih perifernih uređaja korišćeni bitno različiti standardi (serijski port za miša, PS/2 za miša i tastaturu, paralelni port za štampač, itd). Osnovna ideja je da se svi standardi zamijene jednim standardnom koji bi mogao biti univerzalno primjenjiv (*Universal*), a koji bi bio zasnovan na serijskom načinu komunikacije sa ciljem pojednostavljivanja sistema (*Serial*). Najveći uspjeh USB standarda je to što je zaista u stvarnom svijetu ostvario postavljene ciljeve. Danas je izuzetno teško naći uređaj koji koristi žičanu vezu a da ona nije USB. Topologija mreže je u vidu zvijezde u više nivoa, pri čemu USB habovi (*hub*) mogu proširiti topologiju do pet nivoa sa ukupno do 127 uređaja po kontroleru (jedan uređaj može imati više kontrolera). Osnovni hab se naziva *root hub*. Svaki od povezanih uređaja može obavljati više funkcija (npr. veb kamera može biti i video i audio uređaj istovremeno, tastatura sa ugrađenim *touchpadom*, itd).

Sami USB uređaji su podijeljeni u 21 vrstu uređaja od čega su, nama, najzanimljiviji:

- USB Mass Storage - uređaj koji omogućava pohranu podataka, zahtijeva ekskluzivni pristup uređaju, danas izuzetno popularni u kombinaciji sa fleš memorijom u vidu USB stikova i eksternih drajvova.
- Media Transfer Protocol - uređaj koji skladišti multimedijalne sadržaje, ne zahtijeva ekskluzivni pristup uređaju (može se koristiti sa uređajima koju istovremeno koriste iste fajlove) i predstavlja najčešći način prenosa fajlova žičanim putem kod "pametnih" uređaja.
- Human Interface Devices - uređaji namijenjeni za interakciju sa ljudima, ulazni uređaji poput tastature, miševi, tableti, džojstici, itd.
- Audio i Video - uređaji koji predstavljaju izvor audio i video signala, često objedinjeni u jedan multimedijalni uređaj.
- Printer - posebna klasa uređaja namijenjena za rad sa štampačima i srodnim uređajima.
- USB Hub - USB hab.

Pored navedenih, postoje i druge vrste uređaja, od čitača otisaka prsta preko bilborda do Application-specific i Vendor-specific koji mogu biti bilo šta za što nam proizvođač obezbijedi potrebne drajvere.

Od 1994. godine do danas, USB standard je prošao kroz veći broj revizija od kojih su neke najznačajnije karakteristike navedene u sljedećoj listi:

- USB 1.0 i 1.1 su definisali osnovne postavke, dva tipa konektora u dvije varijante veličine (A i B), kao i dvije osnovne brzine rada (*low speed* 1,5 Mbps i *full speed* 12 Mbps).
- USB 2.0 je omogućio bolje performanse (*high speed* 480 Mbps), manje konektore (mini A i mini B, micro B i micro AB), USB On-The-Go (OTG) koji omogućava direktnu vezu dva uređaja bez posrednog hosta, veću dozvoljenu struju napajanja. Komunikacije je i dalje *half-duplex*.

- USB 3.x definiše još brži prenos podataka (*super speed*) brzinama od 5 (USB 3.2 Gen 1, ranije 3.0), 10 (USB 3.2 Gen 2, ranije 3.1) i 20 Gbps (USB 3.2 Gen 2x2, ranije 3.2) i podrškom za *full-duplex* rad, te uvodi u igru nove pinove u konektorima i linije u kablovima. Definisan je i novi (unazad kompatibilan i sa verzijom 2.0) konektor USB-C koji mijenja i A i B strane prethodnih verzija olakšavajući povezivanje, ali i veće dozvoljene struje i napone za napajanje uređaja (USB PD do 100W). USB 3 je čuven i po velikoj zbrci i mijenjanju imena što je dovelo do velike konfuzije na tržištu. USB 3 portovi i konektori su prepoznatljivi po plavoj boji i stilizovanoj oznaci za *super speed*.
- USB4 (bez razmaka) je nova specifikacija standarda i zasnovana je na Thunderbolt 3 specifikaciji (koja se bitno oslanja na PCIe) sa kojom je kompatibilna, kao i sa prethodnim verzijama USB standarda. USB4 omogućuje punu kompatibilnost i sa HDMI, MHL, DisplayPort i VirtualLink standardima (koji su mogli biti podržani i u verzijama 3.x ali su sada dio formalnog standarda). Riječ je trenutno vrlo novoj verziji standarda koja je podržana od strane Linux kernela 5.6, MacOS 11, te Intel procesora 11. generacije i uređaja koji koriste Apple M1 procesor.

Ono što možemo primijetiti iz nastanka i evolucije standarda je poznata priča: paralelni interfejs je nezgodan, kreiramo brzi serijski interfejs sa ciljem smanjivanja broja pinova, ne možemo beskrajno podizati takt, dodajemo nove pinove i linije, GOTO 1. Malo dubljim istraživanjem dolazimo do zaključka da je USB 3.x u mnogim aspektima sličniji PCIe i SATA standardima nego USB 2.0 (a SATA je naslijedio ATA koji je potekao od ISA i time zatvaramo krug). TLDR - dobra rješenja su univerzalna, a dobra znanja možemo primijeniti više puta u različitim oblastima - to je inženjering.

33. Načini povezivanja sekundarne memorije

Za potrebe ovog poglavlja ćemo ignorisati trajnu primarnu memoriju (vidi poglavlje 22) i skoncentrisaćemo se na tradicionalnu trajnu sekundarnu memoriju u vidu HDD i SSD uređaja.

Sekundarna memorija može biti povezana na dva osnovna načina: indirektno (u slučaju da su uređaji povezani preko kontrolera) i direktno (kao uređaj priključen na PCIe). Pri direktnom povezivanju uređaji su klasični PCIe uređaji i sami moraju da implementiraju sve neophodne više nivoe komunikacije (kao i da obezbijede potrebne drajvere za računarske sisteme). Kako PCIe uređaji tipično imaju bolje maksimalne teoretske performanse, ali ne želimo da svaki disk ima svoj način rada, često se koristi pristup zasnovan na upotrebi nekog standarda u kombinaciji sa PCIe standardom.

Indirektna veza podrazumijeva postojanje kontrolera koji je (u modernim računarima) priključen na PCIe vezu ili ugrađen u čipset matične ploče (što je interno realizovano PCIe vezom tako da su oba slučaja ekvivalentna). Kontroler je u potpunosti zadužen za rad sa individualnim uređajima i sistemu ih može predstaviti kao skup individualnih uređaja ili kao jedan uređaj nastao kombinovanjem više individualnih (vidi poglavlje 34). Veza između kontrolera i individualnih uređaja može biti ostvarena na više načina:

- **PATA** (Parallel ATA) - takođe i pod nazivima ATA i IDE, stari standard koji koristi **paralelne veze** za povezivanje (kablovi sa 40 ili 80 žica), danas ima samo istorijski značaj. Moguće je priključiti dva uređaja po kablovima pri čemu jedan ima *master* ulogu, a oba uređaja dijele magistralu. Evolucijom standarda je nastao SATA standard (kao ATA/ATAPI-7). Maksimalna realna brzina je 133 MB/s.
- **SCSI** (*Small Computer System Interface*) - skup standarda koji pokrivaju sve aspekte od fizičkog preko električnog do sloja protokola i namijenjen je za povezivanje sekundarne memorije, ali i drugih uređaja poput skenera i multifunkcionalnih uređaja. Riječ je o **paralelnom** interfejsu koji može da radi u sinhronom i asinhronom režimu, te podržava povezivanje do 16 uređaja. Danas je većinom od istorijske važnosti jer je zamijenjen SAS standardom. Maksimalna realna brzina je 320 MB/s.
- **SAS** (*Serial Attached SCSI*) - moderna verzija SCSI protokola, koristi **serijski način rada i topologiju zvijezde**, a omogućava upotrebu električnih i optičkih veza. Predstavlja tipično **serversko rješenje**. Konektori su fizički i električno kompatibilni sa SATA konektorima. Maksimalna realna brzina je 1200 MB/s za električnu vezu.
- **SATA** (*Serial ATA*) - moderna verzija ATA protokola, koristi **serijski način rada**. Predstavlja dominantan način povezivanja HDD uređaja van serverskog tržišta (iako se koristi i na serverima). **Topologija je zvijezda**. Omogućava izmjenu uređaja u radu (*hot-plug*). Maksimalna realna brzina je 600 MB/s (SATA-3 standard).

SATA standard je poslužio kao osnova za veći broj drugih standarda i pristupa kao što su: eSATA za povezivanje eksternih uređaja, mSATA za povezivanje uređaja malih dimenzija, SATA Express koji omogućava povezivanje i SATA i PCIe uređaja (interno koristi PCIe 3.0 x2 link, propao, čitaj dalje), M.2 (nasljednik mSATA i ubica SATA Expressa) koji je fizički smanjio, a logički proširio SATA Express dodavanjem USB 3.0 porta, te U.2 koji omogućava još brži rad jer koristi PCIe 3.0 x4 link.

Ono što možemo primijetiti je da postoji konstantan proces "ples" različitih tehnologija i prihvatanja rješenja iz drugih oblasti u trenutku kada se to pokaže kao korisno. Na primjer, evolucija i desktop i serverskih rješenja je išla sličnom putanjom - od paralelnog i dijeljenog kabla do serijske veze i point-to-point linkova prema uređajima. Takođe, serverska rješenja tradicionalno imaju finansijski prostor za naprednije tehnologije (npr. SCSI nije samo za HDD, ATA/IDE jeste) ali se vremenom napredne mogućnosti prenose i u jeftinije proizvode (ATAPI je praktično mogućnost "govorenja" SCSI komandi kroz ATA interfejs za ne-HDD uređaje). Tako je *hot-swap* došao u SATA standard, kao i podrška za izmjenu redoslijeda izvršavanja komandi disku (NCQ), ali bez naprednijih opcija dostupnih u SAS standardu (TCQ, npr. zabrana izmjene ili slanje komande sa višim prioritetom, veći red komandi). Pitanje kompatibilnosti je takođe interesantno. Kako ima finansijskog smisla da koristimo jeftinije SATA diskove u serverima (kada nam vrhunske performanse nisu kritične) svi moderni SATA uređaji su upotrebljivi u serverima koji imaju SAS kontroler. Kako nije realno da ćemo koristiti skupi *enterprise* disk u jeftinom računaru, obrnuto ne vrijedi (SAS disk neće raditi priključen na obični SATA kontroler).

Vidljivo je i da jednom definisan kvalitetan standard možemo koristiti i za različite tehnologije povezivanja. Primjeri su FC (*Fibre Channel*) koji omogućava upotrebu SCSI za komunikaciju preko optičkih veza (do 16Gbps i mnogo veće udaljenosti), iSCSI (*Internet SCSI*) koji koristi IP ispod SCSIja (praktično neograničen domet) ili UAS (*USB Attached SCSI*) koji omogućava upotrebu USB 3.0 veze za povezivanje diskova. Takođe, kako je SATA standard relativno siromašan i prvenstveno namijenjen za upotrebu u jeftinijim računarima, većina pravih, hardverskih RAID kontrolera (vidi poglavlje 34) je u stvari SAS kontroler koji, kao što je ranije rečeno, podržava rad sa SATA diskovima.

Sve dok postoji smislen razlog za recikliranje standarda, nema potrebe za izmišljanjem novih. Tako je primjetna evolucija i konstantno prilagođavanje popularnih i pristupačnih standarda za sve brže uređaje koji često nemaju gotovo ništa zajedničko sa izvornim uređajima po pitanju performansi (glave i cilindri ne postoje u SSD uređajima, PCIe i USB 3.0 link na istom portu za povezivanje interne sekundarne memorije, ali i iSCSI za povezivanje 42 km udaljenog servera).

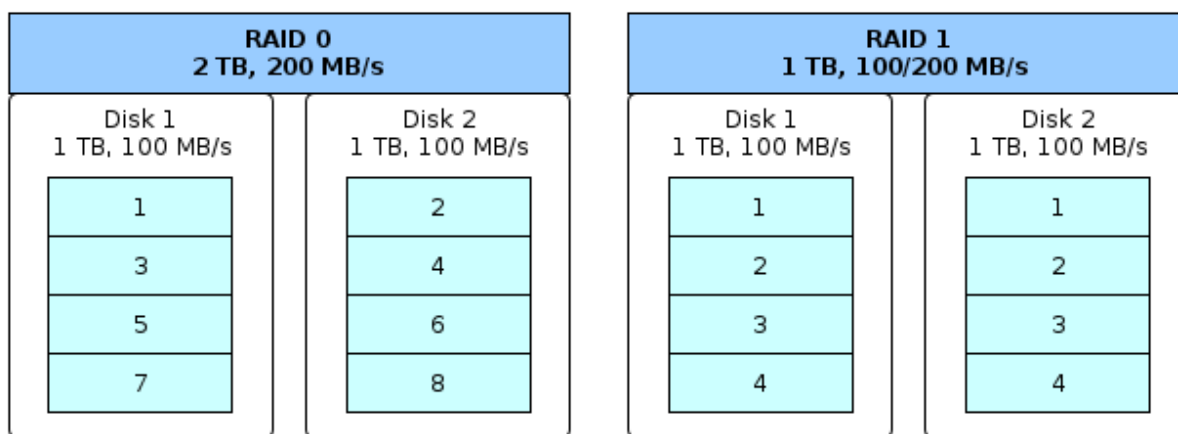
34. RAID

Performanse jednog uređaja za pohranu podataka često ne ispunjavaju zahtjeve koji su nama, kao projektantima rješenja, kritični. U nekim situacijama je moguće pronaći odgovarajući uređaj (po znatno višoj cijeni), ali možemo doći i u situaciju u kojoj jednostavno ne postoji odgovarajući uređaj (npr. trenutno ne postoji HDD kapaciteta 200 TB).

Jedno od rješenja u tim slučajevima se zasniva na ideji da možemo smislenim povezivanjem većeg broja individualnih uređaja napraviti (emulirati postojanje) uređaja koji zadovoljava specifikacije. Taj pristup se naziva RAID (*Redundant Array of Inexpensive Disks* ili *Redundant Array of Independent Disks*) i predstavlja vid virtualizacije sekundarne memorije sa ciljem povećanja performansi, pouzdanosti, kapaciteta ili kombinacije navedenih osobina. Predstavlja antitezu SLED (*Single Large Expensive Disk*) pristupu opisanom ranije.

Osnovna ideja RAID pristupa je da se podaci rasporede na više od jednog uređaja, a sam način zavisi od vrste RAIDa i naših potreba. Vrste RAIDa se obično nazivaju **RAID nivoi** i mogu biti i hibridni (ugnježdeni, višenivovski):

- **RAID 0** - ideja je da se podaci ravnomjerno raspodijele na više uređaja, bez ponavljanja upisa (**striping** - "strajping"). Podaci su izdijeljeni na relativno male dijelove (**stripe** - "strajp") od nekoliko kilobajta do megabajta i smiještaju se naizmjenično (*round-robin*) na sve uređaje. Sve dok radimo sa blokovima podataka koji su veći od strajpa (bitno je pravilno odabrati veličinu strajpa) ostvarujemo bolje performanse. Na primjer, upis 16 MB podataka na 4 diska u RAID 0 se dijeli tako da svaki od diskova upisuje (u paraleli) po 4 MB (umjesto 16 MB na jedan disk) čime ostvarujemo četiri puta bolje performanse. Pri čitanju istih podataka svaki od diskova će pročitati 4 MB i takođe ostvariti četiri puta bolje performanse. Mana je to što otkaz jednog diska znači i gubljenje svih podataka (jer je gotovo svaki veći fajl izdijeljen na više diskova i upravo smo izgubili 25% sadržaja gotovo svakog fajla). Praktično plaćamo performanse i kapacitet pouzdanošću. Minimalan broj diskova je 2.

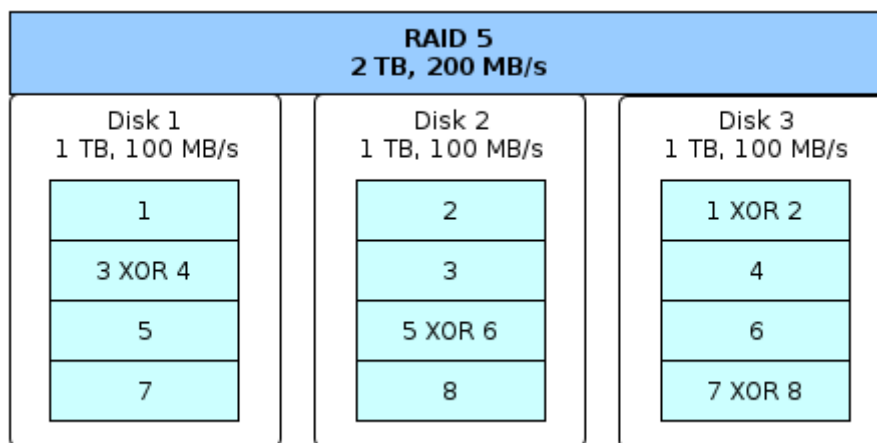


Slika 41: Primjer RAID 0 i RAID 1 nivoa

- **RAID 1** - ideja je da se svi podaci upisuju u paraleli na veći broj diskova (**mirroring**). Podaci se i dalje dijele na relativno male dijelove, ali samo interno, jer se svi dijelovi upisuju na sve diskove. Na ovaj način ne dobijamo bolje performanse (jer svaki disk upisuje sve podatke), ali smo zaštićeni od otkaza bilo kojeg diska (ili više njih). Sve dok postoji makar

jedan disk koji ima ispravne podatke, nemamo problem. Ovaj RAID nivo nas štiti ne samo od otkaza kompletnog diska, nego i od djelimičnog otkaza (npr. neispravljive greške čitanja). U nekim implementacijama je moguća tako da se upis vrši na sve diskove, ali se pri čitanju čita kao da je riječ o RAID 0 vezi (tj, u našem primjeru, svaki disk će pročitati samo 25% podataka i na taj način dobijamo četiri puta brže čitanje). Praktično plaćamo pouzdanost kapacitetom. Minimalan broj diskova je 2.

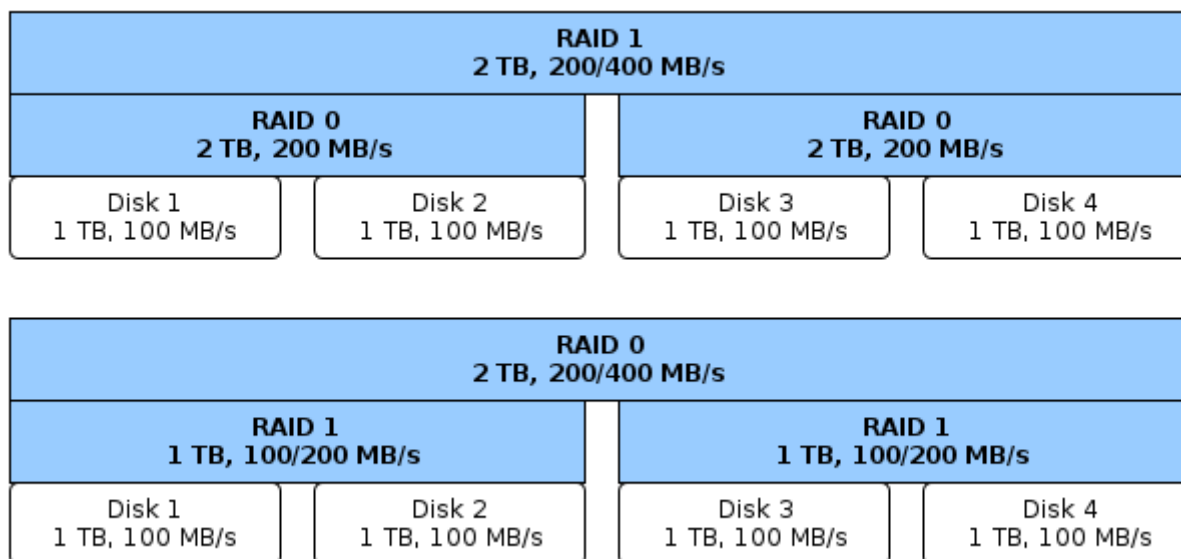
- RAID 2, 3 i 4 - nezanimljivi jer se ne koriste u praksi.
- **RAID 5** - ideja je da se ostvare bolje performanse i veći kapacitet uz bolju pouzdanost. Podaci se dijele, zatim upisuju kao da je riječ o RAID 0, ali sa diskom manje. Zatim se na preostali disk upisuje kontrolna suma (*parity*) obično izračunata kao XOR svih dijelova. Kako se pri svakoj izmjeni podataka ponovo mora računati i upisati parity, ona se distribuira među diskovima (da bi ih ravnomjerno opteretili), kao što je prikazano na slici 34. Pri čitanju čitamo sa svih diskova (i računamo parity za provjeru) i ostvarujemo bolje performanse. Ostvarujemo i veći kapacitet jer je on sada jednak sumi kapaciteta bez jednog diska. Očigledno je da je za RAID 5 minimalan broj diskova 3. Takođe smo zaštićeni od otkaza jednog diska jer njegov sadržaj uvijek možemo izračunati na osnovu sadržaja drugih diskova. Performanse nisu baš linearno skalabilne jer pri izmjeni male količine podataka koji se nalaze samo na jednom disku i dalje moramo da pročitamo i sadržaj drugih diskova da bi mogli izračunati parity, ali za potrebe analize možemo da to zanemarimo (jer postoje načini da se umanje ti negativni efekti). Predstavlja dobro rješenje za servere koji rade sa velikim upisima i čitanjima (bekap), ali ne i za servere koji rade sa velikim brojem malih upisa i čitanja (server baze podataka).



Slika 42: Primjer RAID 5 nivoa

- **RAID 6** - slično kao RAID 5, ali sa dva diska za parity. Na ovaj način smo zaštićeni od otkaza dva diska, ali po cijenu gubitka dijela kapaciteta. Kod većih RAID instalacija je poželjna upotreba RAID 6 umjesto RAID 5 jer proces regenerisanja sadržaja diska pri izmjeni neispravnog može trajati prilično dugo. Ne samo da za to vrijeme imamo degradirane performanse (jer intenzivno koristimo sve diskove) nego i nismo zaštićeni od otkaza (a govorimo o dužoj intenzivnoj upotrebi - idealno za otkaz).
- **RAID 0+1 i 1+0** - ideja je da se kombinuju najbolje osobine RAID 0 i RAID 1 nivoa, te da obezbijedimo bolje performanse i veću pouzdanost (za linearno više novca). Primjer oba pristupa je prikazan na slici 34. Iako oba pristupa obezbjeđuju isti kapacitet i iste

performanse sa istim brojem diskova raspoređenih na isti način, pouzdanost je različita. Ako u prvom slučaju (slika gore) dođe do otkaza bilo kog (jednog) diska, podaci će biti sačuvani. Ako je otkazao disk 1, sada imamo degradirani RAID 1 (jer koristi samo desnu stranu i diskove 3 i 4). Ako otkaze bilo koji od diskova 3 ili 4 - gubimo podatke. U drugom slučaju, ako otkaze disk 1, oba RAID 1 i dalje rade, a zaštićeni smo i od otkaza bilo kog diska osim diska 2. Iz tog razloga se u praksi obično na nižem nivou koristi RAID 1, a na višem RAID 0 (**RAID 10**). Postoje i kontroleri koji će ovakvu organizaciju posmatrati kao jedinstven RAID (bez nivoa) i koji će u oba slučaja sačuvati podatke sve dok su oni dostupni na bilo kojoj kombinaciji diskova.



Slika 43: Primjer RAID 0+1 i RAID 1+0

- JBOD (*Just a Bunch Of Disks*) - jednostavno "nadovezivanje" sadržaja i kapaciteta diskova, kao RAID 0 ali bez boljih performansi i uz veću nadu za oporavak pri otkazu diska (jer većina sačuvanih fajlova neće biti djelimično oštećena). Obično se koristi u okviru većih kompleksnih RAID instalacija.

Nivo	Disk X TB, Y MB/s	Broj diskova N	Kapacitet	Performanse W, R	Otkaz
RAID 0	1 TB 100 MB/s	6	6 TB $X*N$	600 MB/s $Y*N$	0 0
RAID 1	1 TB 100 MB/s	6	1 TB $X*1$	100, 600 MB/s $Y, Y*N$	5 $N-1$
RAID 5	1 TB 100 MB/s	6	5 TB $X*(N-1)$	<500 MB/s < $Y*(N-1)$	1 1
RAID 6	1 TB 100 MB/s	6	4 TB $X*(N-2)$	<400 MB/s < $Y*(N-2)$	2 2
RAID 10	1 TB 100 MB/s	6	3 TB $X*N/2$	300, 600 MB/s $Y*N/2, Y*N$	2 2

35. Telekomunikacioni uređaji

Kako je oblast računarskih telekomunikacija i računarskih mreža oblast koja će biti detaljno izučena u drugim predmetima, u ovom poglavlju će biti dat samo kratki pregled uz specifičnosti interesantne sa stanovišta arhitekture računarskih sistema.

Kritična spoznaja je da su danas praktično sve mreže računarske. Gotovo je nemoguće zamisliti telekomunikacioni sistem koji funkcioniše bez računarske podrške, prije svega zbog činjenice da su svi moderni sistemi digitalne prirode i da se analogni sistemi povlače iz upotrebe. Čim se radi o digitalnim komunikacijama, očigledno je da je svaki uređaj koji radi bilo kakvu operaciju nad podacima - računar. Ovdje možemo da isključimo uređaje koji rade na fizičkom nivou signala (pojačavači, filteri, itd) i koji ne uzimaju u obzir podatke nego samo signal (ovi uređaji nisu od interesa za nas). Svi drugi uređaji vrše odrađenu obradu ili usmjeravanje podataka na osnovu njihovog sadržaja što znači da ih možemo posmatrati kao računarske sisteme koji posjeduju određeni broj mrežnih kartica, koje mogu biti različite prirode i načina rada (npr. ethernet i wireless, ali i ATM, ISDN ili neka druga tehnologija).

Ukoliko je potrebno da računar analizira ili obrađuje kompletan sadržaj toka podataka, taj posao će biti tipično obavljen u procesoru. U većini slučajeva i specijalizovanih mrežnih uređaja (svičevi i ruteri) nije potrebno analizirati sve podatke već se zadržati na osnovnim podacima neophodnim za preusmjeravanje toka podataka (npr. mrežne adrese, portovi i slično). U tom slučaju je većinu posla moguće realizovati na specijalizovanim karticama dok je centralni procesor zadužen za konfigurisanje, upravljanje i nadzor navedenih kartica na višem nivou (ovo je tipičan pristup kod rutera visokih performansi).

Od posebnog značaja za nas su uređaji koji se koriste u okviru računarskih sistema sa ciljem ostvarivanja telekomunikacione veze. U prošlosti je u upotrebi bio veliki broj fundamentalno različitih tehnologija, od modema koji koriste telefonsku ili kablovsku mrežu, preko ISDN, ATM i Frame Relay, raznih vrsta bežičnih adaptera do ethernet kartica. Danas su u upotrebi gotovo isključivo mreže zasnovane na svega nekoliko tehnologija od kojih su dominantne Ethernet (IEEE 802.3) za žičane i Wi-Fi (IEEE 802.11) za bežične komunikacije. U upotrebi i fazi razvoja je još nekoliko tehnologija koje su namijenjene za specifične upotrebe, a koje će biti ukratko obrađene. Mrežni i komunikacioni aspekti različitih tehnologija nisu od interesa za nas, tako da ćemo pažnju posvetiti hardverskim specifičnostima. Takođe, osim ako je navedeno, govorimo o IP zasnovanim mrežama.

Kada govorimo o **Ethernet** mrežama, svi podaci se dijele u datagrame (fragmente, pakete - u zavisnosti od toga na šta se tačno misli) i prenose između dvije kartice. Procesor može periodično propitivati karticu da li ima novih podataka (*polling*) ili kartica može po potrebi obavijestiti procesor da ima nove podatke tako što će inicirati prekid (*interrupt*). Prvi pristup nije efikasan i praksi se gotovo uvijek koristi drugi pristup zasnovan na prekidima. Problem sa ovim pristupom leži u činjenici da su Ethernet datagrami relativno male veličine (1500 do 9000 bajtova) i da u slučaju upotrebe brzih linkova (10G Ethernet) govorimo o velikom broju prekida u sekundi (i to po portu), a sami prekidi nisu trivijalni za obradu i zahtijevaju angažman centralnog procesora. Kako je moguće i da datagrami dolaze u izmijenjenom redoslijedu (zbog prolaska različitim putanjama između krajnjih uređaja) i kako treba i računati zaštitne kodove (*checksum*) i vršiti ponovno objedinjavanje fragmentisanih podataka (i van redoslijeda prijema), očigledno je da sam centralni

procesor sada mora da izvršava prilično kompleksan zadatak samo da bi komunikacija funkcionisala na odgovarajući način. Da bi riješili taj problem, uvedena je podrška za **TOE (TCP offload engine)** koji omogućava da namjenski procesor na mrežnoj kartici izvršava sve navedene poslove i da na efikasan način prenosi podatke u i iz memorije bez opterećivanja centralnog procesora. Ovakve kartice sada na sebe preuzimaju sve operacije vezane za TCP/IP i naročito su prisutne u serverskim rješenjima kao što su kartice sa većim brojem portova ili kartice koje rade na većim brzinama (10G, 40G ili 100G Ethernet).

Wi-Fi nije jedna tehnologija, nego skup tehnologija koji zajedno čine IEEE 802.11 standard. Wi-Fi koristi dva frekventna opsega, prvi na 2,4 GHz (802.11, 802.11b, 802.11g) i drugi na 5 GHz (802.11a, 802.11ac, 802.11n, 802.11ax), ali i kombinaciju oba opsega. Kako je razvijana tehnologija, tako su poboljšavane i performanse i brzina prenosa podataka, od 802.11b i 11 Mbps, preko 802.11g i 54 Mbps, 802.11n i 600 Mbps, 802.11ac i 7 Gbps i 802.11ax i gotovo 10 Gbps. Navedene vrijednosti su **teoretske** maksimalne vrijednosti i zbog prirode bežičnih komunikacija (različita udaljenost terminala, smetnje, prepreke, upijanje i odbijanje signala, itd) u praksi moguće ostvariti bitno manje vrijednosti, naročito u slučaju novijih standarda. Oba navedena frekventna opsega su namijenjena za upotrebu od strane uređaja male snage (100mW) što ograničava i realno ostvarive brzine i maksimalnu udaljenost između antena terminala. U idealnim slučajevima i vrlo malim udaljenostima, moguće je ostvariti brzine od približno 1 Gbps, a maksimalna udaljenost (bez upotrebe posebnih antena, pojačavača signala, itd) je nekoliko stotina metara ako postoji linija vidljivosti, odnosno približno 20 m ako ne postoji (unutrašnjost zgrada, itd, zavisi od prepreka i njihove prirode). Povezivanje na računar je moguće putem USB-a, ali su često integrisani u čipset i veza se ostvaruje putem PCIe linka.

Ukoliko je potrebno ostvariti komunikaciju na manjim udaljenostima, moguće je koristiti i **Bluetooth**. Osnovna razlika između Wi-Fi tehnologije i Bluetootha je u tome što je Wi-Fi namijenjen za bežične lokalne mreže (WLAN), a Bluetooth za bežične lične mreže (WPAN - *Wireless Personal Area Network*) i dominantno se koriste kao zamjena za žičanu vezu između dva uređaja od kojih je jedan periferija, a veza je privremene (*ad-hoc*) prirode. Gdje je ideja Wi-Fi-ja da obezbijedi najbolje performanse, Bluetooth zadovoljava i upotrebu u situacijama u kojima je kritično ograničenje potrošnje energije (Bluetooth LE). Kako su stvari uvijek komplikovanije nego što se čini, Bluetooth od verzije 3.0 može, ali ne mora, koristiti i Wi-Fi za prenos podataka velikom brzinom. Kako ovi uređaji ne ostvaruju brzine prenosa veće od onih koje su u okviru mogućnosti USB-a, često su interno u računaru povezani putem USB interfejsa.

Još jedna vrlo popularna bežična mreža je mobilna mreža. Ova oblast je *izuzetno* kompleksna tako da ćemo samo pobrojati osnovne trenutno aktivne generacije: 2G (GSM) uz EDGE i je omogućio sve osnovne operacije mobilnih mreža (prenos glasa, poruka, podataka, itd), 3G uz HSDPA (3.5G) je obezbijedio dovoljno brz prenos podataka koji je postao upotrebljiv kao način pristupa internetu i sa računara, 4G (LTE) poboljšava dalje performanse i omogućava pristup na kvalitetan način multimedijalnim sadržajima i računarskim resursima na mreži (HD video, cloud computing, itd) uz bitna poboljšanja za operatore infrastrukture koji mogu efikasnije da koriste resurse i da opsluže veći broj korisnika u istom frekventnom opsegu (koji je konačan opseg i mora se pažljivo koristiti), dok 5G predstavlja evoluciju tehnologije i bitno bolje performanse u određenim frekventnim opsezima (mmWave) koji uvode i brojna ograničenja (mala fizička udaljenost i mnogo više ćelija). Računarski pristup ovim mrežama je moguć ili dijeljenjem veze od strane mobilnog terminala ali i putem namjenskih (USB) uređaja koji emuliraju mrežne kartice.

Tradicionalni satelitski pristup internet servisima je male brzine i velikog kašnjenja podataka (jer su sateliti obično geostacionarni i vrlo daleko od površine planete, oko 35786 km, a samim tim i od korisnika, što znači da signal mora da premosti velike udaljenosti što uvodi kašnjenje i smanjuje performanse). Alternativni pristup je upotreba velikog broja satelita koji se nalaze u niskoj orbiti LEO (*Low Earth Orbit*) koja uključuje sve orbite sa udaljenošću (od površine) manjom od 2000 km. Kompanija SpaceX je razvila Starlink skup satelita koji se nalaze na vrlo maloj visini (na 340 ili 550 km) i zbog toga mogu obezbijediti malo kašnjenje i velike brzine prenosa podataka. Nažalost, taj pristup zahtijeva veliki broj (trenutno oko 1700, kompletan sistem sa više od 40.000) relativno malih (~260 kg) satelita.

Ovaj sistem nije zamjena za mobilnu telefoniju jer su korisnički terminali velikih dimenzija (npr. 48x48 cm), ali omogućava gotovo globalno pokrivanje. Kako još nisu lansirani svi sateliti (lansiranje trenutno ide u skupovima od približno 60 satelita po lansiranju), trenutne performanse ne odražavaju performanse konačnog sistema, ali je već moguće ostvariti brzine primanja i slanja podataka od 100 i 30 Mbps uz kašnjenje od 20 do 40 ms što su izuzetne vrijednosti u poređenju sa klasičnim satelitskim internetom (0.5 Mbps i kašnjenje veće od 1000 ms). Ovo je naročito važno pri uspostavljanju konekcije jer je često potrebno razmijeniti kriptografske ključeve i druge poruke što dovodi do kašnjenja od 10+ s prije nego što uopšte veza bude uspostavljena putem klasičnog satelitskog interneta. Jedan od nusproizvoda ovog pristupa je i razvoj potpuno drugačijih sistema za lansiranje (Falcon 9, Falcon Heavy, StarShip). Pored SpaceX i Starlinka, i druge kompanije rade na sličnim pristupima (Amazon Kuiper, OneWeb, itd). Ukoliko sistemi budu u potpunosti realizovani, što uključuje i direktne laserske veze između satelita koji tada aktivno rutiraju saobraćaj, a zbog zakrivljenosti površine planete i manje brzine svjetlosti u vlaknima u odnosu na vakuum, ovakav satelitski internet bi mogao da obezbijedi manje kašnjenje na većim udaljenostima nego direktna optička veza.

36. Ulazni i izlazni uređaji

Jedna od osnovnih funkcija računara je ostvarivanje komunikacije sa ljudima i okolinom, pri čemu je za nas od najvećeg interesa upravo interakcija između čovjeka i računara. Uređaji koji omogućavaju realizaciju te funkcionalnosti se nazivaju UI (ulaz-izlaz) uređaji ili I/O (*input/output*) uređaji. Naravno, ne postoji jasna podjela na ulazne i izlazne uređaje i u većini slučajeva se radi o kombinaciji ove dvije vrste komunikacije. Čak i u slučajevima kada govorimo o uređajima koji nam izgledaju isključivo kao ulazni, recimo tastatura, može postojati i dio koji nam prenosi informacije, poput statusnih oznaka za *num lock* ili *caps lock*. Slična je situacija i u radu sa uređajima koji izgledaju kao isključivo izlazni, poput ekrana, koji i dalje ima dvosmjernu komunikaciju prema računaru i prenosi mu informacije o tome da li je korisnik isključio prikaz ili promijenio neko podešavanje. Za potrebe ovog poglavlja možemo uređaje podijeliti prema osnovnoj funkcionalnosti, bez ulaženja u detalje slične prethodno navedenim.

Računarski ulazni uređaji (ulazne jedinice) su uređaji koji omogućavaju unos podataka i upravljačkih signala u računarski sistem. Najčešći ulazni uređaji su:

- **Tastatura** - prevashodno namijenjena za unos tekstualnog i numeričkog sadržaja. Može biti realizovana kao elektro-mehanička (sa pokretnim tasterima) ili virtuelno-elektronska (virtuelni tasteri, projekcione tastature).
- **Pokazivački uređaji** - ova klasa uređaja omogućava korisniku da računaru saopšti poziciju od interesa i danas predstavlja najčešći ulazni uređaj. Postoji više načina podjele, od kojih su za nas interesantna dva. Prva podjela je po tome da li se pozicija izražava direktno ili indirektno. Direktan unos se vrši pri upotrebi ekrana osjetljivih na dodir jer smo u stanju da *direktno* odaberemo željeni element ili poziciju na ekranu. Indirektni unos nam omogućava da uređajem kontrolišemo pokazivač na ekranu pomoću kojeg određujemo element ili poziciju koju želimo. Ne postoji bolji ili gori pristup, riječ je o dva različita načina saopštavanja iste informacije. Direktni pristup može biti jednostavniji za shvatanje, ali je često ograničen po pitanju preciznosti i naprednih mogućnosti bez dodatnog hardvera (npr. prst je mnogo neprecizniji od olovke, prst nema različite tastere sa posebnim funkcijama kao miš, itd). Osnovni predstavnici ove grupe uređaja su: miš, površine osjetljive na dodir (*touchpad*, grafičke table), uređaji zasnovani na upravljanju objektom (*trackball*, *pointing stick*), svjetlosne olovke, itd.
- **Kontroleri za igre** - predstavljaju namjenske uređaje za upravljanje računarskim igrama. Iako se mogu svesti u jednu od dvije prethodne grupe, dovoljno su prisutni i specifični da ih možemo izdvojiti u i posebnu. Kao osnovna zajednička osobina se uzima mogućnost da pomoću njih vršimo unos više vrsta podataka (npr. tasteri i pokazivački podaci i to u više ulaza). Popularni predstavnici su: džojstik (*joystick*), gejmped (*gamepad*), kontroleri koji simuliraju upravljanje vozilima ili instrumentima, kao i mnogobrojni uređaji zasnovani na detekciji fizičkih pokreta (*Wii remote*, *Oculus controller*, itd).
- **Audio i video uređaji** - služe za akviziciju audio i video podataka. Kako je riječ o vrlo važnim i specifičnim uređajima, izdvojeni su u posebnu sekciju (vidi poglavlje 37). Obuhvataju: mikrofone, kamere, foto aparate, itd.

- **Skeneri** - služe za prenos dvodimenzionalnih slika iz fizičkog svijeta u računarski sistem. Mogu služiti za prenos teksta ili slika sa papira ili srodnog medija (*image scanner*) i tipično se realizuju kao namjenske kamere koje digitalizuju podatke liniju po liniju. Na sličnom principu funkcionišu i skeneri otiska prsta ili bar kodova, ali sa prilagođavanjima za specifične namjene. Posebna klasa uređaja su 3D skeneri koji sami mogu biti realizovani na veliki broj načina, od fizičkog unošenja pozicija tačaka uređaja, preko izračunavanja oblika objekta na osnovu deformacije projektovanog uzorka, do određivanja oblika na osnovu fotogrametrije.

Računarski izlazni uređaji (izlazne jedinice) su uređaji koji omogućavaju konverziju podataka prisutnih u računarskom sistemu u oblik koji je pogodan za konzumiranje od strane korisnika sistema. Kada govorimo o izlaznim uređajima, najčešće vrste su:

- **Monitori** - omogućava prikaz podataka u vidu dvodimenzionalnih slika sastavljenih od piksela. Obično su povezani preko grafičke kartice (zasebne ili integrisane) i može biti realizovan velikim brojem tehnologija. Kako je riječ o izuzetno važnom uređaju, detaljnije je obrađen u narednom poglavlju.
- **Zvučnici i slušalice** - omogućavaju generisanje zvuka na osnovu električnog audio signala. Obično su priključeni posredstvom audio (zvučne) kartice koja pretvara podatke u audio signal.
- **Štampači** - uređaji koji omogućavaju prenos dvodimenzionalne slike predstavljene tačkama (*raster*) na neki medij, najčešće papir. Generisani izlaz može biti namijenjen ljudima, ali i drugim računarskim sistemima (npr. bar i QR kodovi). Postoji veliki broj tehnologija koje se koriste za realizaciju štampača, a najpopularnije vrste štampača su: laserski (koriste elektrostatičko prijanjanje da prenesu toner preko valjka na papir) koji obezbjeđuju izuzetno visoku rezoluciju i kvalitet štampe, *ink-jet* (koriste tečnu tintu koju prenose na papir) koji imaju jednostavniju konstrukciju i manju cijenu uređaja, *solid-ink* (koriste čvrstu tintu koju tope i prenose na papir) koji mogu da rade sa širom paletom medija za štampu, te termalni štampači (prenose sliku zagrijavanjem posebne vrste papira) koji su česti u kasama i sličnim uređajima. Kao posebna vrsta štampača mogu se posmatrati **ploteri** - takođe uređaji koji prenose sliku na medij, ali za taj proces koriste pera i emuliraju proces crtanja. Ne koriste rasterizaciju jer su im potrebne linije što ih čini pogodnim za izradu tehničkih crteža velikih formata (npr. mnogo jednostavnije je proširiti pokretljivost pera nego napraviti mnogo širi valjak i laserski štampač). Razvojem ink-jet i solid-ink štampača polako izlaze iz upotrebe.
- **Brajevi terminali** - uređaji namijenjeni za slijepa i slabovidna lica. Najčešće su realizovani kao matrični uređaji koji mogu predstaviti određeni broj znakova Brajevim pismom u jednom trenutku, a po potrebi mogu znakove osvježavati ili pomijerati kako bi prikazali više sadržaja korisniku.

Postoje i uređaji koji primarno omogućavaju dvosmjernu komunikaciju kao osnovni način rada i nazivaju se ulazno-izlazni uređaji. Najčešće su realizovani kao složeni uređaji koji se sastoje od izlaznog i ulaznog dijela. Najčešći primjer su ekrani osjetljivi na dodir (*touch-screen*) koji se sastoje od ekranske komponente za prikaz podataka i površine osjetljive na dodir koja se nalazi iznad ekrana ili je integrisana u strukturu istog. Danas je izuzetno teško pronaći smartphone koji ne koristi ovakav način interakcije sa korisnikom (direktan korisnički interfejs, prilagodljiv u softveru, bez pokretnih dijelova i pogodan za integrisanu realizaciju - više mogućnosti po manjoj cijeni).

37. Audio i video uređaji

Audio i video uređaji mogu biti ulazni (mikrofon) i izlazni (monitor), kao i ulazno-izlazni (monitor sa integrisanom kamerom), u zavisnosti od potrebe.

Predstavnici koji će biti detaljnije obrađeni u ovom poglavlju su: mikrofoni, zvučnici, monitori i kamere.

Mikrofoni i zvučnici

Mikrofoni su uređaji koji pretvaraju zvučnu energiju u električnu koja se posredstvom zvučne kartice pretvara u digitalni oblik pogodan za pohranu i obradu u računarskim sistemima. Mogu biti i digitalni mikrofoni koji interno vrše konverziju u digitalni oblik (digitalni mikrofon).

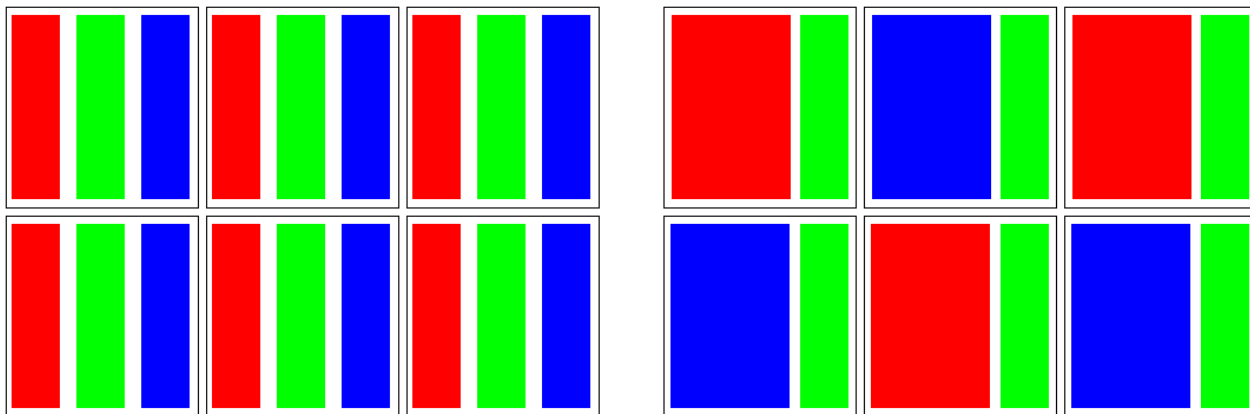
Postoji veliki broj različitih načina realizacije mikrofona, ali su najčešći dinamički i kondenzatorski. Dinamički mikrofoni rade na principu mjerenja promjene induktivnosti tako što u polje permanentnog magneta postavljaju indukcionu namotaj. Pri promjeni vazdušnog pritiska (zvuk) dolazi do pomjeranja membrane spojene na namotaj što dovodi do indukcije i stvaranja električne energije. Tipično su jeftiniji i robusniji, pogodni za vrlo glasne zvukove, ali su slabo osjetljivi na tihe zvukove. Kod kondenzatorskih mikrofona membrana je spojena na pokretljivu ploču kondenzatora što znači da pomjeranje membrane dovodi do promjene kapacitivnosti kondenzatora što je moguće pretvoriti u električni signal. Vrlo su osjetljivi i precizni, ali nisu dobro rješenje za glasna okruženja, tipično su više cijenjene. Posebna klasa uređaja su MEMS (*MicroElectrical-Mechanical System*) mikrofoni koji se realizuju direktno na čipu (silicijumu) i sastoje se od minijaturene dijafragme, integrisanog pojačavača signala i (tipično) kola za ADC (*Analog-Digital Conversion*).

Mikrofoni ne moraju biti jednako osjetljivi na zvukove koji dolaze sa svih strana i prema tome ih možemo podijeliti na više kategorija od kojih su nama interesantne tri osnovne: omnidirekciono mikrofoni su jednako osjetljivi bez obzira na ugao pod kojim zvuk dolazi do mikrofona, kardioidni su osjetljivi na zvukove koji dolaze samo sa jedne hemisfere, a usmjereni su osjetljivi samo na zvuk koji dolazi pod uskim uglom. Pri snimanju je vrlo važno odabrati pravi tip mikrofona, na primjer za potrebe snimanja predavanja omnidirekciono će snimiti i sve zvukove okoline, kardioidni okrenut prema govorniku će ignorisati publiku, a usmjereni će zahtijevati od govornika da bude stacionaran, ali će moći možda čak izdvojiti pojedinačnog govornika iz mnoštva.

Zvučnici pretvaraju električni signal u zvuk. Najčešće korišćena vrsta zvučnika je dinamički zvučnik koji radi na istom principu kao i dinamički mikrofon, ali na suprotan način. Naime, promjena električne struje u namotaju se koristi za pomjeranje membrane što dovodi do nastanka zvuka. Imajući ovo u vidu, očigledno je da je i zvučnike moguće koristiti kao (ne baš dobre) mikrofone (primjer je upotreba za analogne interfonske sisteme).

Monitori

Računarski monitor je izlazni uređaj namijenjen za prikaz dvodimenzionalnih slika. Sama slika se sastoji od matrice piksela (tačaka) pri čemu je svaki piksel tipično sastavljen od tri komponente koje odgovaraju crvenom (R), zelenom (G) i plavom (B) izvoru svjetlosti iz vidljivog dijela spektra što omogućava prikaz kolor slika (RGB). Ukoliko je aktivna samo jedna komponenta, recimo R, piksel će biti crvene boje. Ukoliko su aktivne dvije komponente, recimo R i G, piksel će biti žute boje, a ako su aktivne sve tri komponente piksel će biti bijel. Ukoliko nije aktivna nijedna komponenta, piksel će biti crn. Variranjem intenziteta svake od komponenata moguće je ostvariti reprodukciju velikog broja boja. Na primjer, ako intenzitet predstavimo 8-bitnom vrijednošću dolazimo do 256 mogućih nivoa osvjetljenja za svaku od komponenti (0-255) što znači da je moguće predstaviti 2^{24} , odnosno 16777216 kombinacija (boja, nijansi). Kako ljudsko oko nije jednako osjetljivo na sve talasne dužine (najosjetljivije je na zelenu boju) i često nije moguće generisati jednak intenzitet svjetlosti za sve komponente, one ne moraju biti jednake veličine u svim slučajevima. Takođe, ljudsko oko ne vidi promjene svijetlo-tamno i promjene u bojama na isti način (zeleno boja se potencira kod detalja) tako da je moguće piksel napraviti i na taj način da svaki piksel ne sadrži sve komponente, često bez vidljive degradacije (*pentile RGBG*), slika 44.



Slika 44: Primjer RGB (lijevo) i pentile RGBG (desno) komponenti piksela

Postoje dva osnovna pristupa realizaciji ovog pristupa. Prvi se zasniva na postojanju izvora bijelog svjetla koje zatim filtriramo u R, G i B komponente i po potrebi ih zatamnjujemo, najčešće upotrebom tečnih kristala tako da se ova vrsta ekrana naziva LCD (*Liquid Crystal Display*). Prednost ove vrste ekrana je jednostavna i jeftina i robusna konstrukcija, te niska cijena, dok su mana relativno uzak ugao gledanja u kojem je slika korektna i relativno sporo osvježavanje slike. Obe mane je moguće riješiti određenim tehnologijama (*in-plane switching*, *active-matrix* i *fast-switching* LCD) ali po cijenu kompleksnije i skuplje izrade.

Drugi pristup je zasnovan na upotrebi individualnih izvora svjetla za svaku od komponenata piksela i zasniva se na upotrebi LED (*Light Emitting Diode*) elemenata. Ukoliko se koriste organski elementi za proizvodnju svjetla, tehnologija se naziva OLED (*Organic LED*). Ova vrsta ekrana ima mnogo bolje uglove gledanja i brzinu osvježavanja u odnosu na LCD, ali potrošnja električne energije zavisi od prikaza (crni pikseli gotovo da ne troše energiju, dok je bijeli zahtijevaju mnogo), sami ekrani su kompleksniji i skuplji za izradu. Mane ovog pristupa su vezane za organsku prirodu aktivnih jedinjenja i degradaciju performansi u zavisnosti od vremena upotrebe, te osjetljivost na vremenske uslove. Kako je plava komponenta problematična i najbrže degradira, OLED ekrani obično koriste *pentile* RGBG raspored (slika 44). Alternativa OLEDu je upotreba velikog broja

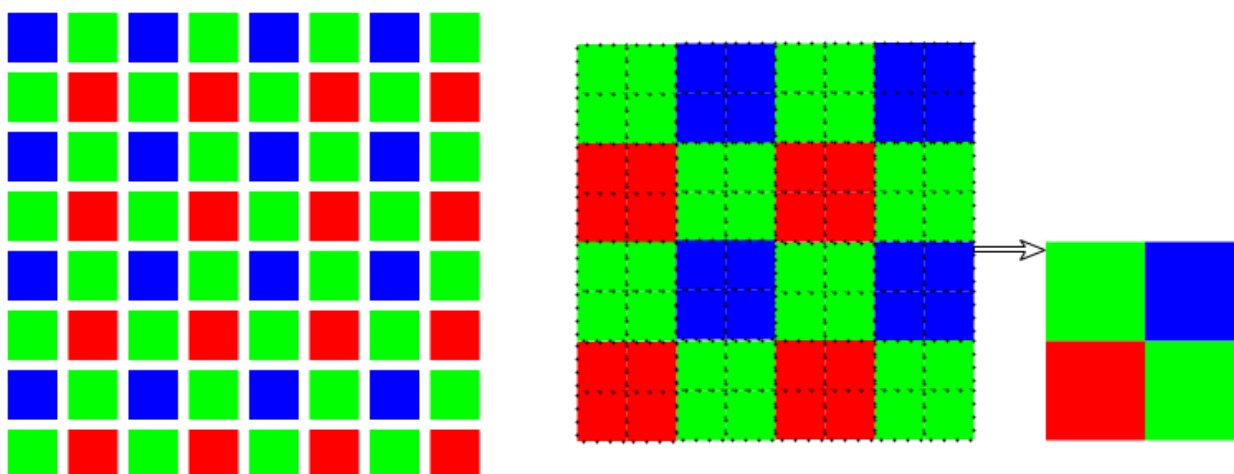
minijaturnih klasičnih LED komponenti koje se sad nazivaju mikro LED (*micro-LED* ili *mLED*). Ovo je tehnologija koja je još u fazi aktivnog razvoja, ali vjerovatno predstavlja budućnost ekrana (makar vrhunskih ekrana). i OLED i mikro LED ekrani mogu biti i providni i savitljivi što ih čini pogodnim za brojne namjene kao što su marketing, industrije zabave, industrijski viziri, itd.

Digitalne kamere

Digitalna kamera je kamera koja generiše sliku u digitalnom obliku. Za razliku od klasičnih analognih kamera koje su koristile film, digitalne kamere su (danas) gotovo sve u mogućnosti da prave i statičnu i pokretnu sliku, tj, mogu da generišu i fotografije i video zapise (izuzeci su vrlo rijetki i tipično predstavljaju ili visokospecijalizovane uređaje ili marketinški trik). Za potrebe analize sa stanovišta ovog predmeta, ne postoji bitna razlika između digitalnog fotoaparata, digitalne kamere i veb kamere, te ih možemo posmatrati kao jedinstven uređaj.

Pored objektiva, koji je potencijalno vrlo kompleksan dio sistema i čija analiza izlazi daleko van opsega ovog predmeta, najvažnija komponenta digitalne kamere je senzor koji pretvara sliku koju objektiv projektuje na njega u električni signal. Ovaj proces se odvija tako što se senzor sastoji od velikog broja elektronskih komponenti osjetljivih na svjetlo koje dolazne fotone pretvaraju u elektrone. Kada se proces ekspozicije završi, svaki od elemenata se "prazni" kroz konvertor koji pretvara količinu naelektrisanja u digitalnu reprezentaciju - ADC (analogno-digitalni konvertor). Na taj način za svaki element dobijamo po numeričku vrijednost koja odgovara količini svjetla koja je pala na njega, a kako su elementi složeni u matricu - dobijamo sliku. Kako je Si kao materijal osjetljiv (približno) isto na sve talasne dužine, slika koju dobijamo nosi samo informaciju o svjetlini što znači da je monohromatska (crno-bijela).

Da bi omogućili akviziciju i podataka o boji u sceni, potrebno je da nekako razdvojimo svjetlost na komponente, obično na R, G i B (kao u ranijem tekstu). Za ovaj proces postoji više načina, od prizmi i upotrebe tri senzora (3-CCD sistemi), preko iskorišćavanja činjenice da različite talasne dužine prodiru do različitih dubina u silicijumu (Foveon senzor) do, danas najčešćeg, upotrebe *Bayer* kolor filtera na senzoru (slika 45 lijevo).



Slika 45: Primjer Bayer (lijevo) i Quad-Bayer (desno) kolor filtera

U ovom slučaju se iznad svakog fotoosjetljivog elementa nalazi filter koji propušta samo jednu boju (R, G ili B). Kako je ljudsko oko najosjetljivije na zelenu boju (555 nm), tako je i broj zelenih filtera veći od broja crvenih i plavih (zelenih ima 50%, a crvenih i plavih po 25%). Sada za

svaki element imamo informaciju o intenzitetu svjetla određene boje, ali samo jedne boje za svaki element. Da bi stvari zakomplikovali, u praksi se najčešće koristi naziv "piksel" za svaki od elemenata, iako sadrži samo jednu komponentu (u odnosu na tradicionalne tri). To znači da kamera rezolucije 24 MP (mega piksela) ima matricu od (tipično) 6000 x 4000 individualnih elemenata, što odgovara klasičnom ekranu sa samo 8 MP. Da bi zaista i dobili sliku odgovarajućih dimenzija i koja za svaki piksel sadrži tri komponente, potrebno je izvršiti proces izračunavanja nedostajućih komponenti boje za svaki piksel (*demosaicing*). Pogodnim odabirom algoritma moguće je ostvariti vizuelno vrlo dobre rezultate i rezoluciju (praktično 12 mega "potpunih" piksela) koja se sa stanovišta ljudskog oka ne razlikuje bitno od marketinške (24 MP).

Naravno, postoje i alternativni pristupi i rasporedi filtera, kao što je Fuji X-Trans, ali je u prethodnih par godina izuzetno popularan postao Quad-Bayer pristup (slika 45 desno), naročito u oblasti mobilnih telefona. Naime, kako mobilni telefoni tradicionalno nemaju mogućnost posjedovanja više objektivna ili objektivna sa promjenljivom žižnom dužinom (*zoom*) postoji problem kako emulirati te osobine. Jedan od pristupa je jednostavna upotreba senzora izuzetno visoke rezolucije i prosto isjecanje (*crop*) srednjeg slike, ali je problem u to što nam nije uvijek potreban rad sa visokim rezolucijama, a i visoka rezolucija tradicionalno donosi lošije performanse pri slabom osvjetljenju i veće zahtjeve za procesorskim i memorijskim resursima. Quad-Bayer pristup omogućava rad u dva režima, prvi koji grupiše po četiri piksela u jedan (tako da sa 64 MP senzora dobijamo 16 MP sliku) i drugi koji nam omogućava sliku pune rezolucije (64 MP). Sada možemo u softveru birati da li želimo da radimo sa većom rezolucijom i omogućimo "zoom" ili da imamo manji šum i bolje rezultate u noćnim uslovima.

Vrijedi napomenuti da kamere gotovo uvijek imaju ugrađene i mikrofone i predstavljaju vrlo kompleksne uređaje. Moderne kamere su praktično kompletni računarski sistemi sa znatnim procesorskim i memorijskim resursima (npr. 4 jezgra, 8 GB RAM, izmjenjiva sekundarna memorija do 1 TB). Često simultano koriste dva operativna sistema, jedan za rad u realnom vremenu (namijenjen za kontrolu operacija kamere i objektivna na niskom nivou) i drugi za interakciju sa korisnikom (jako često derivat Linux ili *BSD operativnog sistema).

Na ovom primjeru možemo posmatrati progresiju vrlo jednostavnog mehaničkog uređaja jedne namjene koji je vremenom postao kompleksan računarski sistem sa više procesora, kompleksnom memorijom, dva ekrana (EVF i LCD), bežičnim komunikacionim interfejsima (a koristi se i Ethernet i USB-C i HDMI), a može i da pravi fotografije.

38. Grafičke kartice

Grafičke kartice, poznate i kao video kartice, omogućavaju generisanje izlaza u vidu slike koju je moguće prikazati na monitoru ili sličnom uređaju. Iako se često koristi i termin GPU (*Graphics Processing Unit*), upotreba ovog naziva nije korektna, jer je GPU samo dio grafičke kartice, na sličan način kao što je procesor dio računarskog sistema, čak i kad je u okviru procesora integrisan značajan dio drugih komponenti.

Razvojem računarstva i pojavom potrebe za grafičko predstavljanje podataka na monitorima, došlo je i do razvoja namjenskih jedinica - grafičkih kartica. U početku je njihova osnovna karakteristika bila da pojednostave prikaz upotrebom video memorije iz koje su podaci prikazivani. Naredni korak je bio uvođenje dvostrukog baferovanja tako da su postojale dvije instance video memorije što je omogućavalo izmjenu jedne dok se prikazuje sadržaj druge, zatim brza izmjena aktivne memorije i prikaza čitave slike. Ove kartice su omogućile upotrebu viših rezolucija i bolju reprodukciju boja što je bilo vrlo bitno za korisnike u određenim oblastima. Vremenom kartice dobijaju i druge funkcionalnosti, od namjenskih elemenata koji omogućavaju brzo kopiranje i pomjeranje memorije (skrolovanje), elementarne 2D transformacije (skaliranje, rotiranje), itd. Osnovne karakteristike su: rezolucija, broj bita po pikselu i frekvencija osvježavanja slike. Frekvencija osvježavanja određuje koliko puta u sekundi se vrši iscrtavanje sadržaja na ekran i tipično iznosi 60 Hz za LCD ekrane, ali ide i do 240 Hz (jer je na taj način, i uz upotrebu aktivnih naočala, moguće prikazati dvije različite slike za lijevo i desno oko i na taj način ostvariti stereoskopski prikaz). Broj bita po pikselu definiše broj boja koje je moguće prikazati (kao stepen broja dva) i kreće se od 1 (za monohromatske) do 24 (3 kanala, obično RGB, po 8 bita, dovoljno za preko 16 miliona boja - *True Color*), ali i do 64 bita (koji se koriste za druge namjene) ili čak i kao *floating point* vrijednost.

Postoji veliki broj rezolucija u upotrebi, ali standardne savremene rezolucije su:

- WXGA 1280 x 720
- HD 1366 x 768
- HD+ 1600 x 900
- FHD 1920 x 1080
- QHD 2560 x 1440
- 4k UHD 3840 x 2160
- DCI 4k 4096 x 2160
- 8k UHD 7680 x 4320

Uz naziv rezolucije se obično stavlja sufiks "p" ili "i" (npr. 720p) da bi se definisalo da li se sve linije iscrtavaju progresivno (*progressive scan*), odnosno jedna za drugom, ili prepleteno (*interlaced*), odnosno parne pa neparne. Danas su praktično sve rezolucije u upotrebi progresivne, što je i podrazumijevani pristup. Vrijedi istaći da je upotreba vrlo visokih rezolucija često namijenjena za obradu a ne za isporuku video zapisa jer korisnici u standardnim uslovima nisu u stanju da primijete razliku. Čak je i kod razlike između FHD i 4k UHD rezolucija značajnija razlika

podrška za bolju reprodukciju boja i veći dinamički raspon (odnos između najveće i najmanje vrijednosti osvjettljenja), naročito za realne video zapise, a ne statične slike i test signale.

Razvojem upotrebe 3D grafike u računarskim sistemima, grafičke kartice sve više prerastaju u sisteme za rad u 3D i obradu video signala (prve popularne grafičke kartice su bile firme 3dfx sa imenima poput Voodoo, Rush i Banshee). Ove kartice su bile implementirane kao kompletni sistemi sa posebnim procesorom i radnom memorijom, te su komunicirale sa ostatkom računarskog sistema upotrebom najnaprednijih magistrala. Uloga grafičkih kartica je od velike važnosti za razvoj internih magistrala u računarima jer je upravo želja za što boljim grafičkim performansama bila razlog za razvoj VLB, PCI, AGP i PCIe standarda. Zanimljivo je da je i danas situacija vrlo slična jer savremene vrhunske kartice i koštaju više od procesora, imaju uporedivu količinu RAM-a, a i po potrošnji električne energije mogu da posrame i manja tijela.

Grafičke kartice mogu biti interne i eksterne. Interne grafičke kartice su ranije bile integrisane u čipset matične ploče, ali su vremenom migrirale u procesor (koji sada slobodno možemo zvati procesorski modul). Na taj način je omogućena jednostavnija i efikasnija komunikacija između procesora i GPUa, uz zajedničku upotrebu dijeljenih komponenti potrebnih za obe jedinice. Eksterne grafičke kartice su povezane pomoću, danas gotovo isključivo, PCIe magistrale i predstavljaju izuzetno kompleksne sisteme koji ne moraju biti ograničeni na upotrebu kao grafičke kartice. Kako je riječ o fizički odvojenim karticama koje je moguće nezavisno napajati i hladiti, mogu imati bitno bolje performanse u odnosu na integrisane (koje su ograničene energetske i termalnim budžetom procesorskog modula). Još jedan razlog za bolje performanse eksternih kartica leži u činjenici da je u tom slučaju moguće koristiti veliki blok namjenske radne memorije optimizovane za rad sa GPUom, dok integrisane grafičke kartice ili dijele osnovnu radnu memoriju ili posjeduju manje količine posebne memorije. Integrisane grafičke kartice je u pravilu moguće isključiti i koristiti samo eksternu karticu, ali su moguće i kombinacije u kojima se koriste resursi oba grafička procesora sa ciljem ostvarivanja boljih performansi ili manje potrošnje električne energije, što je važno kod prenosnih uređaja (Nvidia Optimus, AMD PowerXpress). Određene eksterne grafičke kartice, uz podršku čipseta, omogućavaju simultanu upotrebu više kartica i njihovo posmatranje kao jedne kartice (NVIDIA SLI i AMD CrossFire).

Danas su najvažniji proizvođači GPUa NVIDIA i AMD uz veliki broj proizvođača grafičkih kartica koji koriste gotova AMD i NVIDIA rješenja uz određena prilagođavanja specifičnim potrebama. Osnovnu ponudu NVIDIA GPU čine GeForce (u desktop i mobilnoj varijanti) za sve osnovne primjene, Quadro (u desktop i mobilnoj varijanti) za napredne radne stanice kod kojih je bitna podrška za ECC memoriju i veću floating-point preciznost, te NVIDIA Tesla porodica kartica koje uopšte nisu grafičke već su namijenjene za izračunavanja u superračunarskom okruženju i podržavaju mnogo veću memoriju (40 GB) i više jezgara (6912) i bolje performanse (9,7 TFLOPS). AMD kartice možemo podijeliti u Radeon kao osnovnu verziju, Radeon Pro kao verziju za radne stanice i servere i Radeon Instinct namijenjenu za superračunarska okruženja (32 GB, 7680 jezgara i 11,5 TFLOPS).

39. Pregled ISA

Kao što je ranije navedeno, postoje dvije osnovne grane ISA: CISC i RISC. Sticajem okolnosti i stanja na tržištu, broj aktivnih ISA je bitno smanjen u prethodnom periodu i danas se možemo zadržati na sljedećim ISA:

- **x86** - danas tipično u x86-64 varijanti poznatoj i kao AMD64. Trenutno dominantna ISA na tržištu ličnih (desktop i laptop) i serverskih računara. Vodi porijeklo od Intel 8088 procesora, ali je glavne promjene doživjela uvođenjem 32-bitnog rada (Intel 80386 procesor) i 64-bitnog rada (AMD K8 mikroarhitektura u Opteron i Athlon 64 procesorima). Jedina preživjela CISC ISA, karakteristična po relativno velikoj gustini koda (potreban je manji broj instrukcija za rješavanje nekog problema nego upotrebom RISC pristupa), relativno visokoj potrošnji električne energije i relativno slaboj podršci za ugrađene sisteme, naročito u SoC varijantama. Moderni procesori imaju naprednu podršku za hardverski potpomognutu virtuelizaciju, vektorske i SIMD instrukcije i druge napredne tehnologije (u zavisnosti od konkretne porodice procesora).
- **ARM** - izvorno skraćenica prema Advanced RISC Machines, očigledno RISC ISA. Predstavlja antitezu x86 arhitekturi, apsolutno dominantna u oblasti prenosnih uređaja, uključujući *smartphone*, *tablet*, *smartwatch* i srodne uređaje. Osnovna karakteristika je relativno niska potrošnja električne energije, mogućnost licenciranja ili kompletnih rješenja ili samo instrukcijskog skupa što nudi veliku fleksibilnost u implementaciji, proširivost funkcionalnosti velikim skupom pomoćnih blokova, vrlo pogodna za upotrebu u ugrađenim sistemima i SoC varijantama. Savremeni ARM procesori podržavaju i rad u 64-bitnom režimu, kao i podršku za hardverski potpomognutu virtuelizaciju. Postoji široka lepeza proizvođača, od vrlo jednostavnih i jeftinih sa izuzetno skromnim performansama i minimalnom potrošnjom električne energije, do 64-bitnih rješenja pogodnih za upotrebu i u serverskom okruženju (što je još uvijek relativno rijetko u praksi).
- **AVR** - ISA razvijena od strane kompanije Atmel, namijenjena za upotrebu u pretežno mikrokontrolerskim okruženjima, koristi Harvard arhitekturu. Riječ je o 8-bitnoj arhitekturi na osnovu čega se može i razumjeti da nije direktna konkurencija x86 ili ARM arhitekturama. Iako postoji 32-bitna AVR32 ISA, ona je potpuno nezavisna od AVR ISA, ne postoji kompatibilnost, a i sama ISA polako odlazi u istoriju jer je gotovo u potpunosti potisnuta od strane ARM arhitekture.
- **MIPS** - RISC arhitektura čiji naziv predstavlja skraćenicu od Microprocessor without Interlocked Pipelined Stages. Riječ je o **djelomično otvorenoj** arhitekturi, što najčešće nije dovoljno dobro rješenje jer su otvoreni dijelovi vrlo stari imaju ograničenu praktičnu upotrebnu vrijednost. Podržava i 32-bitni i 64-bitni rad, SIMD instrukcije, itd. Vjerovatno najveći značaj leži u činjenici da je sama arhitektura izuzetno čest primjer za analizu arhitekture procesora, vrlo je popularna u akademskim krugovima i uticala je na razvoj drugih arhitekture.
- **RISC-V** - **potpuno otvorena** (open source) arhitektura RISC tipa, besplatna za eksploataciju i pogodna za proširenje namjenskim procesorskim blokovima. Podržava rad u 32-bitnom, 64-bitnom i 128-bitnom (ne baš još, ali teoretska podrška postoji) režimu. Osnovni cilj je olakšavanje razvoja novih procesora bez komercijalnih opterećenja koja taj

proces tradicionalno nosi sa sobom. Izuzetno popularna u akademskim krugovima, izuzetno je fleksibilna (najjednostavniji cjelobrojni model je dovoljno jednostavan za studentski rad, najkompleksniji pristupi ciljaju superračunare). Kako koristi *permissive* licencu, moguća je kombinacija i proširenje i zatvorenim komponentama, a trenutno aktivni proizvodi se kreću od vrlo jednostavnih, gotovo mikrokontrolerskih, procesora sa zaista impresivno niskom potrošnjom, do višejezgrenih naprednih procesora poput Alibaba Xuantie 910 (16 64-bitnih jezgara, 12-stepeni pipeline, *out-of-order* izvršavanje, blok za neuronske mreže, vektorske instrukcije, itd). Kako je riječ o otvorenoj arhitekturi koja rješava najveći inicijalni problem u projektovanju procesora, primjetno je da su proizvođači procesora često poslovni entiteti koje se ne bave primarno procesorima, ali su sada u prilici da dizajniraju procesor koji će u potpunosti odgovarati njihovim potrebama. Potpuno otvorena arhitektura omogućava i veći nivo povjerenja tako da se sve više koristi u osjetljivim primjenama u kojima je upotreba zatvorenih arhitektura tradicionalno problematična (vojna i obavještajna industrija). Očekivana buduća primjena je u oblasti telekomunikacija i softverski definisanog radija jer predstavlja prirodni komplement njihovim potrebama. Sistemi zasnovani na RISC-V procesorima obično izvršavaju neku od verzija operativnih sistema zasnovanih na Linux kernelu, makar kada je riječ o naprednijim procesorima.

40. Intel procesori

Iako je Intel vjerovatno najvažnija kompanija u istoriji razvoja računarskih procesora i ima dominantnu ulogu na tržištu procesora namijenjenih upotrebi u ličnim računarima i serverima, sva nastojanja da zauzme bitnu poziciju i na tržištu ugrađenih i procesora za mobilne uređaje su imala vrlo malo efekta i na kraju su propala (porodice procesora Atom i Quark). Danas Intel proizvodi gotovo sve procesore u tri osnovne porodice:

- **Pentium** - Najjednostavniji modeli, najskromnijih performansi sa najmanjim brojem procesorskih jezgara (1 ili 2), namijenjeni pretežno za upotrebu u uređajima niske cijene ili u uređajima u kojima je potrebno obezbijediti minimalnu potrošnju električne energije uz zadržavanje kompatibilnosti sa x86 arhitekturom. Gotovo uvijek sadrže i integrisani grafički procesor skromnih mogućnosti, kao i posebne blokove namijenjene za rad sa multimedijalnim zapisima što ih čini pogodnim za upotrebu u multimedijalnim uređajima, ali ne i za upotrebu u računarima za igranje. Današnji Pentium procesori odgovaraju Celeron (i donekle Atom) procesorima iz prethodnih generacija.
- **Core** - Osnovna porodica procesora, namijenjena širokoj paleti korisnika, nudi veliki raspon mogućnosti, performansi i cijena. Obično se proizvode u dvije verzije, jedna za desktop računare i druga za prenosne računare, pri čemu je druge verzija obično ograničena po pitanju termalnog budžeta i potrošnje električne energije (kao i broja memorijskih kanala, PCIe linija, itd). Kako je broj procesora zaista velik, a ovi procesori su vjerovatno najvažniji u ponudi Intela, ova porodica je podijeljena u četiri linije:
 - **Core i3** - Osnovni modeli procesora sa najčešće dva ili četiri jezgra, mogu uključivati podršku za HyperThreading (HT). Podržavaju umjerene količine radne memorije i sadrže integrisanu grafičku karticu dovoljnu za multimedijalne potrebe i nezahtjevnije igrice.
 - **Core i5** - Srednji model u ponudi, obično dovoljno fleksibilan i za zahtjevnije radne zadatke. Posjeduju od 2 do 6 jezgara, mogu podržavati i HT. Neki od procesora podržavaju i napredne tehnike virtuelizacije (Intel vPro), kao i veće brzine rada, podrška za više memorijskih kanala (zavisi od konkretne generacije i modela), veća količina keš memorije, nešto brža integrisana grafika, itd. Koriste se i sa integrisanim i diskretnim grafičkim karticama u zavisnosti od potrebe.
 - **Core i7** - Nekada najnapredniji model, danas gotovo najnapredniji. Procesori sadrže do osam jezgara, podržavaju HT, imaju više keš memorije, podršku za sve bržu memoriju, još viši maksimalni radni takt, itd. Tipično se koriste sa diskretnim grafičkim karticama.
 - **Core i9** - Današnji vrhunski desktop procesori, uključuju i X seriju procesora sa do 18 jezgara. Namijenjeni za najzahtjevnije zadatke, sadrže najbolje i najbrže tehnologije koje Intel može da ponudi. Dominantno se koriste u (najskupljim) računarima za igre i grafičkim radnim stanicama. Potrošnja električne energije ne predstavlja problem tako da su često u upotrebi i namjenska rješenja za hlađenje. Gotovo uvijek se koriste sa diskretnim grafičkim karticama.

- **Xeon** - Porodica procesora namijenjena za rad u serverima i vrhunskim radnim stanicama. Podržavaju rad sa ECC memorijom, za razliku od svih prethodno nabrojanih procesora. Kako je i ovdje riječ o velikom broju različitih procesora, postoje linije:
 - **Xeon W** - Xeon procesori namijenjeni za rad u radnim stanicama. Praktično predstavljaju vrhunske desktop procesore prilagođene potrebama poslovnog okruženja, prije svega po pitanju virtuelizacije, sigurnosti i performansi memorije. Podržavaju samo jedan procesor u sistemu, dok najnaprednija verzija ima 18 fizičkih jezgara.
 - **Xeon E** - Osnovna linija procesora za servere, podržava dvoprocesorski rad, ECC memoriju, veći broj PCIe linija što omogućava upotrebu kartica visokih performansi koji nisu samo grafičke kartice (višeportne mrežne 10G ili 40G kartice, InfiniBand, itd). Određeni modeli podržavaju i integrisanu grafičku karticu jer nam često u serverima nisu bitne grafičke performanse.
 - **Xeon Scalable** - Linija procesora koja omogućava upotrebu više od dva procesora u jednom serveru. Podržavaju do 28 jezgara, HT, ECC, osam (ili čak i više) procesora što znači da možete u jednom serveru koncentrisati nekoliko stotina jezgara.
 - **Xeon D** - Posebna linija procesora namijenjena radu u specifičnim uslovima (od -40°C) u SoC obliku. Koristi se u situacijama u kojima su potrebne vrhunske performanse u ugrađenim sistemima.

Svaka od porodica i linija procesora se još može podijeliti i prema generacijama, pri čemu svaka generacija donosi nove mogućnosti, obično prvo za skuplje porodice i linije, a kasnije i za druge procesore. Stvar komplikuje činjenica da granice između generacija različitih porodica nisu jasne i procesori za desktop, mobilno i serversko tržište iz iste "marketinške generacije" mogu pripadati različitim "stvarnim generacijama".

41. AMD procesori

AMD i Intel predstavljaju jedine kompanije koje danas proizvode procesore sa x86, odnosno x86-64, arhitekturom. Njihovo rivalstvo seže daleko u prošlost i tipično je Intel dominantan uz povremene proboje AMD-a kao što su bili prvi desktop procesor koji je probio granicu od 1 GHz, ali i prva uspješna implementacija x86-64 ISA.

Iako je Intel dugi niz godina važio za apsolutnog vladara x86 tržišta, njihova trenutna ponuda procesora ne predstavlja vrh ponude jer je u prethodnih par godina AMD na tržište izbacio procesore koji nude bolje performanse ili naprednije mogućnosti po nižim cijenama ili uz nižu potrošnju električne energije. Slično Intelu i AMD nudi procesore koji imaju integrisane grafičke procesore, ali za razliku od Intela, AMD integrisani GPU ima vrlo pristojne performanse, što ne iznenađuje imajući u vidu da je AMD jedan od dva najveća proizvođača diskretnih grafičkih kartica. AMD pravi razliku između CPU proizvoda bez integrisane grafike i onih sa integrisanom grafikom tako što one sa integrisanom grafikom naziva APU (*Accelerated Processing Unit*). Još jedna od specifičnosti AMD-a je u tome što relativno dugo zadržavaju isti *socket* za procesor što omogućava jednostavnu nadogradnju sistema izmjenom procesora novije generacije. Takođe, svi procesori iste generacije imaju gotovo identične mogućnosti i razlikuju se samo po broju jezgara, njihovom taktu, te veličini keša.

Slično Intelu, i AMD nudi široku lepezu proizvoda koji su podijeljeni su sljedeće porodice:

- **Athlon** - Najjednostavnija porodica procesora, sadrži dva jezgra i podršku za SMT (HyperThreading pod drugim imenom).
- **Ryzen** - Osnovna porodica procesora, postoji u desktop i mobilnim varijantama, zasnovana je na Zen arhitekturi u kojoj su procesorska jezgra grupisana u CCX module (*Core Complex*), a jedan procesor može imati više modula (pri čemu svaki modul ima i svoj keš, itd). Ova porodica je podijeljena u linije prema broju jezgara, veličini keš memorije, performansama procesora, ali i performansama integrisanog GPUa:
 - Ryzen 3 - 4 jezgra u jednom modulu, 4 MB L3 keša i integrisana Vega 6 grafika.
 - Ryzen 5 - 6 jezgara u jednom modulu, 32 MB L3 keša bez integrisanog GPU ili, za prethodnu generaciju, 2x3 jezgra i 8 MB keša i integrisana Vega 7 grafika.
 - Ryzen 7 - 8 jezgara u jednom modulu, 32 MB L3 keša bez integrisanog GPU ili, za prethodnu generaciju, 2x4 jezgra i 8 MB keša i integrisana Vega 8 grafika.
 - Ryzen 9 - 2x6 ili 2x8 jezgara, 64 MB L3 keša bez integrisane grafike.
- **ThreadRipper** - Porodica koja se može posmatrati i kao podvrsta Zen porodice jer koristi iste gradivne jedinice, ali omogućuje veliki broj jezgara i niti po procesoru (otuda i ime). Trenutno najnapredniji model ima 64 jezgra i 128 niti (2x8x4 jezgra). Takođe podržava i veliki broj kanala prema memoriji (do osam).
- **EPYC** - Skraćenica od *Extreme Performance Yield Computing*. Predstavlja liniju procesora za servere, podržava dvoprocesorski rad, do 64 jezgra i 128 niti po procesoru, 128 PCIe 4.0 linija, osmokanalna veza prema memoriji, ali i povezivanje više *servera* u jedan virtuelni SMP sistem (ScaleMP Versatile SMP sa do 128 servera i 8192 jezgra i 16384 niti).

42. ARM procesori

ARM arhitektura je danas daleko najpopularnija RISC arhitektura, a s obzirom da je broj prenosnih i ugrađenih računarskih sistema mnogo veći od broja desktop, laptop i serverskih računara, možemo reći da je ARM arhitektura daleko najpopularnija arhitektura u upotrebi.

Iako je potekla od ideje da obezbijedi pristojne performanse u vrlo ograničenim uslovima (po pitanju resursa, termalnog budžeta i raspoložive električne energije), danas se koristi u izuzetno širokom rasponu upotreba, od vrlo jednostavnih ugrađenih rješenja (pametni časovnici), preko sveprisutnih pametnih mobilnih telefona, do superračunara. Bitna razlika leži i u činjenici da je ARM procesori gotovo uvijek koriste kao dio SoC rješenja koje implementira gotovo sve funkcionalnosti računarskog sistema u jednom čipu ili modulu (CPU, GPU, RAM, kontroleri, itd).

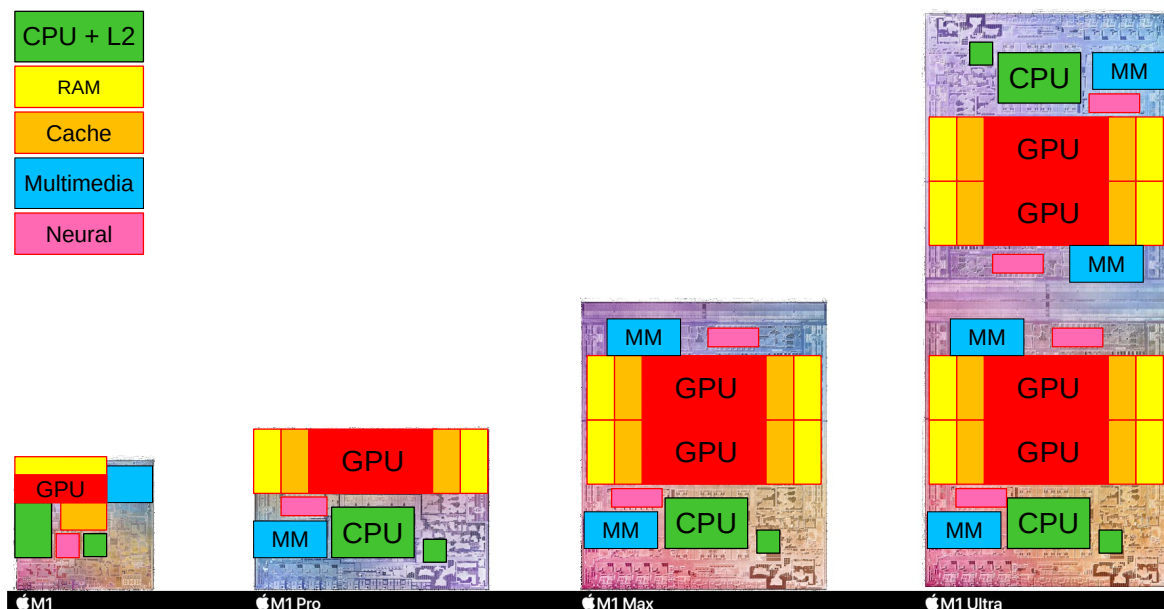
Za razliku od Intela i AMD-a, ARM ne proizvodi procesore nego licencira ISA ili gotovu implementaciju koju je moguće prilagoditi prema potrebama. U slučaju licenciranja ISA, moguće je funkcionalnost realizovati na proizvoljan način uz punu slobodu optimizacije (ovakvu licencu koriste, npr. Apple, Samsung, Qualcomm). Kako ovaj vid realizacije iziskuje značajna sredstva i napor, mnogo češći je slučaj licenciranja kompletne implementacije, ne samo procesorskog jezgra, nego i pratećih blokova koji predstavljaju GPU, memorijske kontrolere, PCIe kontrolere, namjenske procesore za obradu signala (DSP, ISP) ili rad sa neuronskim mrežama. Na ovaj način je moguće relativno brzo i lako razviti kompletan sistem, uz manju raznolikost u odnosu na konkurenciju (i dalje je moguće u sistem ugraditi posebne blokove koji realizuju za nas bitne funkcije).

Razvoj ARM arhitekture traje decenijama i pokriva brojne verzije koje više nisu aktuelne. Postoje dvije osnovne arhitekture u upotrebi, ARMv7 i ARMv8 (uz podverzije) i veći broj porodica:

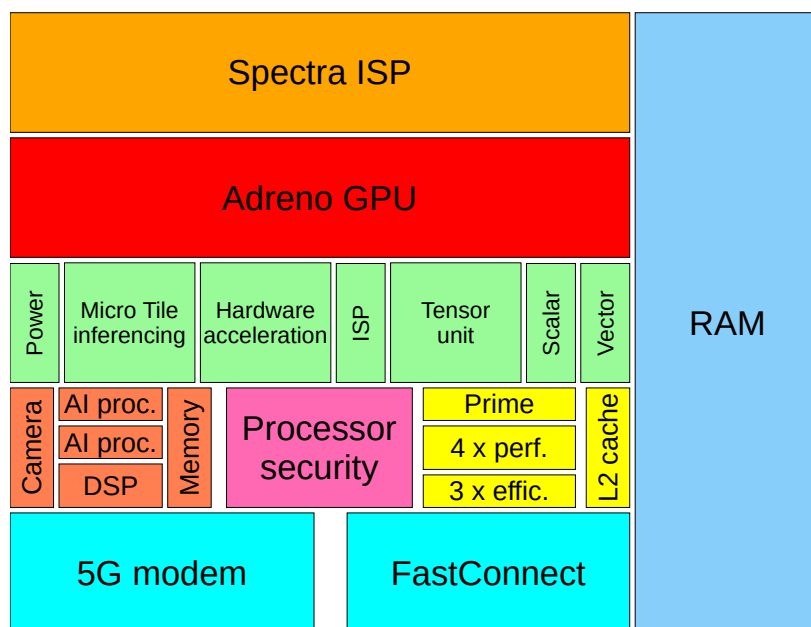
- Cortex-M - za upotrebu u mikrokontrolerskim okruženjima.
- Cortex-R - namijenjen za upotrebu u sistemima za rad u realnom vremenu.
- Cortex-A - osnovna porodica zastupljena u prenosnim uređajima, podržava 32bitni (ARMv7) i 64-bitni rad (ARMv8 i ARMv9) pri čemu najnovije i najnaprednije verzije podržavaju do 4 SMP jezgra, naprednu podršku za SIMD, hardversku virtuelizaciju, superskalarno izvršavanje (6 x fetch, 12 x issue), 11/10-stepeni pipeline, napredno *out-of-order* izvršavanje, velike količine keša, itd.
- Snapdragon - razvijen od strane firme Qualcomm i dominantan u vrhunskim Android uređajima (npr. Snapdragon 888 i Snapdragon 8 gen 1).
- Exynos - razvijen od strane firme Samsung i u upotrebi u nekim njihovim uređajima kao CPU vrhunskih performansi, ali i u smartwatch uređajima kao energetske vrlo efikasan procesor (prilagođen za tu namjenu).
- Axx - razvijen od strane firme Apple, predstavlja osnovu za sve trenutne prenosne uređaje, ali i za sve buduće Apple uređaje, uključujući laptop i desktop tržište. Apple M1 Max predstavlja trenutno najnapredniji procesor i omogućava bolje performanse za Apple uređaje nego prethodna generacija zasnovana na Intel procesorima.

Specifičnost ARM sistema je i u činjenici da broj jezgara ne mora biti paran, da sama jezgra ne moraju biti ista (za razliku od klasičnih Intel i AMD procesora iako se i tu situacija mijenja), pa čak ne moraju biti ni iste generacije ili podržavati istu dužinu procesorske riječi. Npr. Snapdragon 8

gen 1 i Mediatek Dimensity 9000 procesori sadrže 1 brzo jezgro (Cortex-X2 koje radi podržava samo 64b i out-of-order je), 3 srednja jezgra (Cortex-A710 koja podržavaju i 32b i 64b rad i out-of-order je) i 4 efikasna jezgra (Cortex-A510 koja podržavaju samo 64b, ali su in-order jezgra). Vrlo čest primjer je upotreba manjeg broja naprednijih jezgara na većim frekvencijama i većeg broja sporijih jezgara u istom sistemu jer na taj način možemo optimizovati upotrebu prema trenutnim potrebama.



Slika 46: Primjer porodice Apple M1



Slika 47: Primjer Snapdragon 8 Gen 2